





פרויקט גמר

עבודת גמר · סמל שאלון 711918

Compression & Decompression hardware system

מגיש	ישפה כהן
ת.ז	216958868
מנחה	אפרים שלום
מנחה בית ספרי	כהן יצחק
שנת לימודים	תשפ״ז

הצהרת התלמיד

שם התלמיד: ישפה כהן **מספר ת״ז: 216958868**

אני הח״מ, מצהיר בזאת כי פרויקט/עבודת הגמר וספר הפרויקט המצ״ב נעשו על ידי בלבד.

הפרויקט מסכם ידע, מיומנות והרגלים שלמדתי במסגרת לימודי ההתמחות במגמה ובאופן עצמאי.

הפרויקט וספר תיעוד הפרויקט נעשו על בסיס ההנחיות שקיבלתי מהמנחה שלי.

מקורות המידע בהם השתמשתי לביצוע פרויקט מצוינים ברשימת המקורות שבסוף הספר.

אני מודע לאחריות שהנני מקבל על עצמי על ידי חתימתי על הצהרה זו שכל הנכתב בה אמת.

חתימת התלמיד: _____ **תאריך:** _____

<u>אישור מנחה הפרויקט/עבודת הגמר</u>

הריני מאשר שהפרויקט בוצע בהנחייתי, קראתי את ספר הפרויקט ומצאתי כי הוא ראוי להגשה.

שם המנחה: <u>אפרים שלום</u> **חתימה:** _____ **תאריך:** _____

<u>אישור רכז המגמה</u>

הריני מאשר שדרישות הפרויקט ורמתו מתאימים לדרישות והנחיות משרד החינוך המפורסמים בחוזר המפמ״ר ובאתר המגמה.

שם רכז המגמה: <u>שוהם כהן</u> **חתימה:** _____ **תאריך:** _____

תוכן עניינים

הצהרת התלמיד 2

תוכן עניינים 3

רשימת איורים וטבלאות 5

תודות 7

תיאור נושא הפרויקט 8

תיאור הבעיה והצורך 10

הצעת הפרויקט המקורית 12

מחקר על נושאי הפרויקט 16

תרשים מלבנים 17

הסבר תרשים מלבנים 18

תרשים חשמלי: חיבור המערכת 20

הסבר התרשים החשמלי 21

חומרה 23

כרטיס הפיתוח DE10-Nano 24

הבקר Cyclone V SoC 27

כלי הפיתוח: Quartus, ModelSim ו Verilog 29

כרטיס PHY מסוג KSZ8061 ומחבר RJ45 30

מייצב מתח PTR08060W 33

פרוטוקולים 36

MII ו RMII 37

Avalon MM 41

תקשורת ורשתות 47

מודל שבע השכבות OSI Model 48

PHY – שכבה פיזית 51

מחבר RJ45 52

פרוטוקול Ethernet ומבנה הפרוטוקול 53

Media Access Control – MAC 57

ARP 58

IPv4 62

TCP ו UDP 65

דחיסת נתונים 69

תקני דחיסה ברשת – RFC 3173 ו IPComp 70

אלגוריתם LZ77 74

אלגוריתם הופמן (Huffman) 77

תוכנה, קוד החומרה 83

טבלת הסבר קוד החומרה 84

מפרט טכני של כלל הרכיבים 85

רפלקציה 86

מילון מונחים 87

רשימה ביבליוגרפית/ רשימת מקורות 89

תיעוד הפרויקט 91

המערכת המורכבת 92

בניית מערכת הפעלה Embedded Linux ל DE10-Nano 93

סביבת הבדיקות ואימות המערכת 103

תיעוד השימוש ב AI 109

תיעוד העבודה בתוכנה 110

נספח א: תרשים חשמלי של מייצב המתח PTR08060W 111

נספח ב: תרשים חשמלי של כרטיס PHY מסוג KSZ8061 113

נספח ג: דפי נתונים 115

רשימת איורים וטבלאות

איורים

איור 1: דחיסה בתוכנה מול דחיסה בחומרה באמצע הרשת 11
איור 2: הצעת הפרויקט, עמוד 1 12
איור 3: הצעת הפרויקט, עמוד 2 13
איור 4: הצעת הפרויקט, עמוד 3 14
איור 5: הצעת הפרויקט, עמוד 4 15
איור 6: תרשים מלבנים 17
איור 7: חיבור המערכת 20
איור 8: כרטיס הפיתוח במערכת 25
איור 9: מתג ה MSEL 26
איור 10: זרימת המידע בתוך ה FPGA 28
איור 11: כרטיסי ה PHY במערכת 31
איור 12: הקצאת הפינים ב Quartus 31
איור 13: המייצב PTR08060W 33
איור 14: הרגליים של PTR08060W 35
איור 15: אזור אספקת המתח 35
איור 16: ממשק MII 37
איור 17: ממשק RMII 38
איור 18: אות CRS_DV 39
איור 19: Avalon MM 41
איור 20: HPS ו FPGA 43
איור 21: מבנה המערכת 46
איור 22: שבע השכבות 48
איור 23: אמצעי התקשורת בשכבה הפיזית 50
איור 24: מסלול השידור והקליטה 51
איור 25: מחבר RJ45 52
איור 26: מבנה מסגרת Ethernet 53
איור 27: כתובת MAC 54
איור 28: בדיקת FCS 56
איור 29: ARP Cache 58
איור 30: ARP Request ו ARP Reply 59
איור 31: מבנה הודעת ARP 59
איור 32: ARP ו Ethernet 61
איור 33: IPv4 Header 62
איור 34: עיטוף המידע 63
איור 35: Three Way Handshake 65
איור 36: TCP Header מול UDP Header 67
איור 37: חבילה לפני ואחרי IPComp 71
איור 38: דחיסה לעומת הצפנה 72
איור 39: חלון ההזזה של LZ77 74

איור 40: דוגמה ל LZ77 75
איור 41: עץ הופמן 79
איור 42: דוגמה לעץ הופמן 79
איור 43: פתיחת הדחיסה 81
איור 44: DEFLATE 82
איור 45: המערכת המורכבת 92
איור 46: סדר העלייה של המערכת 93
איור 47: קימפול צולב 94
איור 48: טעינת ה FPGA ב U-Boot 95
איור 49: ביטול הוספת גרסה אוטומטית 96
איור 50: הפעלת Overlay filesystem 97
איור 51: הפעלת CONFIGFS 97
איור 52: שני שלבי בניית ה RootFS 98
איור 53: המחיצות בכרטיס ה microSD 101
איור 54: המערכת עולה על הכרטיס 102
איור 55: סביבת הבדיקות 103
איור 56: מדידת יחס הדחיסה 104
איור 57: שלבי התקשורת ב FTP 105
איור 58: הלכידה ב Wireshark 106
איור 59: תוכן חבילה 35 107
איור 60: מייצב מתח PTR08060W 111
איור 61: כרטיס PHY מסוג KSZ8061 113

טבלאות

טבלה 1: מאפיינים עיקריים 24
טבלה 2: מאפיינים עיקריים 30
טבלה 3: חיבור כרטיסי ה PHY ל FPGA 32
טבלה 4: חיבור כרטיסי ה PHY ל FPGA 32
טבלה 5: מאפיינים עיקריים 33
טבלה 6: הרכיבים הנלווים 34
טבלה 7: מפרט טכני של כלל הרכיבים 85
טבלה 8: מילון מונחים 87
טבלה 9: קבצי המערכת לכרטיס ה SD 100
טבלה 10: תוכנית הבדיקות 104
טבלה 11: שלבי התקשורת 106

תודות

פרק זה יושלם בהגשה הסופית

תיאור נושא הפרויקט

בפרויקט הגמר שלי אני מתכנן ובונה מערכת חומרה לדחיסה ופענוח של חבילות קבצים ברשת תקשורת. המערכת יושבת בין השרת לבין המשתמש, מקבלת את חבילות המידע שעוברות ברשת, דוחסת אותן ומעבירה אותן הלאה בזמן אמת.

בחיל האוויר, כמו בכל רשת תקשורת מודרנית, יש צורך להעביר הרבה מידע וקבצים בצורה מהירה ויעילה. הפרויקט הזה מציג מערכת מחשב שיודעת לדחוס Compress ו Decompress חבילות קבצים העוברות באינטרנט ממש בזמן אמת.

המערכת מבוססת על תקן שנקרא RFC 3173 ומשתמשת באלגוריתמים שהולכים יד ביד Huffman ו LZ77 כדי לדחוס קבצים בצורה חכמה, אמינה ויעילה ועונה על צורכי חיל האוויר.

ייחודיות הפרויקט באה לידי ביטוי בשילוב בין תוכנה לחומרה: המערכת מקשרת בין שני כרטיסי PHY, סביבת Linux, והמערכת כמכלול יושבת על ערכת הפיתוח DE10-Nano, שהייחודיות שלה היא השילוב בין HPS ל FPGA.

המערכת יושבת באמצע הרשת כנקודת מעבר, מעבדת את חבילות הקבצים ודוחסת אותן ישירות ברמת החומרה. הפתרון הזה חוסך ומונע המון זמן עיבוד מהמעבד הראשי.

בנוסף לכך, המערכת משלבת כמה רכיבים. הבסיס הוא כרטיס הפיתוח DE10-Nano, שעליו נמצאים מעבד HPS ולוגיקה מתכנתת FPGA. בכל צד של המערכת מחובר כרטיס PHY מסוג KSZ8061 עם מחבר RJ45, שמחבר אותה לכבלי ה Ethernet, את המתח לשני כרטיסי ה PHY מספק מייצב המתח PTR08060W, וכרטיס הפיתוח מקבל מתח ממתאם משלו.

על ה HPS רצה סביבת Embedded Linux שבניתי בעצמי, והיא מנהלת את המידע שעובר ברשת. את הדחיסה עצמה עושים האלגוריתמים Huffman ו LZ77, שעובדים יחד בשיטת DEFLATE, ודרך ממשק בקרה ייעודי אפשר לנטר את החבילות ואת תהליך הדחיסה.

תפקיד המערכת הוא לנהל, לדחוס ולהעביר קבצים במהירות גבוהה, תוך הקלה על העומס ברשתות חיל האוויר. היא בעצם מאפשרת לנו מעקב ובקרה בזמן אמת על כל חבילות המידע והקבצים שרצים ברשתות התקשורת של החיל.

תפקיד הפרויקט

מטרת הפרויקט היא פיתוח מערכת להאצת תעבורת רשת באמצעות דחיסה ופענוח של חבילות קבצים בזמן אמת, תוך שימוש בתקן RFC 3173. ליבת המערכת היא פיתוח מאיץ חומרתי על גבי פלטפורמת SoC המשלבת FPGA & HPS, המבצע את עיבוד הנתונים באופן עצמי על גבי כרטיס הפיתוח עצמו.

המערכת משלבת ניהול תוכנה על גבי Embedded Linux לניהול המידע העובר ברשת, יחד עם עיבוד חומרתי מהיר המבוצע בלוח הפיתוח DE10-Nano. המערכת כוללת ממשקי PHY (RJ45) לחיבור לרשת וממשק בקרה ייעודי לניטור החבילות והדחיסה.

שילוב זה מאפשר הפחתה משמעותית בעומס על משאבי העיבוד החיצוניים, ומספק פתרון אמין, יציב, יעיל וניתן להתאמה לניהול תעבורת רשת פנימית ברשתות החיל.

תיאור הבעיה והצורך

הבעיה הקיימת

בחיל האוויר יש צורך ברשתות תקשורת מודרניות המנוהלות על ידי גורם אמין ואבטחת מידע. כיום רשתות אלו נדרשות להתמודד עם היקפי תעבורה נרחבים המחייבים העברה יעילה ומהירה של נתונים, אך הן לא עונות על צורכי החיל. פרויקט זה נועד לממש פתרון להעברת חבילות קבצים אשר יוביל לניצול אופטימלי של רשת התקשורת, וימנע השהיות מיותרות, ובמקרים של עומס, אף אובדן נתונים.

כיום קיימים פתרונות תוכנתיים אך שימוש בהם פוגע בקצב התקשורת הכולל ולא מספק מענה לתשתיות עתירות עומסים, כמו מערכות המחשוב של חיל האוויר. בנוסף, הם דורשים כוח עיבוד משמעותי מהמעבד הראשי של המחשב.

כתוצאה מכך, נדרש פתרון חומרתי המאפשר דחיסה ופענוח מהירים ובלתי תלויים במשאבי מחשבי הקצה.

כיצד הפרויקט שלי פותר את הבעיה

המערכת שלי יושבת באמצע הרשת כנקודת מעבר, והדחיסה מתבצעת ב FPGA שעל כרטיס הפיתוח DE10-Nano. כך הדחיסה לא תלויה במעבד של מחשבי הקצה, ולא צריך להוסיף חומרה לכל מחשב ברשת.

בנוסף, הדחיסה מתבצעת בחומרה ולא בתוכנה, ולכן היא לא פוגעת בקצב התקשורת ומתאימה גם לרשתות עם היקפי תעבורה גדולים, כמו הרשתות של חיל האוויר.

מה קיים היום בשוק

לפני שתכננתי את המערכת בדקתי איך מתמודדים היום עם דחיסת מידע ברשת. מצאתי שני סוגים עיקריים של פתרונות, ובדקתי במה המערכת שלי שונה מהם.

דחיסה בתוכנה במערכת ההפעלה

במערכת ההפעלה Linux יש מימוש של IPComp בתוך ה kernel. הדחיסה והפתיחה מתבצעות דרך ספריית הדחיסה של מערכת ההפעלה, כלומר על המעבד הראשי של

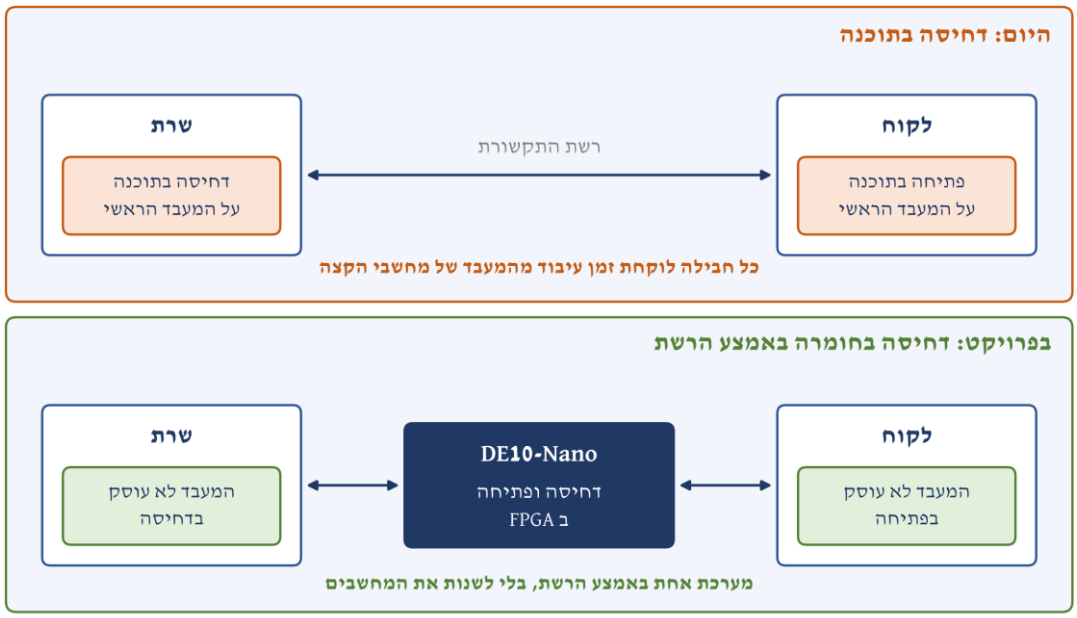
המחשב. כל חבילה שנדחסת לוקחת זמן עיבוד מהמעבד, וכשיש הרבה תעבורה העומס על המעבד גדל.

כרטיס האצה ייעודי

חברת Intel מציעה טכנולוגיה בשם QuickAssist, שמעבירה את הדחיסה מהמעבד לרכיב חומרה ייעודי שנמצא בתוך השרת. הרכיב תומך בשיטות דחיסה כמו DEFLATE, ובכך מוריד את העבודה מהמעבד. החיסרון הוא שהמאיץ צריך להיות בתוך כל שרת שרוצים שידחוס מידע.

מה הפרויקט מציע במקום

במקום לדחוס את המידע בתוכנה על כל מחשב, המערכת בפרויקט יושבת באמצע הרשת, בין השרת ללקוח. כל החבילות עוברות דרכה, והדחיסה והפתיחה מתבצעות ב FPGA, בחומרה. המעבד של מחשבי הקצה לא עושה את העבודה הזו, ואין צורך להוסיף רכיב לכל מחשב או שרת ברשת.



איור 1: דחיסה בתוכנה מול דחיסה בחומרה באמצע הרשת

הצעת הפרויקט המקורית

אלה העמודים של הצעת הפרויקט כפי שהוגשה ואושרה בתחילת הדרך:

הצעת פרויקט גמר

Compression & Decompression

hardware system

פרטי הפרויקט

סמל שאלון	גוף מבצע
711918	צבא
שנת לימודים: תשפ"ז	

פרטים אישיים וצוות פדגוגי

סגל פדגוגי	שם המגיש
מנחה: אפרים שלום **רכז.ת מגמה:** שוהם כהן	**שם:** ישפה כהן **ת.ז:** 216958868

איור 2: הצעת הפרויקט, עמוד 1



תקציר

בחיל האוויר, כמו בכל רשת תקשורת מודרנית, יש צורך להעביר הרבה מידע וקבצים בצורה מהירה ויעילה . הפרויקט הזה מציג מערכת מחשב שיודעת לדחוס Compress ו Decompress חבילות קבצים העוברות באינטרנט ממש בזמן אמת. המערכת מבוססת על תקן שנקרא RFC 3173 ומשתמשת באלגוריתמים שהולכים יד ביד Huffman ו lz77 כדי לדחוס קבצים בצורה חכמה, אמינה ויעילה ועונה על צורכי חיל האוויר.

ייחודיות הפרויקט באה לידי ביטוי בשילוב בין תוכנה לחומרה: המערכת מקשרת בין שני כרטיסי PHY, סביבת Linux , והמערכת כמכלול יושבת על כרטיס פיתוח והייחודיות שלו היא השילוב בין HPS ל FPGA ערכת הפיתוח DE10-NANO.

המערכת יושבת באמצע הרשת כנקודת מעבר, מעבדת את חבילות הקבצים ודוחסת אותן ישירות ברמת החומרה. הפתרון הזה חוסך ומונע המון זמן עיבוד מהמעבד הראשי,

תפקיד המערכת הוא לנהל, לדחוס ולהעביר קבצים במהירות גבוהה, תוך שמירה על רשתות חיל האוויר מאיומי סייבר. היא בעצם מאפשרת לנו מעקב ובקרה בזמן אמת על כל חבילות המידע והקבצים שרצים ברשתות התקשורת של החיל

איור 3: הצעת הפרויקט, עמוד 2



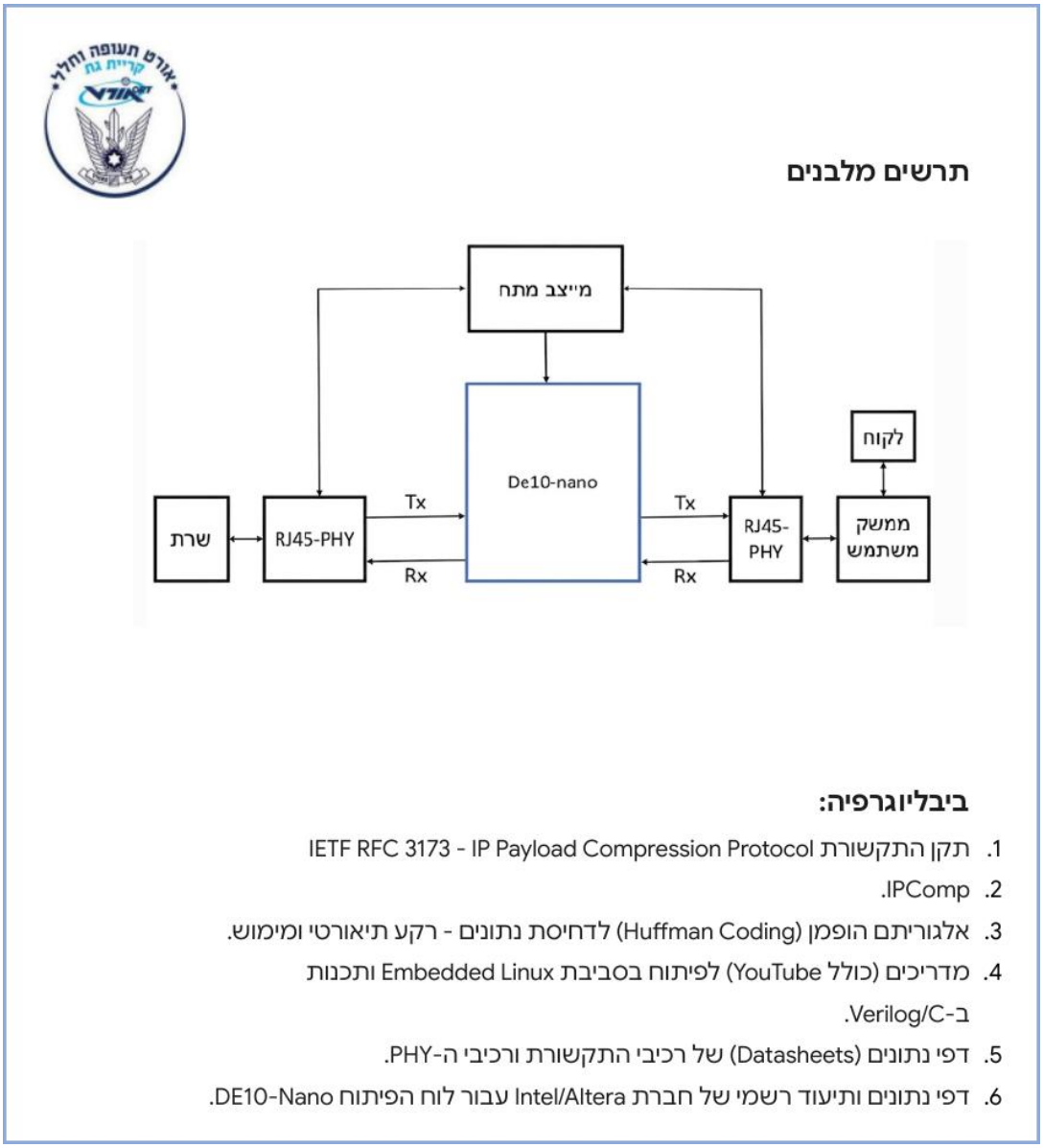
תיאור הבעיה או הצורך

בחיל האוויר יש צורך ברשתות תקשורת מודרניות המנוהלות על ידי גורם אמין ואבטחת מידע. כיום רשתות אלו נדרשות להתמודד עם היקפי תעבורה נרחבים המחייבים העברה יעילה ומהירה של נתונים אך לא עונה על צורכי החיל. פרויקט זה נועד לממש פתרון להעברת חבילות קבתים אשר יובילו לניצול אופטימלי של רשת התקשורת, להשהיות מיותרות, לסכנות סייבר ובמקרים של עומס, אף לאובדן נתונים. אך הם דורשים כוח עיבוד משמעותי מהמעבד הראשי של המחשב.
כיום קיימים פתרונות תוכנתיים אך שימוש בהם פוגע בקצב התקשורת הכולל ולא מספק מענה לתשתיות עתירות עומסים, כמו מערכות המחשוב של חיל האוויר.
כתוצאה מכך, נדרש פתרון חומרתי המאפשר דחיסה ופענוח מהירה ובלתי תלויה במשאבי מחשבי הקצה

תפקיד הפרויקט

מטרת הפרויקט היא פיתוח מערכת להאצת תעבורת רשת באמצעות דחיסה ופענוח של חבילות קבצים בזמן אמת, תוך שימוש בתקן RFC 3173. ליבת המערכת היא פיתוח מאיץ חומרתי על גבי פלטפורמת SOC המשלבת FPGA & HPS , המבצע את עיבוד הנתונים באופן עצמי על גבי כרטיס הפיתוח עצמו. המערכת משלבת ניהול תוכנה על גבי Embedded Linux לניהול המידע העובר ברשת, יחד עם עיבוד חומרתי מהיר המבוצע בלוח הפיתוח DE10-Nano. המערכת כוללת ממשקי PHY (RJ45) לחיבור לרשת וממשק בקרה ייעודי לניטור החבילות והדחיסה. שילוב זה מאפשר הפחתה משמעותית בעומס על משאבי העיבוד החיצוניים, ומספק פתרון אמין, יציב, יעיל וניתן להתאמה לניהול תעבורת רשת פנימית ברשתות החיל.

איור 4: הצעת הפרויקט, עמוד 3



איור 5: הצעת הפרויקט, עמוד 4

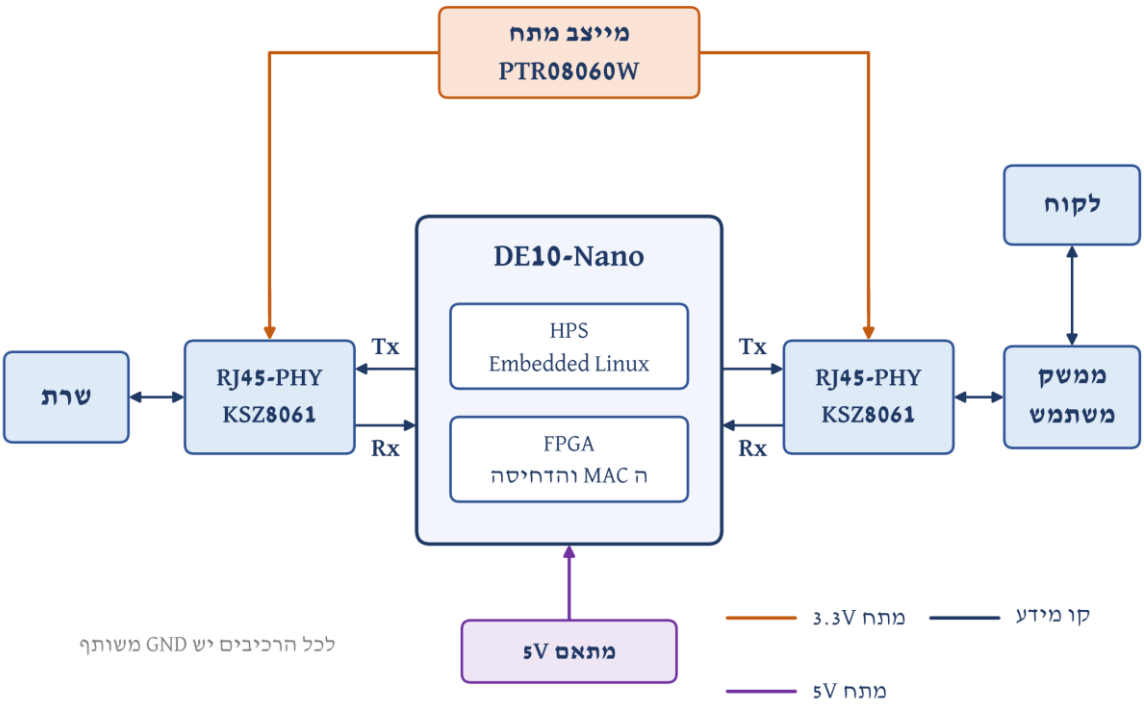
מחקר על נושאי הפרויקט

כדי לבנות את המערכת חקרתי את הנושאים שהפרויקט נשען עליהם, מהשכבה הפיזית של הרשת ועד אלגוריתמי הדחיסה והממשקים שבתוך ה FPGA.

נושא חקר:

- **MII ו RMII**
- **Avalon MM**
- **מודל שבע השכבות OSI Model**
- **PHY – שכבה פיזית**
- **מחבר RJ45**
- **פרוטוקול Ethernet ומבנה הפרוטוקול**
- **Media Access Control – MAC**
- **ARP**
- **IPv4**
- **TCP ו UDP**
- **תקני דחיסה ברשת – RFC 3173 ו IPComp**
- **אלגוריתם LZ77**
- **אלגוריתם הופמן (Huffman)**

תרשים מלבנים



איור 6: תרשים מלבנים

הסבר תרשים מלבנים

התרשים מציג את מבנה המערכת שאני בונה בפרויקט. המידע עובר בין השרת לבין המשתמש דרך כרטיס הפיתוח, ובדרך הוא נדחס או נפתח. להלן הסבר על כל אחד מהמרכיבים במערכת:

שרת

השרת הוא הצד ברשת שממנו מגיעות חבילות הקבצים ואליו הן נשלחות. הוא מחובר למערכת דרך כבל Ethernet.

כרטיסי RJ45-PHY

בכל צד של כרטיס הפיתוח נמצא כרטיס PHY מבוסס הרכיב KSZ8061 של חברת Microchip, עם מחבר RJ45. ה PHY ממיר את האותות החשמליים שמגיעים מהכבל למידע דיגיטלי, ובכיוון השני ממיר את המידע הדיגיטלי לאותות שעוברים בכבל.

החיבור בין כרטיס ה PHY לכרטיס הפיתוח עובר דרך ממשק RMII.

כרטיס הפיתוח DE10-Nano

כרטיס הפיתוח הוא היחידה המרכזית של המערכת. בתוכו נמצא ה HPS, שמריץ את סביבת ה Linux, ונמצא ה FPGA, שבו ממומש ה MAC ושבו מתבצעת הדחיסה והפענוח של חבילות הקבצים ברמת החומרה.

קווי Tx ו Rx

החיצים בתרשים מראים את כיוון המידע. Tx הוא הקו שבו כרטיס הפיתוח משדר מידע אל ה PHY, ו Rx הוא הקו שבו הוא מקבל מידע מה PHY.

בכל צד של הכרטיס יש גם שידור וגם קליטה, כך שהמידע יכול לעבור בשני הכיוונים.

ממשק משתמש

ממשק המשתמש מחובר לכרטיס ה PHY בצד השני של המערכת. דרכו הלקוח שולח ומקבל קבצים, ודרכו אפשר לעקוב אחרי החבילות ואחרי תהליך הדחיסה.

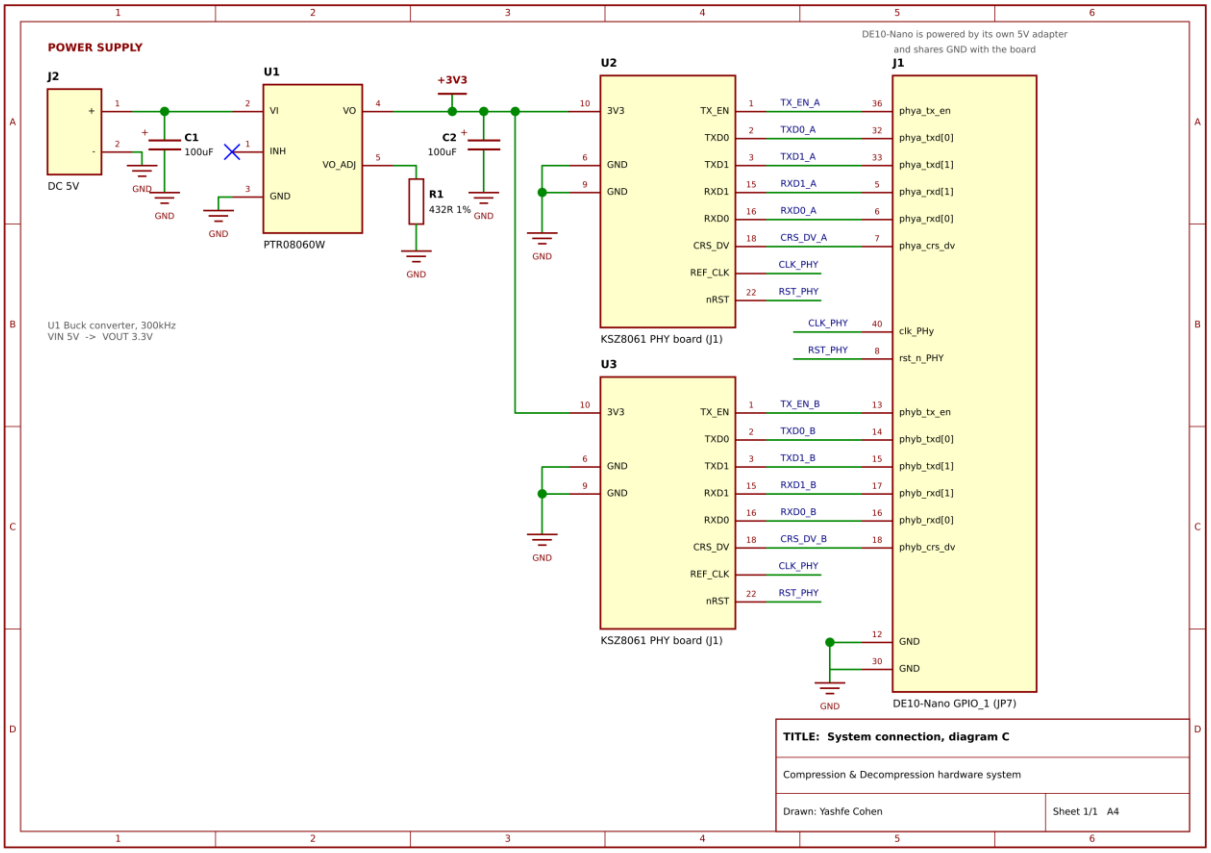
לקוח

הלקוח הוא המשתמש שעובד מול המערכת ושולח או מקבל את הקבצים.

מייצב מתח

מייצב המתח PTR08060W של חברת Texas Instruments מספק מתח יציב של 3.3V לשני כרטיסי ה PHY. כרטיס הפיתוח מקבל מתח של 5V ממתאם משלו, ולכל הרכיבים יש GND משותף.

תרשים חשמלי: חיבור המערכת



איור 7: חיבור המערכת

הסבר התרשים החשמלי

התרשים מציג את החיבורים בין שלושת החלקים של המערכת, מייצב המתח PTR08060W, שני כרטיסי ה PHY מסוג KSZ8061 וכרטיס הפיתוח DE10-Nano. את התרשים שרטטתי בתוכנת EasyEDA, עם סמלי הרכיבים, ערכי הרכיבים ושמות הקווים. בתרשים U1 הוא המייצב, U2 ו U3 הם שני כרטיסי ה PHY, ו J1 הוא מחבר GPIO_1 של כרטיס הפיתוח.

חלק מהחיבורים מסומנים בשם הקו, כמו CLK_PHY ו RST_PHY. כל הנקודות שמסומנות באותו שם מחוברות זו לזו.

אספקת המתח

מתח של 5V נכנס דרך השקע שעל הלוח לרגל 2, VI, של המייצב, ובכניסה מחובר הקבל C1 של 100uF. מרגל 4, VO, יוצא מתח מיוצב של 3.3V, ואחרי הקבל C2 של 100uF הוא מגיע לרגל 10 במחבר J1 של שני כרטיסי ה PHY.

הנגד R1, שהוא ה RSET של 432 אוהם ומחובר בין רגל 5 לבין ה GND, קובע שמתח היציאה של המייצב יהיה 3.3V. רגל 1, Inhibit, לא מחוברת, ולכן המייצב פועל כל הזמן.

כרטיס הפיתוח DE10-Nano

כרטיס הפיתוח לא מקבל מתח מהמייצב. הוא מחובר למתאם 5V משלו, ומהלוח הוא מקבל רק את ה GND המשותף.

אדמה משותפת

ה GND של לוח המטריצה מחובר לרגליים 12 ו 30 במחבר ה GPIO של כרטיס הפיתוח, וגם לרגליים 6 ו 9 בשני כרטיסי ה PHY. בלי אדמה משותפת אי אפשר להעביר אותות בין הכרטיסים, כי לכל כרטיס הייתה נקודת ייחוס אחרת למתח.

קווי השידור

הקווים TXD0, TXD1 ו TX_EN יוצאים מה FPGA אל רגליים 1 עד 3 במחבר J1 של כל כרטיס PHY. בכרטיס הראשון הם מחוברים לרגליים 32, 33 ו 36 במחבר GPIO_1, ובכרטיס השני לרגליים 13 עד 15.

קווי הקליטה

הקווים RXD1, RXD0 ו CRS_DV יוצאים מרגליים 15, 16 ו 18 של כרטיס ה PHY אל ה FPGA. בכרטיס הראשון הם מחוברים לרגליים 5 עד 7 במחבר GPIO_1, ובכרטיס השני לרגליים 16 עד 18.

שעון ואיפוס

שני הכרטיסים מקבלים את אותו שעון ייחוס של 50MHz, clk_PHy, שה FPGA מוציא מרגל 40 במחבר. באותה צורה קו האיפוס rst_n_PHY יוצא מרגל 8 ומגיע לרגל nRST של שני הכרטיסים, כך שה FPGA יכול לאפס את שני כרטיסי ה PHY בבת אחת.

חומרה

כרטיס הפיתוח DE10-Nano

הבקר Cyclone V SoC

כלי הפיתוח: Quartus, ModelSim ו Verilog

כרטיס PHY מסוג KSZ8061 ומחבר RJ45

מייצב מתח PTR08060W

כרטיס הפיתוח DE10-Nano

כרטיס הפיתוח DE10-Nano של חברת Terasic הוא הבסיס של המערכת. על הכרטיס נמצא שבב Cyclone V SoC של חברת Intel, שמשלב בתוך שבב אחד מעבד HPS ולוגיקה מתכנתת FPGA.

בפרויקט זה ה HPS מריץ את סביבת ה Embedded Linux ומנהל את המידע שעובר ברשת, וב FPGA ממומשים ה MAC ומאיץ החומרה שדוחס ופותח את חבילות הקבצים.

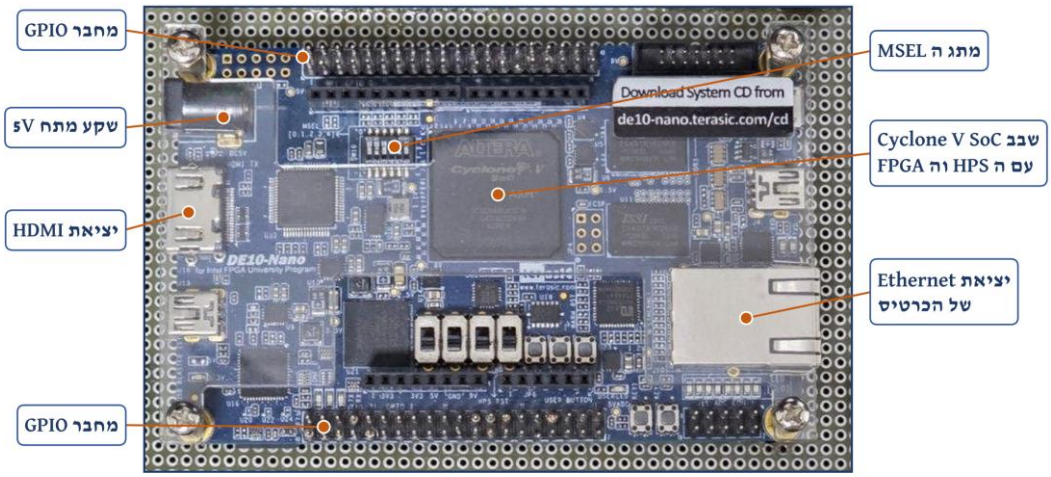
מאפיינים עיקריים

טבלה 1: מאפיינים עיקריים

מאפיין	פירוט
מעבד HPS	ARM Cortex-A9 עם שתי ליבות, בתדר של 800MHz
לוגיקה מתכנתת FPGA	110K תאים לוגיים ו 145 פינים לשימוש כללי
זיכרון	1GB זיכרון DDR3, וכרטיס microSD שעליו נמצאת מערכת ההפעלה
מחברי הרחבה	שני מחברי GPIO של 40 פינים ומחבר בתצורת Arduino Uno
תקשורת	יציאת Ethernet של 1Gbps, יציאת USB ויציאת HDMI
כפתורים ונורות	שני לחצנים, ארבעה מתגים ושמונה נורות LED שמחוברים ל FPGA
מתח הפעלה	5V DC

הכרטיס במערכת

בתמונה מסומנים החלקים של הכרטיס שחשובים לפרויקט. במרכז נמצא שבב ה Cyclone V SoC, ובחלק העליון ובחלק התחתון נמצאים מחברי ה GPIO, שדרכם הכרטיס מתחבר לכרטיסי ה PHY.



איור 8: כרטיס הפיתוח במערכת

מחברי ה GPIO

על הכרטיס יש שני מחברי GPIO של 40 פינים, GPIO_0 ו GPIO_1, שמחוברים ישירות לפינים של ה FPGA. בכל מחבר יש גם פינים של מתח והארקה, רגל 11 של 5V, רגל 29 של 3.3V ורגליים 12 ו 30 של GND.

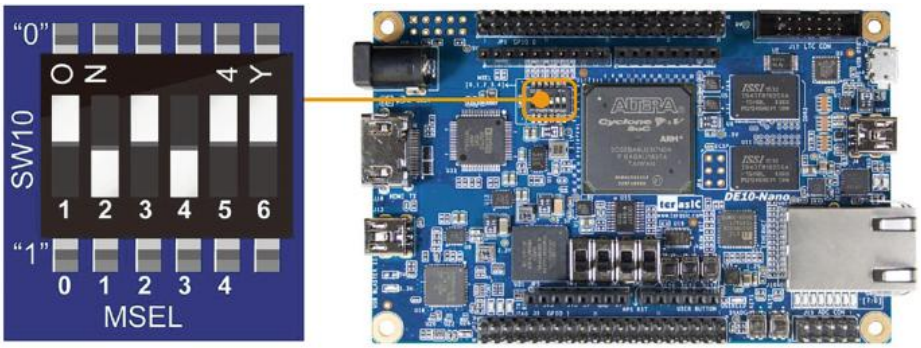
בפרויקט נעשה שימוש רק במחבר GPIO_1. דרכו עוברים כל קווי ה RMII של שני כרטיסי ה PHY, השעון של 50MHz וקו האיפוס, ודרך רגליים 12 ו 30 שלו עובר ה GND המשותף עם לוח המטריצה. את הרשימה המלאה של הפינים פירטתי בפרק על כרטיס ה PHY.

מתג ה MSEL

על הכרטיס נמצא מתג קטן עם שש ידיות לבנות, SW10. דרכו נקבע ה MSEL, כלומר את הדרך שבה ה FPGA מקבל את קובץ התכנון שלו כשהכרטיס נדלק.

המתג חשוב לאתחול של המערכת. בפרויקט זה U-Boot הוא זה שצורב את הקובץ soc_system.rbf אל ה FPGA בכל הדלקה, ולכן המתג צריך להיות במצב שמאפשר ל HPS לתכנת את ה FPGA. כשהמתג לא נמצא במצב הנכון, U-Boot לא מצליח לצרוב את התכנון וה FPGA לא עולה.

בתמונה רואים את המתג SW10 ואת המקום שלו על הכרטיס, ליד שבב ה Cyclone V:



איור 9: מתג ה MSEL

מחבר ה Arduino

על הכרטיס יש גם מחבר בתצורת Arduino Uno, שאליו אפשר לחבר כרטיסי הרחבה של Arduino. בפרויקט זה המחבר לא בשימוש, וכל החיבורים של המערכת עוברים דרך מחבר GPIO_1.

הבקר Cyclone V SoC

הבקר של המערכת הוא שבב ה Cyclone V SoC של חברת Intel, שנמצא במרכז כרטיס הפיתוח. המיוחד בשבב הזה הוא שיש בתוכו שני חלקים שונים, ה HPS וה FPGA, שעובדים יחד על אותה פיסת סיליקון.

HPS

ה HPS, Hard Processor System, הוא החלק בשבב שבו נמצא המעבד. הוא בנוי ממעבד ARM Cortex-A9 עם שתי ליבות בתדר של 800MHz, ומחובר לזיכרון DDR3 של 1GB שנמצא על הכרטיס.

בפרויקט זה ה HPS מריץ את מערכת ההפעלה Embedded Linux שנבנתה עבור הכרטיס. הוא עולה מכרטיס ה microSD דרך U-Boot, מנהל את המידע שעובר ברשת, ודרכו מתבצע החיבור למערכת בחיבור סריאלי וב SSH. יציאת ה Ethernet שעל הכרטיס שייכת ל HPS.

FPGA

ה FPGA הוא החלק בשבב שבו נמצאת הלוגיקה המתכנתת, עם 110K תאים לוגיים. את התכנון שלו כותבים בשפת Verilog ומקמפלים בתוכנת Quartus, ומתקבל קובץ תכנון שנטען אל ה FPGA.

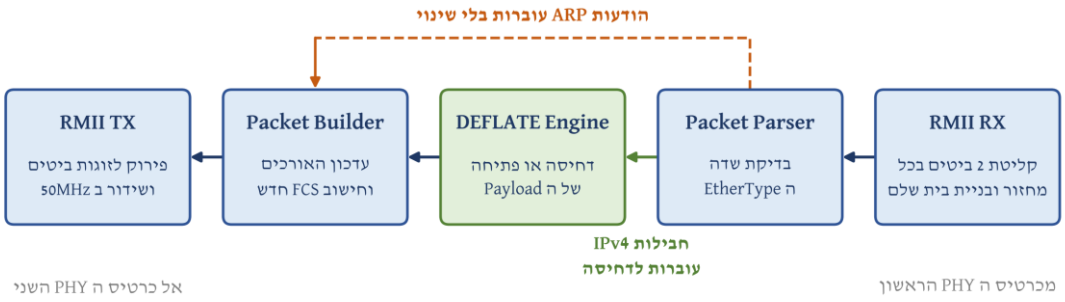
בפרויקט זה ממומשים ב FPGA ה MAC ומאיץ הדחיסה, ואליו מחוברים שני כרטיסי ה PHY דרך פיני מחבר GPIO_1.

השילוב בין HPS ל FPGA

היתרון של הכרטיס הוא שהמעבד והלוגיקה המתכנתת נמצאים על אותו שבב ומחוברים ביניהם בממשק AXI. כך התוכנה שרצה ב Linux יכולה לגשת ישירות לרכיבי החומרה שממומשים ב FPGA, כמו שאסביר בפרק על Avalon MM.

זרימת המידע בתוך ה FPGA

בתרשים המלבנים ה FPGA מופיע כמלבן אחד. לפי התכנון של הפרויקט, המידע עובר בתוכו בחמישה שלבים:



איור 10: זרימת המידע בתוך ה FPGA

ה RMII RX קולט מכרטיס ה PHY שני ביטים בכל מחזור של שעון 50MHz ואוסף אותם לבית שלם. אחריו מפענח המסגרות, Packet Parser, בודק את שדה ה EtherType. הודעת ARP, עם הערך 0x0806, עוברת הלאה בלי שינוי, וחבילת IPv4, עם הערך 0x0800, עוברת למאיץ הדחיסה.

מאיץ הדחיסה דוחס את ה Payload בשיטת DEFLATE, בעזרת LZ77 וקידוד הופמן, ובכיוון ההפוך הוא פותח אותו. אחר כך ה Packet Builder מעדכן את האורכים בחבילה ומחשב FCS חדש, וה RMII TX מפרק את הבתים בחזרה לזוגות של ביטים ומשדר אותם בקצב של 50MHz אל כרטיס ה PHY בצד השני.

כלי הפיתוח: Quartus, ModelSim ו Verilog

Quartus Prime

Quartus Prime היא תוכנת הפיתוח של חברת Intel עבור רכיבי ה FPGA שלה. בתוכנה כותבים את קוד החומרה, מקמפלים אותו ומקבלים קובץ שאפשר לצרוב ל FPGA.

בפרויקט השתמשתי בכמה חלקים של התוכנה. ב Platform Designer חיברתי את הרכיבים לגשרים שבין ה HPS ל FPGA, ב Pin Planner הגדרתי לאילו פינים של ה FPGA מחובר כל אות של כרטיסי ה PHY, ובסוף הקימפול התקבל קובץ התכנון שממנו נוצר soc_system.rbf, הקובץ ש U-Boot צורב בכל הדלקה.

ModelSim

ModelSim היא תוכנת סימולציה לקוד חומרה. לפני שצורבים את התכנון לכרטיס, אפשר להריץ אותו בסימולציה עם קובץ בדיקה, Testbench, שמכניס אותות לרכיב, ולראות בגרף הגלים איך כל אות מתנהג בכל מחזור שעון.

הסימולציה חוסכת הרבה זמן. כשמשהו לא עובד על הכרטיס קשה לדעת איפה הבעיה, ובסימולציה רואים בדיוק באיזה מחזור שעון ובאיזה אות היא מתחילה, למשל בזמנים של קווי ה RMII.

הבחירה ב Verilog

קוד החומרה בפרויקט כתוב בשפה Verilog. השפה הנוספת שנפוצה לתכנון FPGA היא VHDL, ושתיהן נתמכות ב Quartus וב ModelSim.

הבחירה ב Verilog נעשתה כי התחביר שלה פשוט וקצר יותר, והוא דומה לשפת C. ב VHDL צריך לכתוב הרבה יותר שורות כדי להגדיר את אותו רכיב, וההגדרות של הטיפוסים בה מחמירות יותר. בנוסף, Verilog נפוצה מאוד בתעשייה ויש עליה הרבה חומר ודוגמאות, וזה עזר בזמן הלמידה.

כרטיס PHY מסוג KSZ8061 ומחבר RJ45

כרטיס ה PHY הוא החלק במערכת שמחבר בין כרטיס הפיתוח לבין כבל הרשת. במערכת יש שני כרטיסים כאלה, אחד בכל צד, וכל אחד מהם בנוי מהרכיב KSZ8061 של חברת Microchip ומחבר RJ45.

הרכיב KSZ8061

ה KSZ8061 הוא רכיב PHY של Ethernet שעובד במהירויות של 10 ו 100Mbps ובמתח של 3.3V. הוא מקבל מה MAC את המידע הדיגיטלי דרך ממשק RMII, ממיר אותו לאותות חשמליים ומעביר אותם לכבל. בכיוון הקבלה הוא מבצע את הפעולה ההפוכה.

מאפיינים עיקריים

טבלה 2: מאפיינים עיקריים

מאפיין	פירוט
מהירות	10 או 100Mbps בפורט Ethernet אחד
ממשק ל MAC	RMII, שני ביטים בכל מחזור שעון ושעון ייחוס של 50MHz
ניהול	ממשק MDC ו MDIO לקריאה ולשינוי ההגדרות של הרכיב
Quiet-WIRE	מסנן שמקטין את הפליטה החשמלית מהכבל ומשפר את העמידות להפרעות
LinkMD+	מדידה של איכות האות שמתקבל ובדיקה של הכבל
נגדי סיום	מובנים בתוך הרכיב, ולא צריך להוסיף אותם על הכרטיס
מצב שינה	חיסכון באנרגיה, הרכיב מתעורר כשמגיע אות בכבל

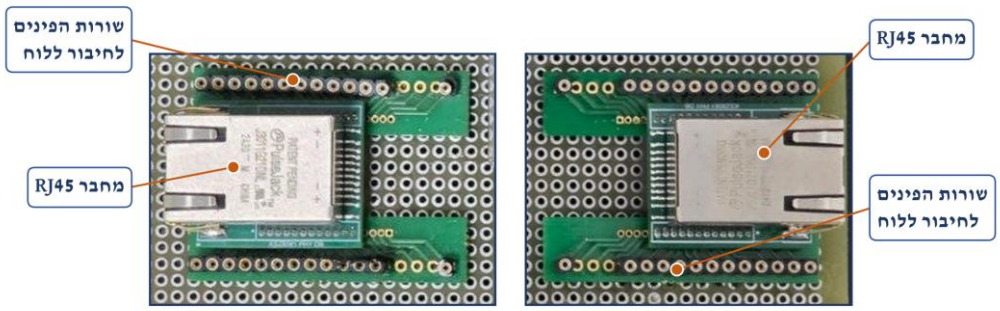
מחבר RJ45

מחבר ה RJ45 מחובר ישירות ל KSZ8061. הוא כולל שמונה מגעים, ודרכו עוברים האותות החשמליים שה PHY יוצר אל ארבעת זוגות החוטים השזורים בכבל.

כמו שאסביר בהמשך, המחבר לא מעבד את המידע. הוא משמש כחיבור הפיזי בין ה PHY לבין כבל הרשת.

הכרטיסים במערכת

אלה שני כרטיסי ה PHY שמחוברים במערכת. בכל כרטיס נמצא מחבר ה RJ45, ומעליו ומתחתיו שורות הפינים שדרכן הכרטיס מחובר ללוח ומשם לכרטיס הפיתוח.



איור 11: כרטיסי ה PHY במערכת

חיבור כרטיסי ה PHY ל FPGA

את הפינים של ה FPGA הגדרתי בתוכנת Quartus, בחלון ה Pin Planner. כל האותות עובדים בתקן 3.3-V LVTTL, ובשמות שלהם phya הוא כרטיס ה PHY הראשון, PHY A, ו phyb הוא השני, PHY B:

Named: * Edit: Filter: Pins: all

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	er Analog Setting:	GXB/VC
clk	Input	PIN_V11	3B	B3B_N0	PIN_V11	3.3-V LVTTL		16mA (default)				
clk_PHy	Output	PIN_AE17	4A	B4A_N0	PIN_AE17	3.3-V LVTTL		16mA (default)	1 (default)			
phya_rx_crs_dv	Input	PIN_AE25	5A	B5A_N0	PIN_AE25	3.3-V LVTTL		16mA (default)				
phya_rxd[1]	Input	PIN_AG28	4A	B4A_N0	PIN_AG28	3.3-V LVTTL		16mA (default)				
phya_rxd[0]	Input	PIN_AF28	4A	B4A_N0	PIN_AF28	3.3-V LVTTL		16mA (default)				
phya_tx_en	Output	PIN_AF20	4A	B4A_N0	PIN_AF20	3.3-V LVTTL		16mA (default)	1 (default)			
phya_txd[1]	Output	PIN_AG18	4A	B4A_N0	PIN_AG18	3.3-V LVTTL		16mA (default)	1 (default)			
phya_txd[0]	Output	PIN_AH19	4A	B4A_N0	PIN_AH19	3.3-V LVTTL		16mA (default)	1 (default)			
phyb_rx_crs_dv	Input	PIN_AF23	4A	B4A_N0	PIN_AF23	3.3-V LVTTL		16mA (default)				
phyb_rxd[1]	Input	PIN_AG23	4A	B4A_N0	PIN_AG23	3.3-V LVTTL		16mA (default)				
phyb_rxd[0]	Input	PIN_AF25	4A	B4A_N0	PIN_AF25	3.3-V LVTTL		16mA (default)				
phyb_tx_en	Output	PIN_AG25	4A	B4A_N0	PIN_AG25	3.3-V LVTTL		16mA (default)	1 (default)			
phyb_txd[1]	Output	PIN_AH24	4A	B4A_N0	PIN_AH24	3.3-V LVTTL		16mA (default)	1 (default)			
phyb_txd[0]	Output	PIN_AH26	4A	B4A_N0	PIN_AH26	3.3-V LVTTL		16mA (default)	1 (default)			
rst_n	Input	PIN_AH17	4A	B4A_N0	PIN_AH17	3.3-V LVTTL		16mA (default)				
rst_n_PHY	Output	PIN_AF27	4A	B4A_N0	PIN_AF27	3.3-V LVTTL		16mA (default)	1 (default)			
<<new node>>												

All Pins

איור 12: הקצאת הפינים ב Quartus

בטבלה מרוכזים קווי ה RMII של שני הכרטיסים, עם הפין ב FPGA שאליו מחובר כל קו בכל אחד מהכרטיסים.

טבלה 3: חיבור כרטיסי ה PHY ל FPGA

אות	פין ב FPGA, PHY A	פין ב FPGA, PHY B	כיוון
TXD0	PIN_AH19	PIN_AH26	מה FPGA אל ה PHY
TXD1	PIN_AG18	PIN_AH24	מה FPGA אל ה PHY
TX_EN	PIN_AF20	PIN_AG25	מה FPGA אל ה PHY
RXD0	PIN_AF28	PIN_AF25	מה PHY אל ה FPGA
RXD1	PIN_AG28	PIN_AG23	מה PHY אל ה FPGA
CRS_DV	PIN_AE25	PIN_AF23	מה PHY אל ה FPGA

בנוסף לקווי הנתונים יש ארבעה אותות משותפים:

טבלה 4: חיבור כרטיסי ה PHY ל FPGA

אות	פין ב FPGA	תפקיד	כיוון
clk	PIN_V11	שעון 50MHz של הכרטיס	נכנס ל FPGA
clk_PHy	PIN_AE17	שעון ייחוס ל PHY	יוצא מה FPGA
rst_n	PIN_AH17	איפוס הלוגיקה	נכנס ל FPGA
rst_n_PHY	PIN_AF27	איפוס כרטיסי ה PHY	יוצא מה FPGA

בפרויקט בחרתי לא להשתמש בקווים MDC, MDIO ו RXER. כרטיסי ה PHY עולים עם הגדרות ברירת המחדל שלהם, ואלה ההגדרות שאני צריך, כך שלא היה צורך לקרוא או לשנות את הרגיסטרים שלהם. לכן חיברתי ל FPGA רק את קווי הנתונים, את השעון ואת קו האיפוס.

מייצב מתח PTR08060W

מייצב המתח PTR08060W של חברת Texas Instruments הוא מודול אספקת מתח שמספק מתח של 3.3V לשני כרטיסי ה PHY. כרטיס הפיתוח DE10-Nano לא מקבל ממנו מתח, הוא מחובר למתאם 5V משלו וחולק עם שאר הרכיבים אדמה משותפת. המייצב בנוי כמודול שלם שכבר כולל בתוכו את הסליל ואת רכיבי ההספק, כך שצריך להוסיף לו רק קבל בכניסה, קבל ביציאה ונגד אחד שקובע את מתח היציאה.

מאפיינים עיקריים

טבלה 5: מאפיינים עיקריים

מאפיין	פירוט
מתח כניסה	4.5V עד 14V
מתח יציאה	0.6V עד 5.5V, לפי הנגד RSET
זרם יציאה	עד 6A
יעילות	עד 96%
תדר מיתוג	300kHz
הגנות	הגנה מזרם יתר עם חזרה אוטומטית לפעולה, והגנה ממתח כניסה נמוך מדי
טווח טמפרטורות	בין מינוס 40 ל 85 מעלות צלזיוס
רגל Inhibit	הדלקה וכיבוי של המודול

איך המייצב עובד



איור 13: המייצב PTR08060W

ה PTR08060W הוא ממיר ממותג יורד, Step-Down או Buck. הוא לא "שורף" את הפרש המתחים כמו מייצב ליניארי, אלא מדליק ומכבה את מתח הכניסה בקצב של

300kHz ומעביר אותו דרך סליל פנימי. הסליל והקבלים מחליקים את הפולסים, ובסוף יוצא מתח קבוע ונמוך יותר מהכניסה.

מכיוון שהמתג נמצא כל הזמן או במצב פתוח או במצב סגור, כמעט לא מתבזבזת בו אנרגיה. בזכות זה המייצב מגיע ליעילות של עד 96% ולא מתחמם, גם כשהוא מוריד את המתח מ 5V ל 3.3V.

הרכיבים הנלווים

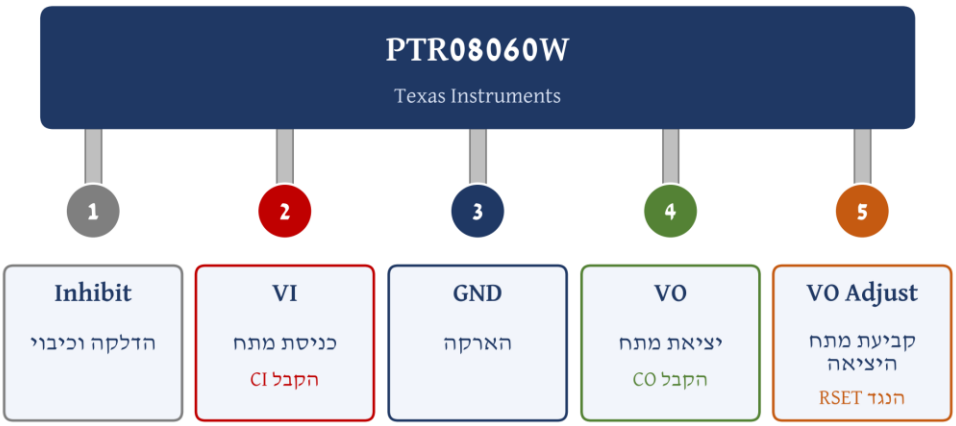
טבלה 6: הרכיבים הנלווים

רכיב	ערך	תפקיד
הקבל CI	100uF, אלקטרוליטי	מחובר בכניסת המייצב ומסנן רעשים, כך שמתח הכניסה נשאר יציב גם כשהמייצב מושך זרם בפולסים
הקבל CO	100uF	מחובר ביציאת המייצב ומחליק את המתח, כך שלא יהיו קפיצות כשצריכת הזרם של כרטיסי ה PHY משתנה
הנגד RSET	432 אוהם, דיוק של 1%	קובע את מתח היציאה של המייצב ל 3.3V

את הערכים לקחתי מדף הנתונים של המייצב. לפי היצרן, בשביל מתח יציאה של 3.3V צריך נגד RSET של 432 אוהם, שנותן מתח של 3.327V, ובכניסה וביציאה צריך קבלים של 100uF לפחות.

הרגליים של המייצב

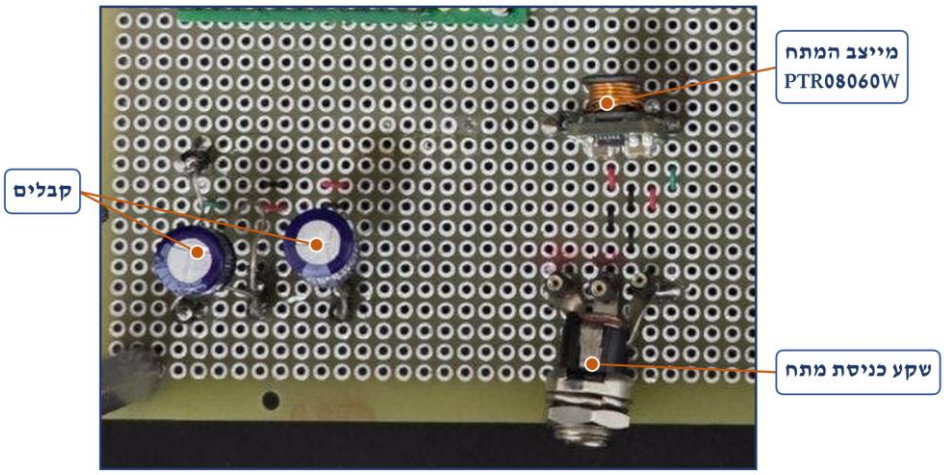
למודול יש חמש רגליים, ולכל אחת מהן תפקיד משלה. אלה אותן רגליים שמופיעות בתרשים החשמלי של המייצב בנספח א, וליד כל רגל מופיע גם הרכיב שמחובר אליה.



איור 14: הרגליים של PTR08060W

המייצב במערכת

כך נראה אזור אספקת המתח על הלוח. המתח נכנס דרך השקע, עובר דרך המייצב PTR08060W, ומשם יוצא מתח מיוצב של 3.3V אל שני כרטיסי ה PHY.



איור 15: אזור אספקת המתח

פרוטוקולים

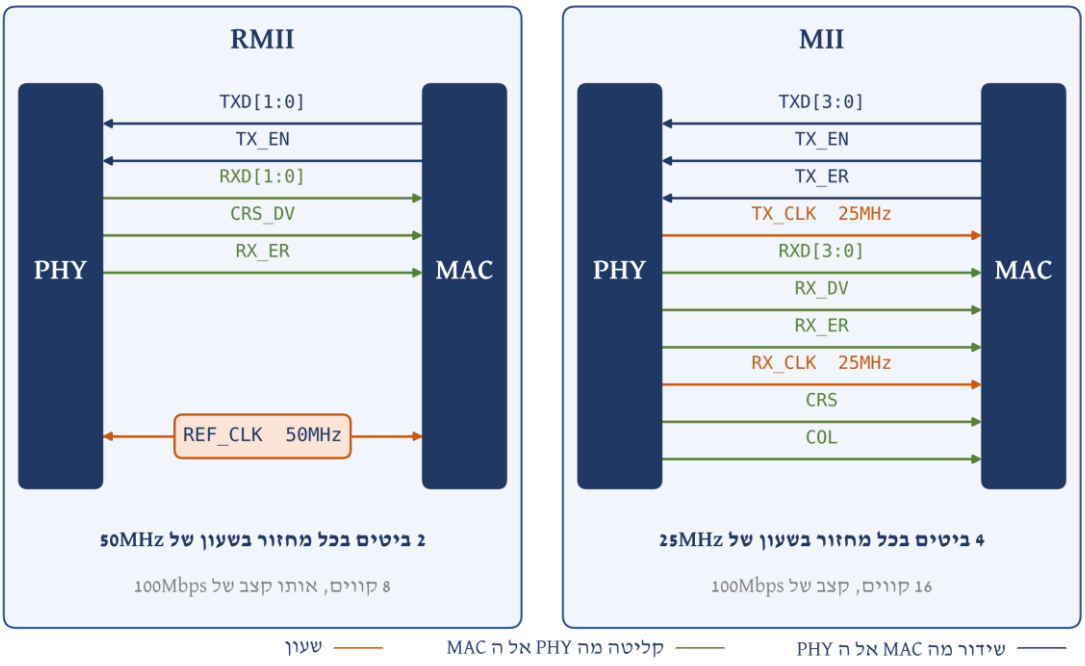
RMII ו MII

Avalon MM

RMII ו MII

בצד השני של ה PHY נמצא ה MAC, שאחראי על הטיפול במסגרת ה Ethernet. כדי להעביר את המידע מה MAC אל ה PHY נדרש ממשק דיגיטלי. אחד הממשקים הראשונים שנעשה בהם שימוש הוא MII, וממנו התפתח ממשק RMII שנועד לצמצם את מספר החיבורים הנדרשים בין שני הרכיבים.

MII



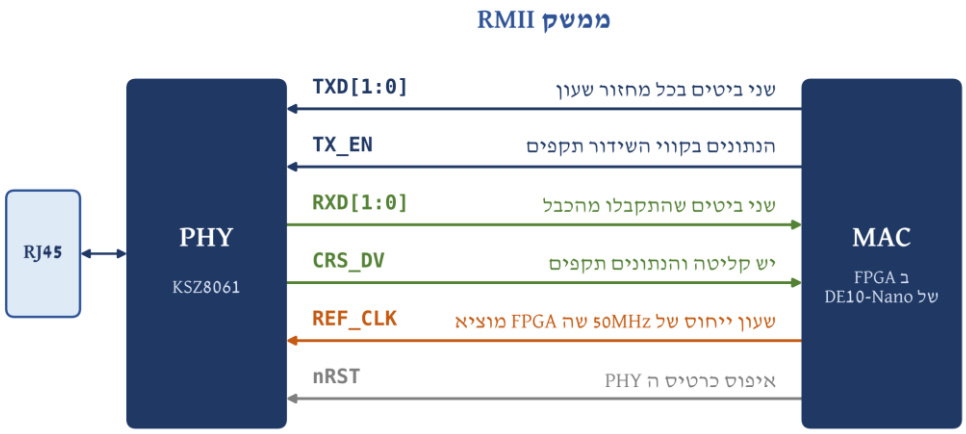
איור 16: ממשק MII

Media Independent Interface הוא ממשק דיגיטלי המחבר בין ה MAC לבין ה PHY. הממשק מעביר את המידע בצורה דיגיטלית לפני שה PHY מבצע את ההמרה לאותות החשמליים המועברים בכבל.

ב MII קיימים ארבעה קווי נתונים להעברת המידע. בתקשורת של 100Mbps הממשק משתמש בשעון של 25MHz, כאשר בכל מחזור שעון מועברים ארבעה ביטים. כך מתקבל קצב העברת מידע של 100Mbps.

למרות שממשק MII מספק את קצב התקשורת הנדרש, הוא דורש מספר יחסית גדול של קווים בין ה MAC לבין ה PHY. במערכות משובצות שבהן מספר הפינים מוגבל, הדבר יכול להוות חיסרון.

RMII



איור 17: ממשק RMII

Reduced Media Independent Interface, פותח כדי לצמצם את מספר הקווים הנדרשים בין ה MAC לבין ה PHY. הרעיון המרכזי הוא להעביר פחות ביטים בכל מחזור שעון, ובמקביל להעלות את תדר השעון.

ב RMII מועברים שני ביטים בכל מחזור שעון במקום ארבעה ביטים ב MII. לכן, כדי לשמור על קצב תקשורת של 100Mbps, תדר השעון עולה מ 25MHz ל 50MHz.

כך ניתן לשמור על אותו קצב העברת מידע, אך להשתמש במספר קטן יותר של קווי נתונים.

הכפלת תדר השעון ב RMII היא למעשה הפיצוי על הקטנת רוחב ממשק הנתונים. במקום להעביר כמות גדולה יותר של ביטים בכל מחזור שעון, הממשק מעביר פחות ביטים אך מבצע יותר מחזורי שעון באותו פרק זמן.

היתרון של RMII משמעותי במיוחד במערכות כמו FPGA ו SoC, שבהן כל חיבור פיזי בין רכיבים דורש פין מתאים. צמצום מספר קווי הנתונים מאפשר לחסוך בפינים ולהפוך את החיבור בין ה MAC לבין ה PHY לפשוט יותר.

קווי הנתונים

TXD0, TXD1

שני קווי הנתונים לשידור. ה MAC מעביר דרכם את המידע אל ה PHY, כאשר בכל מחזור שעון מועברים 2 ביטים.

TX_EN – Transmit Enable

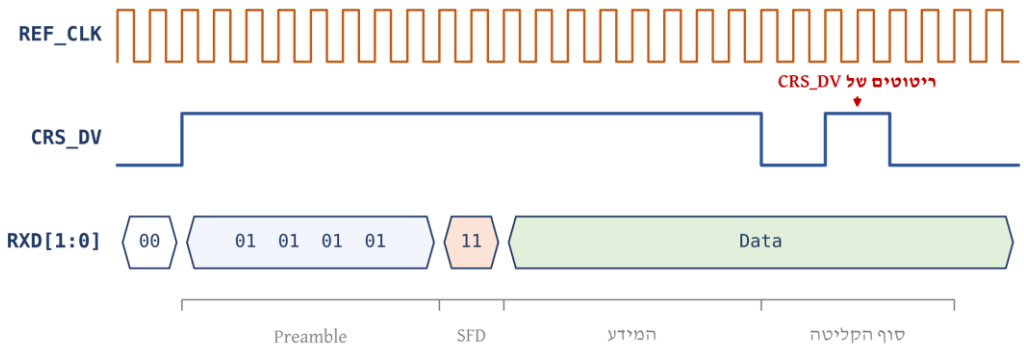
מציין שהמידע המועבר בקווי TXD0 ו TXD1 תקף ושיש לשדר אותו. כאשר האות אינו פעיל, ה PHY אינו מתייחס לנתונים שבקווי השידור.

RXD0, RXD1

שני קווי הנתונים לקבלה. ה PHY מעביר דרכם את המידע שהתקבל מהכבל אל ה MAC, גם כאן 2 ביטים בכל מחזור שעון.

CRS_DV – Carrier Sense Data Valid

אות המשמש את ה PHY כדי להודיע ל MAC שמתקבל מידע ושקווי RXD0 ו RXD1 מכילים נתונים תקפים. בסוף הקליטה קו זה מכיל ריטוטים של עלייה וירידה בין 0 ל 1, ולכן סיום קליטה מתרחש רק כאשר יש לנו רצף של שני אפסים.



איור 18: אות CRS_DV

REF_CLK – Reference Clock

שעון הייחוס של ממשק RMII, בתדר של 50MHz. השעון משמש לסנכרון העברת הנתונים בין ה MAC לבין ה PHY.

RMII בפרויקט

בפרויקט זה ממשק ה RMII מחבר בין ה MAC שממומש ב FPGA לבין שני כרטיסי ה PHY מסוג KSZ8061. לכל כרטיס יש קווי שידור וקווי קליטה משלו, TXD0, TXD1, TX_EN, RXD0, RXD1 ו CRS_DV, ושני הכרטיסים מקבלים את אותו שעון ייחוס של 50MHz שה FPGA מוציא.

את החיבור המלא, כולל מספרי הרגליים במחבר של כרטיס ה PHY ופיני ה FPGA, פירטתי בפרק החומרה ובתרשים החשמלי.

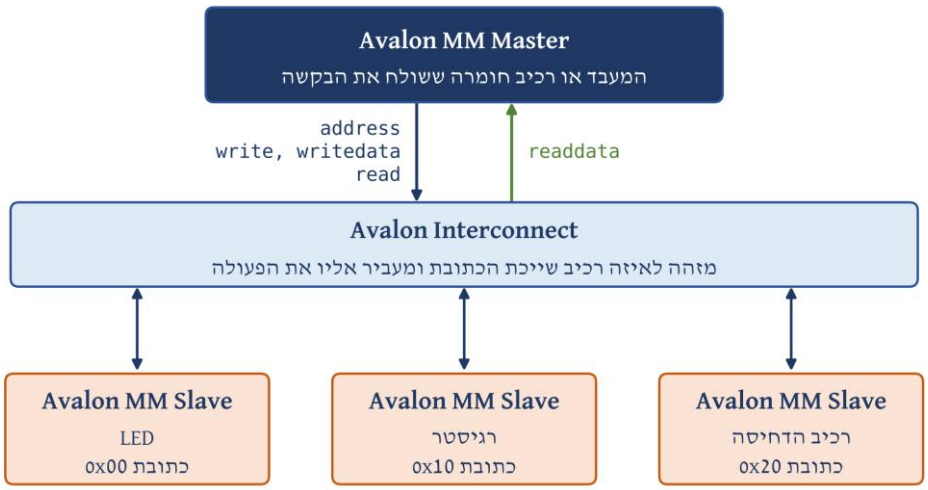
Avalon MM

ממשק **Avalon Memory Mapped** משמש להעברת מידע בין רכיבים שונים בתוך מערכת FPGA. הממשק מאפשר לרכיב אחד במערכת לגשת לרכיב אחר באמצעות כתובות, בדומה לגישה לזיכרון.

הרעיון המרכזי של Avalon MM הוא שכל רכיב מקבל כתובת במרחב הכתובות של המערכת. כאשר רכיב רוצה לקרוא או לכתוב מידע, הוא שולח כתובת יחד עם אותות הבקרה והמידע הדרוש. המערכת מזהה לאיזה רכיב שייכת הכתובת ומעבירה אליו את הפעולה.

בצורה זו ניתן לחבר רכיבים שונים בתוך ה FPGA, כאשר כל רכיב מקבל כתובת משלו וניתן לגשת אליו מתוך תוכנה או מרכיב חומרה אחר.

לדוגמה, אם יצרנו רכיב חומרה שמחובר ל LED או לרשם ב FPGA, ניתן להגדיר לו כתובת מסוימת. כאשר מתבצעת כתיבה לכתובת הזו, הרכיב מקבל את הנתון ופועל בהתאם. באופן דומה, ניתן לקרוא נתונים מרכיב חומרה על ידי ביצוע פעולת קריאה מהכתובת המתאימה.



איור 19: Avalon MM

כיצד Avalon MM עובד

בפעולת כתיבה, הרכיב השולח מעביר כתובת, נתון ואותות בקרה. הרכיב שאליו פונים מקבל את הנתון ושומר אותו ברשם או מבצע את הפעולה שהוגדרה עבור הכתובת.

בפעולת קריאה, הרכיב השולח מעביר כתובת ומבקש לקרוא ממנה מידע. הרכיב שמחזיק בכתובת מחזיר את הנתון אל הרכיב שביצע את הקריאה.

לכן Avalon MM מאפשר ליצור תקשורת מסודרת בין רכיבים כאשר הכתובת קובעת לאיזה רכיב מיועדת הפעולה.

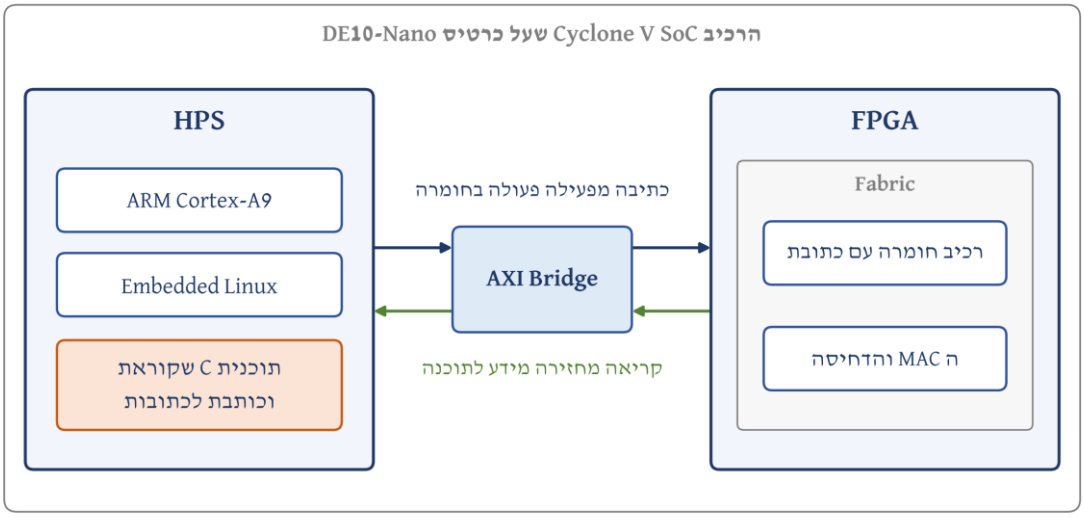
בפרויקט ניתן להשתמש במבנה כזה כדי לחבר בין הלוגיקה שממומשת ב FPGA לבין רכיבים נוספים במערכת, כך שהמעבד או רכיב אחר יוכל לגשת לרכיבי החומרה דרך כתובות מוגדרות.

Avalon MM במערכת עם HPS ו FPGA

בפרויקט יש שני חלקים מרכזיים, ה HPS וה FPGA. ה HPS מכיל את המעבד ואת מערכת Linux, ואילו ה FPGA משמש למימוש הלוגיקה והחומרה שמתוכננות בפרויקט.

כדי שהמעבד יוכל לעבוד עם רכיבי החומרה שנמצאים ב FPGA, נדרש חיבור בין שני העולמות. כאן נכנס לתמונה מנגנון הזיכרון הממופה והחיבורים הפנימיים במערכת.

המעבד יכול לבצע קריאה או כתיבה לכתובת מסוימת, והמערכת מנתבת את הפעולה אל הרכיב המתאים בתוך ה FPGA. בצורה זו תוכנה שרצה על ה HPS יכולה לשלוט ברכיבי חומרה שנמצאים בתוך ה FPGA. לדוגמה, ניתן להגדיר רכיב חומרה עם כתובת מסוימת ולגשת אליו מתוך תוכנית C שרצה ב Linux. כתיבה לכתובת תפעיל פעולה בחומרה, וקריאה מאותה כתובת תחזיר מידע מהרכיב אל התוכנה. כך נוצרת תקשורת ישירה בין התוכנה לבין החומרה.



איור 20: FPGA ו HPS

גישה לחומרה מתוך Linux בעזרת mmap

כדי שתוכנית C ב Linux תוכל לגשת לרכיב שנמצא ב FPGA, היא צריכה להגיע לכתובת הפיזית שלו. תוכנית רגילה ב Linux עובדת עם כתובות וירטואליות ולא יכולה לגשת ישירות לכתובת פיזית, ולכן אני משתמש בקובץ המיוחד /dev/mem ובפונקציה mmap.

ב Cyclone V הגשר הקל בין ה HPS ל FPGA, Lightweight HPS to FPGA Bridge, מתחיל בכתובת הפיזית 0xFF200000. כל רכיב שאני מחבר לגשר הזה ב Platform Designer מקבל כתובת יחסית, Offset, מתחילת הגשר.

כדי להבין איך זה עובד כתבתי תוכנית קצרה. היא ממפה את אזור הגשר לזיכרון של התוכנית, כותבת ערך לרגיסטר שנמצא ב Offset של 0, וקוראת אותו בחזרה:

```
#include <stdio.h>
#include <stdint.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>

// lightweight HPS to FPGA bridge
#define LW_BRIDGE_BASE  0xFF200000
#define LW_BRIDGE_SPAN  0x00200000
// register offset inside the FPGA
#define REG_OFFSET      0x0

int main(void)
{
    // open the physical memory of the system
    int fd = open("/dev/mem", O_RDWR | O_SYNC);
    if (fd < 0) { perror("open"); return 1; }

    // map the bridge into the address space of the program
    void *base = mmap(NULL, LW_BRIDGE_SPAN, PROT_READ | PROT_WRITE,
                      MAP_SHARED, fd, LW_BRIDGE_BASE);
    if (base == MAP_FAILED) {
        perror("mmap");
        close(fd);
        return 1;
    }

    // pointer to the register inside the FPGA
    volatile uint32_t *reg =
        (volatile uint32_t *)((char *)base + REG_OFFSET);

    // write, becomes an Avalon write in the FPGA
    *reg = 0x5;
    // read, becomes an Avalon read from the FPGA
    printf("value = 0x%X\n", *reg);

    munmap(base, LW_BRIDGE_SPAN);
    close(fd);
    return 0;
}
```

הפונקציה open פותחת את /dev/mem, ו mmap מחזירה מצביע שמייצג את תחילת הגשר. מהרגע הזה, כל כתיבה דרך המצביע reg הופכת לפעולת כתיבה של Avalon MM ברכיב שב FPGA, וכל קריאה דרכו הופכת לפעולת קריאה.

המילה volatile אומרת לקומפיילר שהערך בכתובת יכול להשתנות מחוץ לתוכנית, ולכן הוא חייב לקרוא ולכתוב לחומרה בכל פעם ולא לשמור את הערך בצד. את התוכנית אני מריץ עם הרשאות root, כי רק לו מותר לפתוח את /dev/mem.

Fabric

ה Fabric הוא למעשה אזור הלוגיקה המתכנתת של ה FPGA שבו נמצאים הרכיבים והחיבורים שמגדירים בתכנון.

בתוך ה Fabric ניתן לממש רכיבים שונים כמו רגיסטרים, זיכרונות, ממשקי תקשורת, מעבדים ורכיבי עיבוד ייעודיים.

החיבורים בין הרכיבים מתבצעים באמצעות תשתית פנימית שמאפשרת להעביר כתובות, נתונים ואותות בקרה בין הרכיבים השונים.

בפרויקט הלוגיקה שמפותחת ב FPGA נמצאת בתוך ה Fabric, ולכן ה Fabric הוא למעשה האזור שבו מתבצע העיבוד החומרתי של המערכת.

AXI

ממשק AXI, ראשי התיבות של **Advanced eXtensible Interface**, נפוץ מאוד במערכות SoC וב FPGA, במיוחד במערכות המבוססות על מעבדים ולוגיקה מתכנתת.

בדומה ל Avalon MM, גם AXI מאפשר לרכיבים במערכת לבצע פעולות קריאה וכתיבה באמצעות כתובות ונתונים. ההבדל הוא ש AXI הוא חלק ממשפחת ממשקים המשמשת בעיקר בסביבות המבוססות על ARM ו FPGA.

במערכות מסוימות ה HPS או המעבד משתמשים ב AXI כדי להגיע אל רכיבים שנמצאים ב FPGA. במקרה כזה AXI משמש כדרך להעביר את הבקשה מהמעבד אל הלוגיקה, ובתוך ה FPGA ניתן לחבר את המידע אל רכיבים המשתמשים ב Avalon או בממשקים אחרים.

לכן AXI ו Avalon MM מבצעים רעיון דומה של העברת מידע בין רכיבים באמצעות כתובות, אך הם שייכים לתשתיות ממשק שונות אחד של ה HPS ואחד של ה FPGA.

הקשר בין AXI, Avalon MM וה Fabric

המעבד שולח בקשת קריאה או כתיבה, הבקשה עוברת דרך ממשק התקשורת ומגיעה אל ה Fabric. בתוך ה Fabric היא מנותבת אל הרכיב המתאים לפי הכתובת שלו.

כאשר הפעולה מסתיימת, הנתון או התוצאה חוזרים דרך אותו מסלול אל המעבד.

המבנה הזה מאפשר להפריד בין התוכנה שרצה על ה HPS לבין החומרה שנמצאת ב FPGA, ועדיין לאפשר ביניהן תקשורת ישירה ומסודרת.



איור 21: מבנה המערכת

תקשורת ורשתות

מודל שבע השכבות OSI Model

שכבה פיזית – PHY

מחבר RJ45

פרוטוקול Ethernet ומבנה הפרוטוקול

Media Access Control – MAC

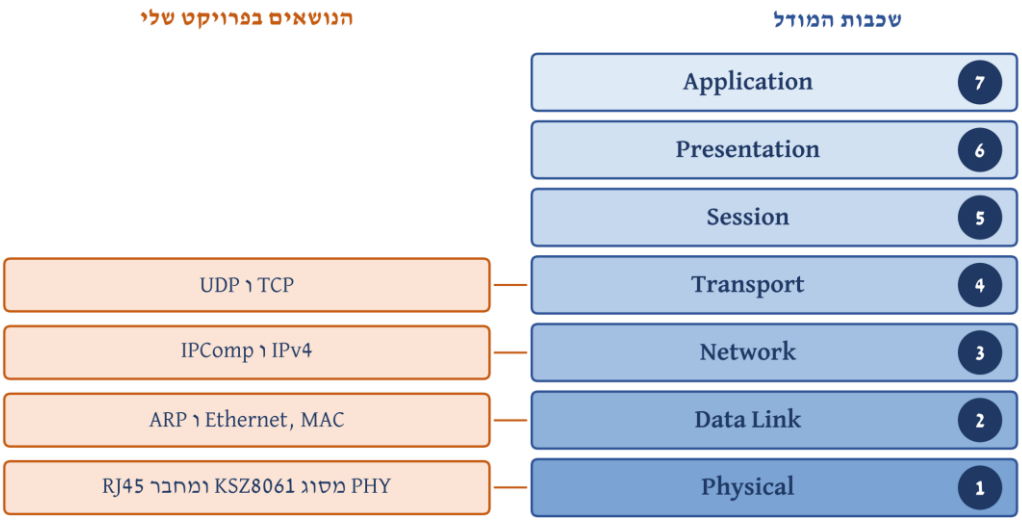
ARP

IPv4

UDP ו TCP

מודל שבע השכבות OSI Model

מודל OSI הוא מודל תאורטי המתאר את אופן העברת המידע בין מחשבים והתקני תקשורת ברשת. המודל מחלק את תהליך התקשורת לשבע שכבות, כאשר לכל שכבה יש תפקיד מוגדר. חלוקה זו מאפשרת להבין בצורה מסודרת כיצד מידע עובר ממחשב אחד לאחר.



איור 22: שבע השכבות

שכבה 7 Application

זוהי השכבה הקרובה ביותר למשתמש. היא מספקת שירותי רשת לתוכנות וליישומים שבהם המשתמש עובד.

השכבה אינה אחראית על העברת הביטים עצמם, אלא על מתן השירותים והפרוטוקולים שבאמצעותם יישומים יכולים לתקשר ברשת.

שכבה 6 Presentation

תפקידה של שכבת הייצוג הוא לטפל באופן שבו הנתונים מיוצגים בין מערכות שונות.

השכבה יכולה להיות קשורה לפעולות כגון שינוי פורמט הנתונים, קידוד, דחיסה והצפנה. המטרה היא לוודא שהמידע שנשלח יהיה בפורמט שהמערכת בצד השני יכולה להבין ולעבד.

שכבה 5 Session

שכבת השיחה אחראית על ניהול הקשר בין שני צדדים בתקשורת.

היא עוסקת ביצירה, ניהול וסיום של שיחות תקשורת בין מערכות. בנוסף, במודלים מסוימים היא יכולה לסייע בניהול נקודות סנכרון, כך שניתן יהיה להמשיך תהליך תקשורת במקרה של הפרעה.

שכבה 4 Transport

שכבת התעבורה אחראית על העברת הנתונים בין שני מחשבים על ידי ניהול התקשורת ביניהם.

השכבה יכולה לחלק מידע גדול לחלקים קטנים יותר ולבצע את הפעולות הנדרשות כדי להעביר אותם ליעד. בנוסף, היא משתמשת במספרי פורטים כדי לאפשר למספר יישומים להשתמש בתקשורת הרשת במקביל.

שני הפרוטוקולים המרכזיים הקשורים לשכבה זו הם TCP ו UDP.

TCP מספק תקשורת אמינה יותר, הכוללת מנגנונים לזיהוי אובדן נתונים, סדר נכון של המידע ושליחה מחדש במקרה הצורך. לכן הוא מתאים לדוגמאות כמו העברת קבצים ותקשורת שבה חשוב לקבל את כל המידע בצורה תקינה.

UDP הוא פרוטוקול פשוט ומהיר יותר שאינו מבצע את כל מנגנוני הבקרה של TCP. לכן הוא מתאים ליישומים שבהם מהירות וזמן תגובה חשובים, כגון משחקי רשת, שידורי וידאו ותקשורת בזמן אמת.

שכבה 3 Network

שכבת הרשת אחראית על העברת מידע בין רשתות שונות ועל ניתוב החבילות אל היעד.

בשכבה זו נעשה שימוש בכתובות IP, המאפשרות לזהות את המקור והיעד של התקשורת ברמת הרשת.

התקן המרכזי בשכבה זו הוא הנתב Router. הנתב מקבל חבילות מידע ובוחר את הנתיב המתאים להעברתן לרשת הבאה עד שהן מגיעות ליעד.

שכבה 2 Data Link

שכבת קישור הנתונים אחראית על העברת מידע בין התקנים הנמצאים באותה רשת מקומית LAN.

בשכבה זו המידע מועבר במסגרת הנקראת Frame, אשר מכילה בין היתר את כתובת ה MAC של התקן המקור ושל התקן היעד.

אחד ההתקנים המרכזיים בשכבה זו הוא מתג Switch. המתג לומד את כתובות ה MAC של ההתקנים המחוברים אליו ושומר אותן בטבלה. כאשר מגיעה אליו מסגרת מידע, הוא בודק את כתובת ה MAC של היעד ומעביר את המסגרת לפורט המתאים.

חשוב להבדיל בין כתובת IP לבין כתובת MAC. כתובת IP משמשת לזיהוי ולניתוב של התקנים בין רשתות שונות, בעוד שכתובת MAC משמשת לזיהוי ממשק הרשת ולתקשורת בתוך רשת מקומית.

שכבה 1 Physical

השכבה הפיזית היא השכבה הנמוכה ביותר במודל OSI. היא עוסקת בהעברה הפיזית של ביטים באמצעות אמצעי התקשורת.

בשכבה זו המידע הדיגיטלי מיוצג באמצעות אותות פיזיים המתאימים לסוג המדיה שבה משתמשים.



איור 23: אמצעי התקשורת בשכבה הפיזית

השכבה הפיזית אינה עוסקת בכתובות IP או MAC ואינה מבינה את המשמעות של המידע עצמו. תפקידה הוא להעביר את האותות הפיזיים ולייצג באמצעותם את הביטים המועברים.

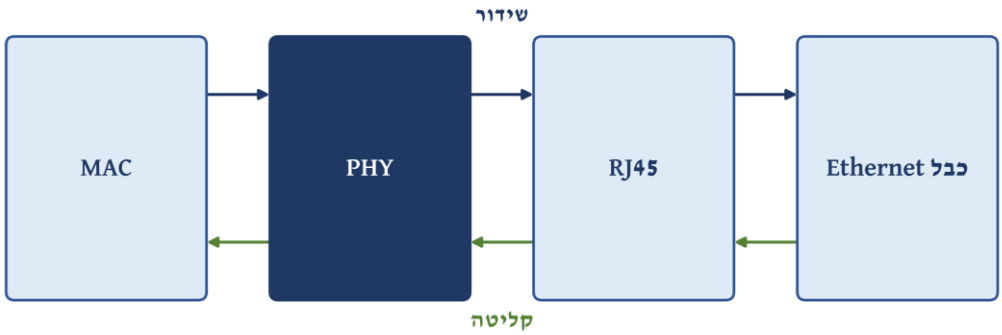
שכבה פיזית – PHY

Physical Layer Transceiver – PHY הוא רכיב חומרה האחראי על העברת המידע בין המערכת הדיגיטלית לבין אמצעי התקשורת הפיזי, לדוגמה כבל Ethernet.

ה PHY נמצא בשכבה הפיזית (Layer 1) של מודל OSI. תפקידו הוא לקבל מידע דיגיטלי מהמערכת, להמיר אותו לאותות המתאימים להעברה על גבי המדיה הפיזית, ובצד המקבל לבצע את הפעולה ההפוכה ולהחזיר את האותות למידע דיגיטלי.

במערכת Ethernet, ה PHY נמצא בין בקר ה Ethernet לבין חיבור ה RJ45 והכבל.

ניתן לתאר את מבנה התקשורת באופן הבא, כאשר בכיוון הקבלה התהליך מתבצע בצורה הפוכה:



איור 24: מסלול השידור והקליטה

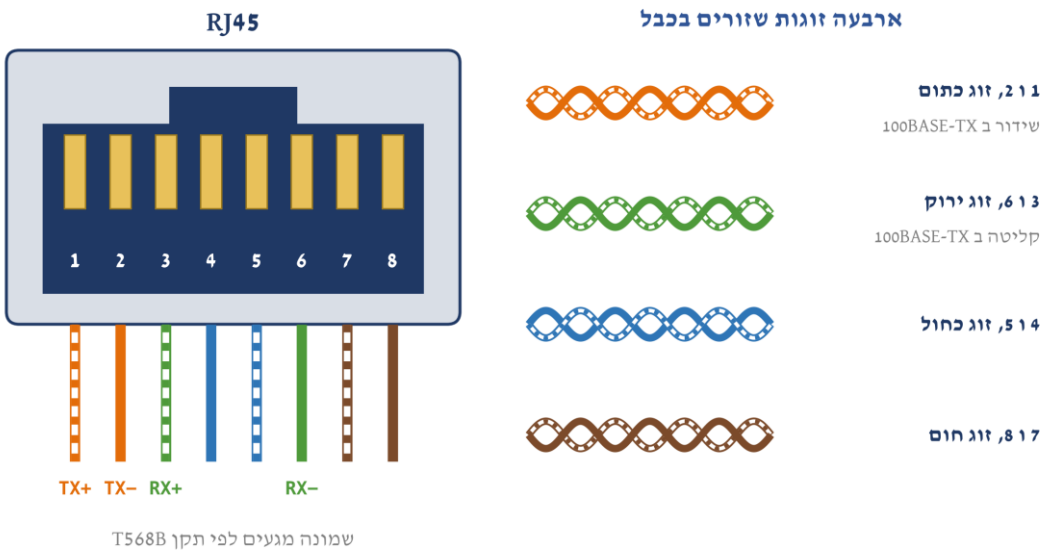
תפקיד ה PHY

ה PHY מטפל בהיבטים הפיזיים של התקשורת. בין היתר הוא אחראי על קידוד ופענוח של המידע, יצירת האות החשמלי המתאים להעברה בכבל, קבלת האות מהכבל והמרתו חזרה למידע דיגיטלי.

בנוסף, ה PHY מבצע פעולות הנדרשות כדי לאפשר העברה אמינה של המידע על גבי המדיה הפיזית, כגון סנכרון בין המשדר למקלט והתאמת האות למאפייני ערוץ התקשורת.

מחבר RJ45

מחבר ה RJ45 הוא המחבר הפיזי המשמש לחיבור מערכת ה MAC לכבל הרשת. המחבר כולל שמונה מגעים, כאשר כל מגע מחובר לחוט המתאים בתוך כבל הרשת. כבל Ethernet מסוג Twisted Pair בנוי מארבעה זוגות של חוטים, כאשר החוטים בכל זוג שזורים זה סביב זה. השזירה נועדה לצמצם הפרעות חשמליות ורעשים בין הזוגות ובכך לשמור על איכות האות במהלך העברת המידע.



איור 25: מחבר RJ45

בכבל קיימים בסך הכל שמונה חוטים, המחולקים לארבעה זוגות. השימוש בכל זוג תלוי בתקן Ethernet ובמהירות התקשורת.

מחבר ה RJ45 אינו מבצע עיבוד של המידע. הוא משמש כחיבור הפיזי בין ה PHY לבין כבל הרשת ומעביר את האותות החשמליים שנוצרים על ידי ה PHY.

לכן ניתן לראות את ה RJ45 כחלק מהחיבור הפיזי של מערכת ה Ethernet, ולא כרכיב שמבצע עיבוד או קידוד של המידע.

פרוטוקול Ethernet ומבנה הפרוטוקול

Ethernet הוא אחד מפרוטוקולי התקשורת המרכזיים ברשתות מקומיות. הוא פועל בעיקר בשכבה 2 של מודל שבע השכבות, ומשמש להעברת מידע בין התקנים הנמצאים באותה רשת מקומית.

ב Ethernet המידע מועבר בצורה של מסגרת הנקראת Frame. המסגרת מכילה את הנתונים שצריך להעביר, יחד עם מידע נוסף הדרוש כדי להעביר את המסגרת אל היעד ולבדוק שהמידע שהתקבל תקין.

מבנה מסגרת Ethernet



איור 26: מבנה מסגרת Ethernet

מסגרת Ethernet רגילה מורכבת ממספר שדות עיקריים

Preamble, SFD, Destination MAC,
Source MAC, EtherType, Data, FCS

כל אחד מהשדות ממלא תפקיד שונה בתהליך העברת המידע.

1. Preamble

ה Preamble נמצא בתחילת המסגרת ומשמש לסנכרון בין המשדר למקלט.

הוא מאפשר להתקן המקבל לזהות שמתחילה העברת מסגרת ולסנכרן את עצמו עם האות המתקבל.

ה Preamble הוא בגודל של 7 בתים.

Start Frame Delimiter – 2. SFD

ה SFD מסמן את תחילת מסגרת ה Ethernet עצמה.

הוא מגיע מיד לאחר ה Preamble ומאפשר למקלט לדעת שהחל מהשדה הבא מתחילים פרטי המסגרת.

גודל ה SFD הוא בית אחד.

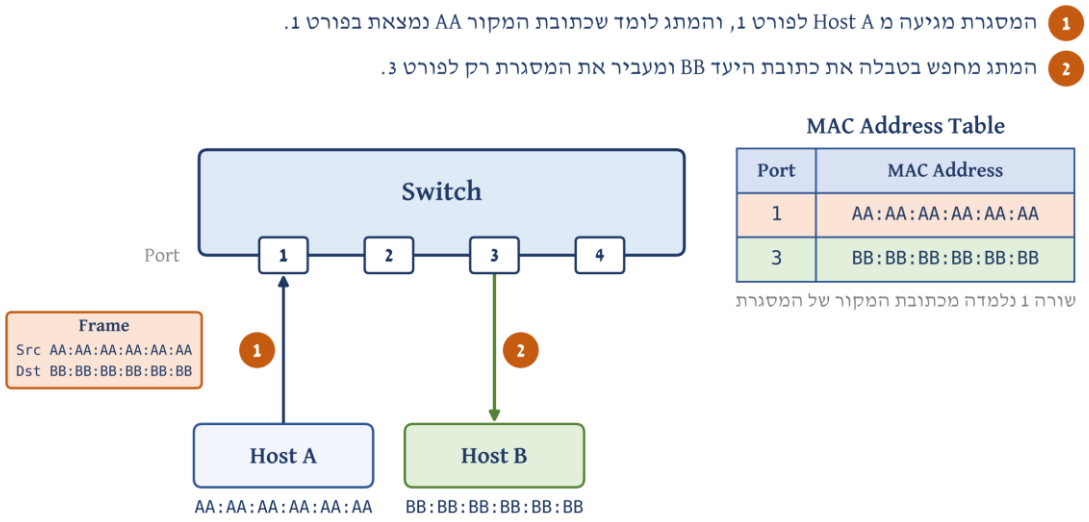
3. Destination MAC Address

זהו שדה המכיל את כתובת ה MAC של התקן היעד.

כתובת MAC היא כתובת ייחודית של ממשק רשת ומשמשת לזיהוי התקנים ברשת המקומית.

גודל כתובת ה MAC הוא 6 בתים (48 ביט).

מתג Switch משתמש בכתובות ה MAC כדי לקבוע לאיזה פורט יש להעביר את המסגרת.



איור 27: כתובת MAC

4. Source MAC Address

זהו שדה המכיל את כתובת ה MAC של התקן המקור, כלומר ההתקן ששלח את המסגרת.

גם שדה זה הוא בגודל של 6 בתים (48 ביט).

לדוגמה:

"A4:5E:60:12:34:56"

כאשר מתג מקבל מסגרת, הוא יכול ללמוד ממנה באיזה פורט נמצאת כתובת ה MAC של השולח ולשמור מידע זה בטבלת ה MAC שלו.

5. EtherType / Length

שדה זה מכיל מידע לגבי סוג התוכן שנמצא בתוך המסגרת.

ב Ethernet II, הצורה הנפוצה כיום, השדה נקרא EtherType והוא מציין איזה פרוטוקול נמצא בתוך ה Data.

לדוגמה:

0x0800 – IPv4
0x0806 – ARP
0x86DD – IPv6

גודל השדה הוא 2 בתים (16 ביט).

לדוגמה, אם ערך ה EtherType הוא "0x0800", ניתן להבין שהמידע שבתוך המסגרת מכיל חבילת IPv4.

6. Data / Payload

זהו החלק שבו נמצאים הנתונים שמועברים.

לדוגמה, כאשר מחשב שולח חבילת IP, חבילת ה IP מוכנסת לתוך אזור ה Data של מסגרת Ethernet.

כלומר, Ethernet אינו בהכרח מכיל את המידע של היישום ישירות. הוא יכול להעביר בתוכו פרוטוקול משכבה גבוהה יותר.

גודל אזור ה data ב Ethernet הוא בדרך כלל בין 46 ל 1500 בתים במסגרת Ethernet רגילה.

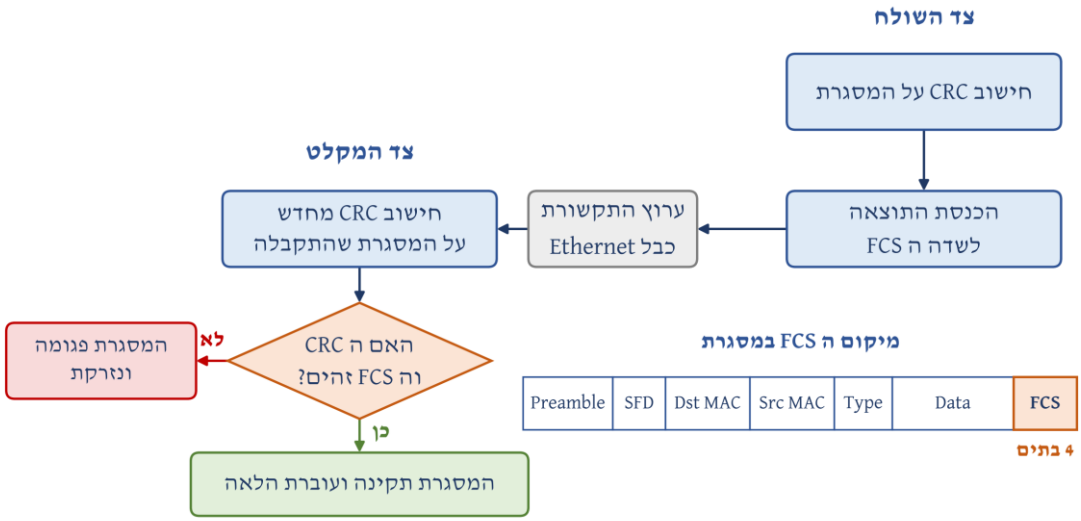
אם הנתונים קטנים מדי, מתווסף Padding כדי שהמסגרת תעמוד בגודל המינימלי הנדרש.

Frame Check Sequence – 7. FCS

ה FCS נמצא בסוף המסגרת ומשמש לבדיקת תקינות המידע שהתקבל.

המכשיר השולח מחשב ערך CRC ומכניס אותו ל FCS. כאשר המקלט מקבל את המסגרת, הוא מבצע חישוב דומה ומשווה את התוצאה לערך שהתקבל.

אם התוצאה אינה תואמת, ניתן להסיק שהמסגרת נפגעה במהלך ההעברה, והמסגרת נחשבת פגומה. גודל ה FCS הוא 4 בתים (32 ביט).



איור 28: בדיקת FCS

Media Access Control – MAC

ה MAC הוא החלק במערכת ה Ethernet שאחראי על הטיפול הדיגיטלי במידע ועל העברת מסגרות Ethernet בין המערכת לבין ה PHY.

בפרויקט ה MAC אינו רכיב חיצוני נפרד, אלא ממומש כחלק מהלוגיקה של ה FPGA. הקוד והלוגיקה שממומשים ב FPGA אחראים לקבל את המידע, לטפל בו ולהעביר אותו בקצב ובתזמון המתאימים אל ממשק ה RMII.

ה MAC מקבל את המידע מהמערכת ומבצע את פעולות העיבוד הנדרשות לפני העברתו אל ה PHY. בזמן השידור הוא צריך לנהל את הנתונים ואת האותות של ממשק ה RMII, כולל קווי הנתונים **TXD0** ו **TXD1**, אות **TX_EN** והשעון של הממשק.

ה RMII עובד עם שעון של **50MHz**, ולכן הלוגיקה של ה MAC צריכה לעבוד בצורה מסונכרנת עם השעון הזה ולדאוג שהנתונים יוצגו בקווי המידע בזמן המתאים. בכל מחזור שעון מועברים שני ביטים דרך TXD0 ו TXD1.

בכיוון הקבלה התהליך מתבצע בצורה הפוכה. ה PHY מעביר את המידע שהתקבל מהכבל דרך **RXD0** ו **RXD1**, וה MAC הממומש ב FPGA מקבל את הנתונים ומעבד אותם בהתאם ללוגיקה של המערכת. האות **CRS_DV** משמש כדי לציין שהנתונים המתקבלים תקפים.

לכן בפרויקט ה MAC הוא למעשה חלק מהקוד והלוגיקה של המערכת, והוא משמש כנקודת החיבור בין עיבוד הנתונים שמתבצע ב FPGA לבין ממשק ה RMII וה PHY.

ARP

ARP הוא פרוטוקול שמוצא את כתובת ה MAC המתאימה לכתובת IP בתוך רשת מקומית. השם המלא שלו הוא Address Resolution Protocol. הצורך ב ARP נובע מכך שבתקשורת ברשת משתמשים גם בכתובת IP וגם בכתובת MAC, כאשר לכל אחת מהן תפקיד שונה.

כתובת ה IP משמשת לזיהוי היעד ברמת שכבת הרשת, בעוד שכתובת ה MAC משמשת להעברת ה Frame בפועל בתוך הרשת המקומית. לכן כאשר מחשב יודע את כתובת ה IP של היעד אך אינו יודע את כתובת ה MAC שלו, הוא משתמש ב ARP כדי למצוא את ההתאמה ביניהן.

כיצד ARP עובד

כאשר מחשב רוצה לשלוח מידע להתקן ברשת המקומית, הוא בודק תחילה האם כתובת ה MAC של היעד כבר נמצאת אצלו ב **ARP Cache**. אם ההתאמה קיימת, ניתן להשתמש בה ישירות ואין צורך לבצע בקשת ARP חדשה.

```
Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

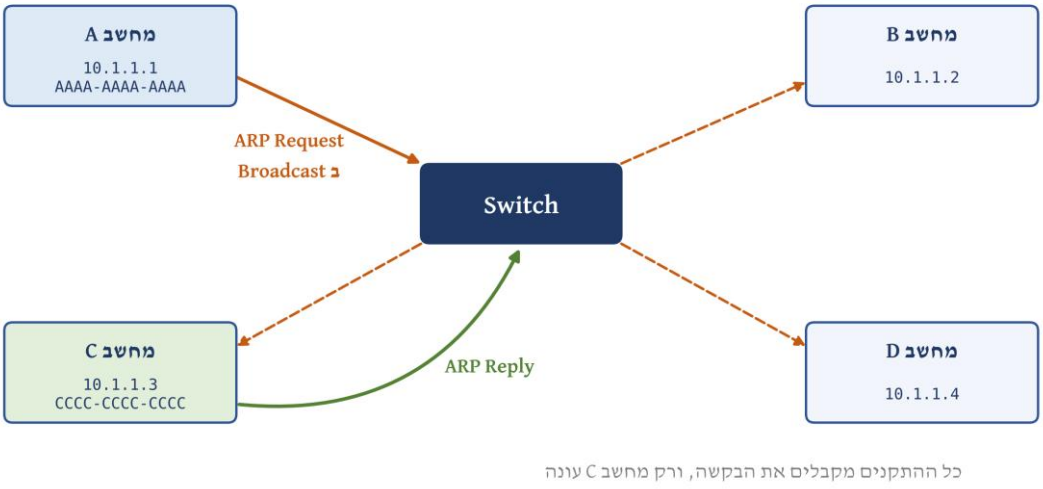
C:\Users\noone>arp -a

Interface: 10.225.32.207 --- 0x5
  Internet Address      Physical Address      Type
  10.225.32.34          7e-23-5a-13-58-39     dynamic
  10.225.32.255         ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

איור 29: ARP Cache

כאשר כתובת ה MAC אינה ידועה, המחשב שולח הודעת **ARP Request**. ההודעה נשלחת לכל ההתקנים ברשת המקומית באמצעות Broadcast ומכילה את כתובת ה IP שאותה המחשב מחפש.

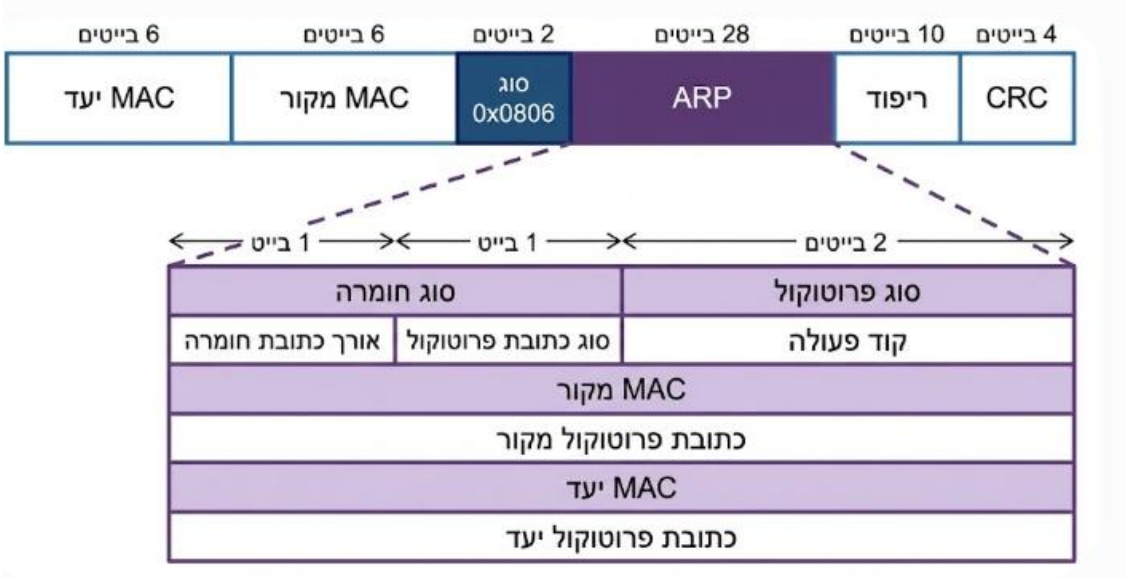
ההודעה למעשה שואלת מי מחזיק בכתובת ה IP המבוקשת ומהי כתובת ה MAC שלו. ההתקן שהכתובת שייכת אליו מזהה את עצמו ושולח בחזרה הודעת **ARP Reply** המכילה את כתובת ה MAC שלו.



איור 30: ARP Request ו ARP Reply

לאחר קבלת התשובה, המחשב יכול לשמור את ההתאמה בין כתובת ה IP לכתובת ה MAC ב ARP Cache ולהשתמש בה בהעברות הבאות.

מבנה הודעת ARP



איור 31: מבנה הודעת ARP

הודעת ARP מורכבת ממספר שדות שמופיעים בסדר קבוע, כאשר כל שדה מכיל מידע שדרוש כדי לבצע את ההתאמה בין כתובת IP לכתובת MAC.

בתחילת ההודעה נמצא השדה **Hardware Type**, שמציין באיזה סוג רשת משתמשים. כאשר ARP משמש ברשת Ethernet, השדה מציין שהרשת היא מסוג Ethernet.

אחריו נמצא השדה **Protocol Type**, שמציין עבור איזה פרוטוקול רשת מתבצעת ההתאמה. כאשר משתמשים ב ARP עבור IPv4, השדה מציין שהכתובת שאיתה עובדים היא כתובת IPv4. לאחר מכן מופיע השדה **Hardware Length**, שמציין את גודל כתובת החומרה. ברשת Ethernet כתובת החומרה היא כתובת MAC שגודלה 6 בתים.

אחריו נמצא **Protocol Length**, שמציין את גודל כתובת פרוטוקול הרשת. ב IPv4 גודל כתובת ה IP הוא 4 בתים.

לאחר השדות האלה נמצא השדה **Operation**, שמציין מה סוג הודעת ה ARP. בדרך כלל מדובר ב ARP Request כאשר המחשב מחפש כתובת MAC, או ב ARP Reply כאשר המחשב מחזיר את כתובת ה MAC המבוקשת.

בהמשך ההודעה נמצאות הכתובות של השולח והיעד. תחילה מופיעה **Sender MAC**, כלומר כתובת ה MAC של ההתקן ששולח את הודעת ה ARP, ולאחר מכן **Sender IP**, שהיא כתובת ה IP של אותו התקן.

לאחר מכן מופיעות **Target MAC** ו **Target IP**, המתארות את ההתקן שאליו מתייחסת הבקשה. ב ARP Request כתובת ה IP של היעד כבר ידועה, אך כתובת ה MAC שלו אינה ידועה ולכן שדה ה Target MAC אינו מכיל את הכתובת המבוקשת.

כך, לפי סדר השדות, הודעת ARP מכילה את המידע הדרוש כדי לזהות את סוג הרשת והפרוטוקול, לקבוע את סוג הבקשה ולאחר מכן להעביר את כתובות ה MAC וה IP של השולח והיעד.

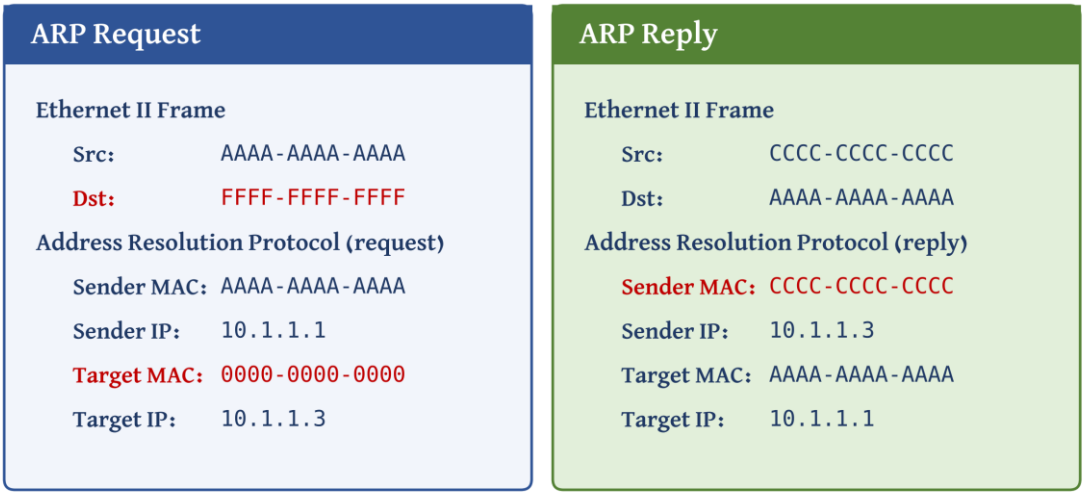
ARP ו Ethernet

ARP הוא פרוטוקול שעובד בתוך הרשת המקומית ונשלח בתוך Frame של Ethernet. כאשר נשלחת הודעת ARP Request, כתובת ה MAC של היעד היא בדרך כלל כתובת Broadcast:

FF:FF:FF:FF:FF:FF

המשמעות היא שה Frame נשלח לכל ההתקנים הנמצאים באותה רשת מקומית. כל התקן מקבל את הבקשה ובודק האם כתובת ה IP המבוקשת שייכת אליו.

לאחר שההתקן המתאים מחזיר ARP Reply, ניתן להשתמש בכתובת ה MAC שהתקבלה כדי ליצור את ה Ethernet Frame ולשלוח את המידע ליעד.



איור 32: Ethernet ו ARP

IPv4

IPv4, הגרסה הרביעית של Internet Protocol, פועל בשכבת הרשת ומשמש להעברת חבילות מידע בין התקנים ורשתות שונות. IPv4 אחראי על זיהוי המקור והיעד באמצעות כתובות IP ועל העברת החבילה דרך הרשת בהתאם לכתובת היעד.

כתובת IPv4 היא בגודל של **32 ביט** ומיוצגת בדרך כלל באמצעות ארבעה מספרים עשרוניים, כאשר כל מספר נמצא בטווח של 0 עד 255. לדוגמה:

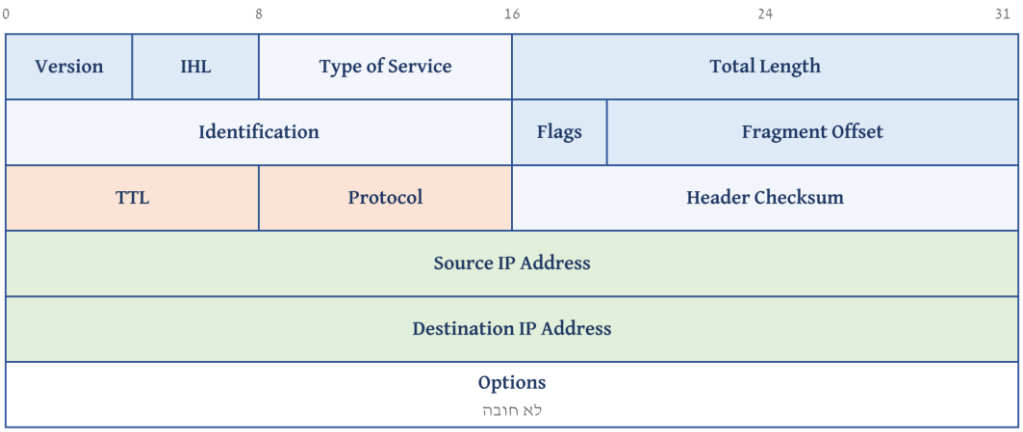
192.168.1.10

כתובת ה IP משמשת לזיהוי של התקן ברשת ולניתוב המידע אל היעד. כאשר המידע צריך לעבור בין רשתות שונות, הנתבים משתמשים בכתובת ה IP של היעד כדי לקבוע לאן להעביר את החבילה.

כיצד IPv4 עובד

כאשר מערכת רוצה להעביר מידע ברשת, המידע שמגיע משכבת התעבורה מוכנס לתוך חבילת IPv4. לנתונים מצורף **IPv4 Header**, המכיל את המידע הדרוש להעברת החבילה.

בחבילת IPv4 קיימות בין היתר כתובת ה IP של המקור וכתובת ה IP של היעד. כתובת המקור מציינת מאיזה התקן החבילה נשלחה, וכתובת היעד מציינת לאיזה התקן היא מיועדת.



כל שורה היא 32 ביט, ובלי Options הכותרת היא בגודל 20 בתים

איור 33: IPv4 Header

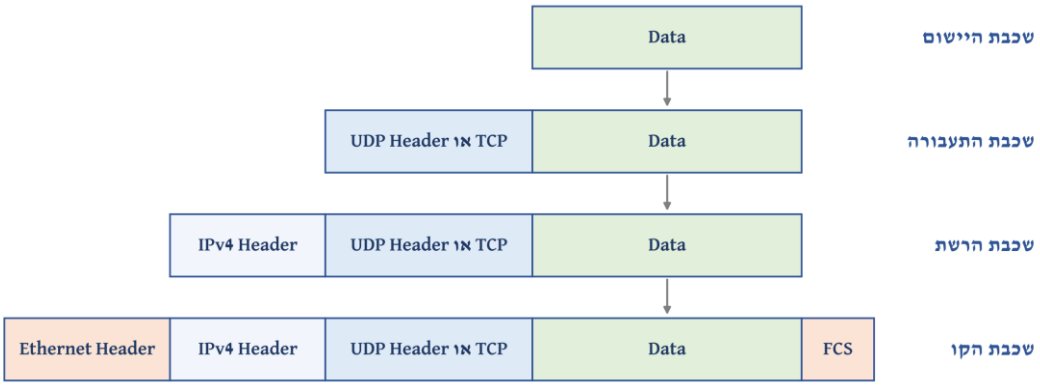
כאשר החבילה מגיעה לנתב, הנתב בודק את כתובת היעד ומחליט לאיזה ממשק או נתיב להעביר את החבילה. התהליך יכול להימשך דרך מספר נתבים עד שהחבילה מגיעה לרשת שבה נמצא היעד.

IPv4 אחראי על העברת החבילה בין הרשתות, אך הוא אינו מבטיח שהחבילה אכן תגיע ליעד או שהחבילות יגיעו בסדר מסוים. מנגנוני האמינות, כאשר הם נדרשים, יכולים להתבצע בשכבות אחרות כמו TCP.

IPv4 ו TCP או UDP

IPv4 עובד יחד עם פרוטוקולי שכבת התעבורה כמו TCP ו UDP. מקבלים את המידע מהיישום ומוסיפים את ה Header שלהם. לאחר מכן המידע מועבר אל IPv4, שמוסיף את ה Header שלו ויוצר את חבילת ה IP.

ניתן לתאר את המבנה כך:



איור 34: עיטוף המידע

לאחר מכן חבילת ה IPv4 עוברת לשכבת קישור הנתונים, שם היא מוכנסת לתוך Ethernet Frame ומועברת ברשת המקומית.

כך כל שכבה מוסיפה את המידע הדרוש לתפקיד שלה ומאפשרת להעביר את המידע בצורה מסודרת מהיישום ועד להתקן היעד.

IPv4 בקצרה

IPv4 הוא פרוטוקול מרכזי בשכבת הרשת, שמשתמש בכתובות IP בגודל של 32 ביט כדי לזהות את המקור ואת היעד. חבילת IPv4 כוללת Header עם מידע כמו כתובות IP, TTL,

סוג הפרוטוקול ואורך החבילה, ולאחריו נמצא המידע עצמו. IPv4 אינו אחראי על אמינות התקשורת או על שמירת סדר המידע, ולכן כשנדרשת תקשורת אמינה משתמשים ב TCP, וכשנדרשת תקשורת פשוטה ומהירה יותר משתמשים ב UDP.

TCP ו UDP

TCP פועל בשכבת התעבורה ומעביר מידע בצורה אמינה ומסודרת בין שני התקנים. השם המלא שלו הוא Transmission Control Protocol. TCP דואג לכך שהמידע יגיע ליעד בצורה תקינה, לפי הסדר, ובמקרה של אובדן נתונים ניתן יהיה לשלוח אותם מחדש.

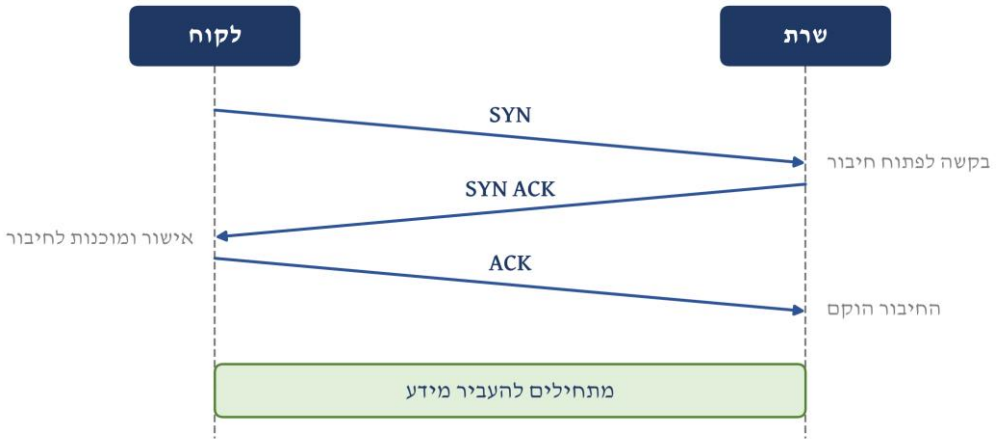
TCP משתמש במספרי **Port** כדי לזהות לאיזה יישום מיועד המידע. כך מספר יישומים יכולים להשתמש באותה תקשורת במקביל, כאשר לכל תקשורת יש את מספר הפורט המתאים.

כיצד TCP עובד

לפני תחילת העברת המידע, TCP צריך להקים חיבור בין שני הצדדים. תהליך זה נקרא **Three Way Handshake**, והוא מאפשר לשני הצדדים לתאם את תחילת התקשורת.

בתחילה הלקוח שולח הודעת **SYN** לשרת כדי לבקש לפתוח חיבור ולסנכרן את מספרי ה Sequence. השרת מחזיר הודעת **SYN ACK**, אשר מאשרת שקיבל את הבקשה וגם מציינת שהוא מוכן להקים את החיבור. לאחר מכן הלקוח שולח הודעת **ACK**, ובכך החיבור נחשב כמוקם ושני הצדדים יכולים להתחיל להעביר מידע.

התהליך מתבצע בצורה הבאה:



איור 35: Three Way Handshake

לאחר יצירת החיבור, TCP מתחיל להעביר את הנתונים. לכל חלק מהמידע מוקצה **Sequence Number**, אשר מאפשר ל TCP לזהות את סדר הנתונים ולוודא שהם מגיעים בצורה הנכונה.

כאשר הצד המקבל מקבל את המידע, הוא שולח **ACK** כדי לאשר שהנתונים התקבלו. אם חלק מהמידע לא הגיע או שהתקבלה בעיה במהלך ההעברה, TCP יכול לזהות זאת ולבצע **Retransmission**, כלומר לשלוח את המידע מחדש.

באופן זה TCP מספק תקשורת אמינה יותר, במיוחד במקרים שבהם חשוב לקבל את כל המידע ובסדר הנכון.

מבנה TCP Header

לכל מקטע TCP מצורף Header המכיל את המידע הדרוש לניהול התקשורת.

בתחילת ה Header נמצא **Source Port**, המציין את מספר הפורט של היישום ששולח את המידע.

אחריו נמצא **Destination Port**, המציין את הפורט של היישום שאליו המידע מיועד.

לאחר מכן נמצאים **Sequence Number** ו **Acknowledgment Number**, המשמשים לניהול סדר המידע ואישור קבלת הנתונים.

בהמשך קיימים שדות בקרה, כאשר בין החשובים נמצאים הדגלים **SYN, ACK** ו **FIN**, המשמשים בין היתר להקמת החיבור ולסגירתו.

בנוסף קיימים שדות כמו **Window Size**, המשמש לשליטה בכמות המידע שניתן להעביר לפני קבלת אישור, ו **Checksum**, המשמש לבדיקת תקינות המידע.

UDP

גם **UDP**, User Datagram Protocol, פועל בשכבת התעבורה, והוא מעביר מידע בין יישומים. בניגוד ל TCP, פרוטוקול UDP אינו מקים חיבור לפני תחילת העברת המידע ואינו מספק מנגנונים מובנים להבטחת הגעת הנתונים או לסידור שלהם.

UDP משתמש גם הוא במספרי **Port**, ולכן הוא מאפשר לזהות לאיזה יישום מיועד המידע.

כיצד UDP עובד

ב UDP ניתן להתחיל להעביר מידע ללא ביצוע של Three Way Handshake. היישום מוסר את המידע ל UDP, ה UDP מוסיף את ה Header שלו והמידע ממשיך לשכבות שמתחתיו.

UDP אינו ממתין לאישור קבלה מהצד השני ואינו מבצע שליחה מחדש של מידע שאבד. לכן אם Datagram מסוים לא הגיע ליעד, UDP עצמו אינו מבצע פעולה לתיקון הבעיה.

היתרון בכך הוא שמבנה הפרוטוקול פשוט יותר והתקורה של התקשורת נמוכה יותר. הדבר מאפשר זמן תגובה מהיר ומתאים ליישומים שבהם העברת המידע בזמן חשובה יותר מהבטחה שכל המידע יגיע.

מבנה UDP Header

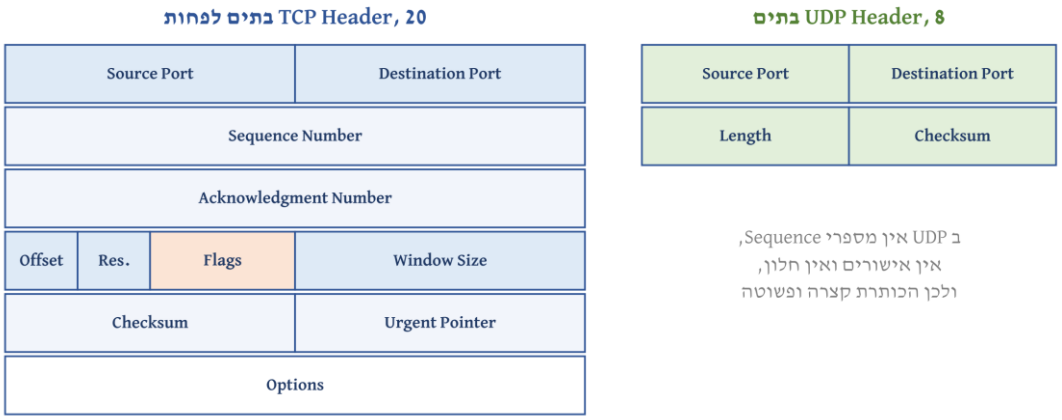
UDP Header פשוט יחסית ומכיל מספר שדות עיקריים.

בתחילת ה Header נמצא **Source Port**, המציין את הפורט שממנו נשלח המידע, ולאחריו **Destination Port**, המציין את הפורט שאליו המידע מיועד.

בהמשך נמצא **Length**, המציין את אורך הודעת ה UDP, ולאחר מכן **Checksum**, המשמש לבדיקת תקינות המידע.

לאחר ה Header נמצא המידע עצמו שהיישום רוצה להעביר.

כך נראים שני ה Headers זה ליד זה:



איור 36: TCP Header מול UDP Header

TCP לעומת UDP

TCP ו UDP משמשים שניהם בשכבת התעבורה, אך צורת העבודה שלהם שונה.

TCP מבצע יצירת חיבור, משתמש ב Sequence Numbers וב ACK, שומר על סדר המידע ויכול לבצע שליחה מחדש במקרה של אובדן.

UDP אינו מבצע יצירת חיבור ואינו מספק מנגנונים מובנים לשליחה מחדש או לשמירה על סדר המידע. לכן הוא פשוט ומהיר יותר מבחינת מנגנון התקשורת.

TCP מתאים כאשר חשוב שהמידע יגיע בצורה מלאה ומסודרת, לדוגמה בהעברת קבצים ובתקשורת שבה אובדן נתונים אינו רצוי.

UDP מתאים כאשר חשוב יותר לקבל מידע במהירות ובזמן תגובה נמוך, לדוגמה בתקשורת בזמן אמת, משחקי רשת ושירותים שבהם ניתן להתמודד עם אובדן של חלק מהמידע.

בסופו של דבר, הבחירה בין TCP ל UDP תלויה בדרישות של היישום ובחשיבות של אמינות המידע לעומת מהירות וזמן תגובה.

דחיסת נתונים

תקני דחיסה ברשת – RFC 3173 ו IPComp

אלגוריתם LZ77

אלגוריתם הופמן (Huffman)

תקני דחיסה ברשת – RFC 3173 ו IPComp

במהלך העברת מידע ברשת, הנתונים שנשלחים בין מחשבים יכולים להיות גדולים. אחת הדרכים לצמצם את כמות המידע המועברת היא באמצעות דחיסת נתונים.

דחיסת נתונים היא תהליך שבו מנסים לייצג את המידע בצורה יעילה יותר, כך שניתן יהיה להעביר כמות קטנה יותר של נתונים ברשת. לאחר שהמידע מגיע ליעד, ניתן לבצע תהליך הפוך של פתיחת הדחיסה ולהחזיר את הנתונים לצורתם המקורית.

אחד התקנים שנועדו להגדיר דחיסה הוא RFC 3173.

RFC 3173

RFC הוא מסמך טכני שמתאר תקנים, פרוטוקולים או מנגנונים הקשורים לאינטרנט ולתקשורת ברשתות. RFC 3173 מגדיר את פרוטוקול IPComp, שמטרתו לאפשר דחיסה של המידע שנמצא בתוך חבילת IP.

חשוב להבדיל בין IPComp לבין פרוטוקולים כמו TCP או TCP .IPv4 אחראי על אופן העברת המידע בצורה אמינה, בעוד IPv4 אחראי על כתובות וניתוב. IPComp מתמקד בדחיסת המידע כדי להקטין את גודל החבילות.

IP Payload Compression Protocol – IPComp

IPComp הוא פרוטוקול המאפשר לדחוס את ה Payload של חבילת IP לפני שליחתה ברשת. ה Payload הוא החלק בחבילת ה IP שמכיל את המידע שמועבר.

במחשב המקבל מתבצע תהליך של Decompression, כלומר פתיחת הדחיסה, כדי לקבל בחזרה את המידע המקורי.

המטרה היא לא לשנות את תוכן המידע עצמו, אלא לייצג אותו בצורה קטנה יותר בזמן ההעברה.

אלגוריתם הדחיסה

IPComp עצמו מגדיר את הדרך שבה ניתן להעביר מידע דחוס, אך פעולת הדחיסה עצמה מתבצעת באמצעות אלגוריתם דחיסה.

האלגוריתם מקבל את המידע המקורי ומנסה למצוא בו מבנים או רצפים שחוזרים על עצמם. במקום לשמור ולהעביר את אותו מידע שוב ושוב, ניתן לייצג אותו בצורה יעילה יותר.

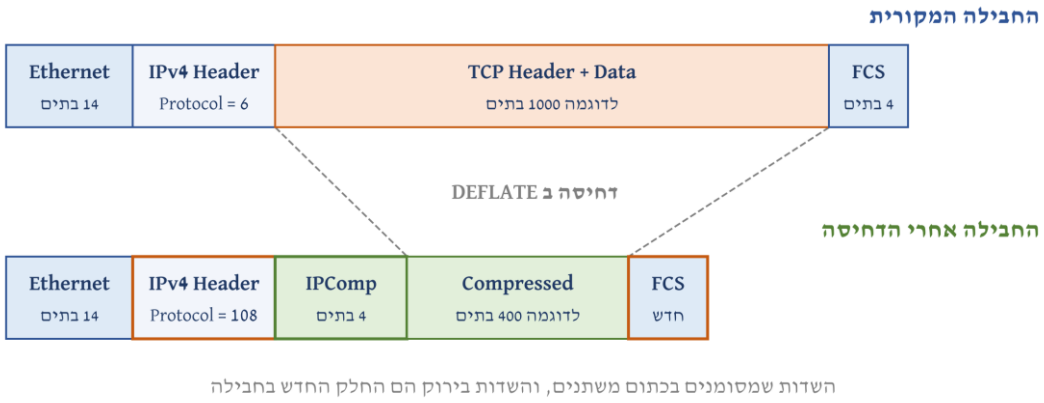
לדוגמה, אם במידע מסוים מופיע אותו רצף פעמים רבות, אלגוריתם דחיסה יכול להשתמש בייצוג מקוצר של אותו רצף. כאשר המידע מגיע ליעד, האלגוריתם מבצע את הפעולה ההפוכה ומחזיר את הנתונים למצבם המקורי.

חשוב שהשולח והמקבל ידעו באיזה אלגוריתם משתמשים, אחרת המקבל לא יוכל לפתוח את הדחיסה בצורה נכונה.

מבנה החבילה ב IPComp

IPComp לא מחליף את IPv4 או IPv6 ולא אחראי על ניתוב החבילה. הוא משנה רק את ה Payload ומוסיף לחבילה כותרת קטנה, וכך הצד המקבל יודע שה Payload דחוס ובאיזה אלגוריתם לפתוח אותו.

כך נראית חבילה לפני הדחיסה ואחריה:



איור 37: חבילה לפני ואחרי IPComp

בזמן הדחיסה נוסף לחבילה IPComp Header בגודל 4 בתים. בתוכו נמצא שדה ה Next Header, שבו נשמר הפרוטוקול המקורי של החבילה, ומזהה הדחיסה CPI.

בשדה ה Protocol של ה IPv4 Header נכתב הערך 108, שדה ה Total Length וה Checksum מתעדכנים לפי הגודל החדש של החבילה, ובסוף מחושב FCS חדש, CRC-32, למסגרת ה Ethernet.

למה צריך דחיסה ברשת?

הסיבה העיקרית לשימוש בדחיסת מידע היא חיסכון במשאבי הרשת.

כאשר גודל המידע קטן יותר, נדרש פחות רוחב פס כדי להעביר אותו. במקרים מסוימים הדבר יכול גם להפחית את זמן ההעברה ואת כמות התעבורה ברשת.

הדבר יכול להיות משמעותי במיוחד ברשתות שבהן רוחב הפס מוגבל או כאשר יש כמות גדולה של מידע שעוברת בין התקנים.

לדוגמה, אם מערכת מסוימת מעבירה כמות גדולה של נתונים באופן קבוע, אפילו הקטנה של גודל הנתונים יכולה לחסוך כמות משמעותית של תעבורת רשת לאורך זמן.

עם זאת, דחיסה דורשת גם משאבי עיבוד. המחשב צריך לבצע פעולת דחיסה בצד השולח ופתיחת דחיסה בצד המקבל. לכן השימוש בדחיסה אינו תמיד משתלם, ויש לבדוק האם החיסכון בתעבורה מצדיק את זמן העיבוד הנוסף.

דחיסה לעומת הצפנה

חשוב להבדיל בין דחיסת מידע לבין הצפנת מידע.

דחיסה נועדה להקטין את גודל הנתונים ולייעל את העברתם. היא אינה מיועדת להגן על המידע מפני קריאה.

הצפנה, לעומת זאת, נועדה להפוך את המידע לבלתי קריא למי שאין לו את המפתח המתאים.



איור 38: דחיסה לעומת הצפנה

לכן IPComp אינו מנגנון אבטחה בפני עצמו. הוא עוסק ביעילות העברת המידע ולא בסודיות שלו.

אלגוריתם LZ77

LZ77 הוא אלגוריתם דחיסה ללא איבוד מידע שפותח בשנת 1977 על ידי אברהם למפל ויעקב זיו, ומכאן גם השם שלו. זהו החלק הראשון בשיטת DEFLATE שבה אני משתמש בפרויקט, ורק אחריו מגיע קידוד הופמן.

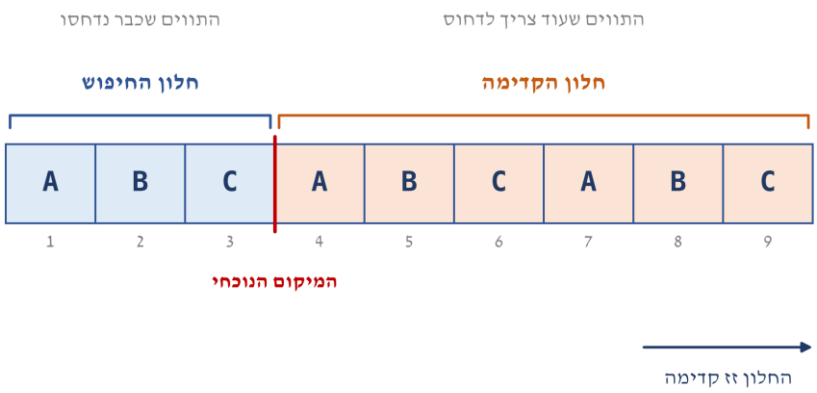
הרעיון המרכזי של LZ77

במידע אמיתי, כמו קבצי טקסט או חבילות רשת, יש הרבה רצפים שחוזרים על עצמם. LZ77 מחפש את החזרות האלה, ובמקום לשמור את אותו רצף פעם נוספת הוא שומר הפניה לרצף שכבר הופיע קודם.

ההפניה בנויה משני מספרים, מרחק ואורך. המרחק אומר כמה תווים צריך לחזור אחורה כדי להגיע לתחילת הרצף, והאורך אומר כמה תווים להעתיק משם.

חלון ההזזה

כדי למצוא חזרות, LZ77 עובד עם חלון שזז לאורך המידע. החלון מתחלק לשני חלקים.



איור 39: חלון ההזזה של LZ77

חלון החיפוש מכיל את התווים שכבר עברו דחיסה, ושם האלגוריתם מחפש התאמה. חלון הקדימה מכיל את התווים שעוד צריך לדחוס.

בכל שלב האלגוריתם בודק מה הרצף הארוך ביותר בתחילת חלון הקדימה שכבר מופיע בחלון החיפוש. אם נמצאה התאמה, הוא כותב הפניה. אם לא נמצאה, הוא כותב את התו כמו שהוא. אחרי זה החלון זז קדימה והבדיקה ממשיכה.

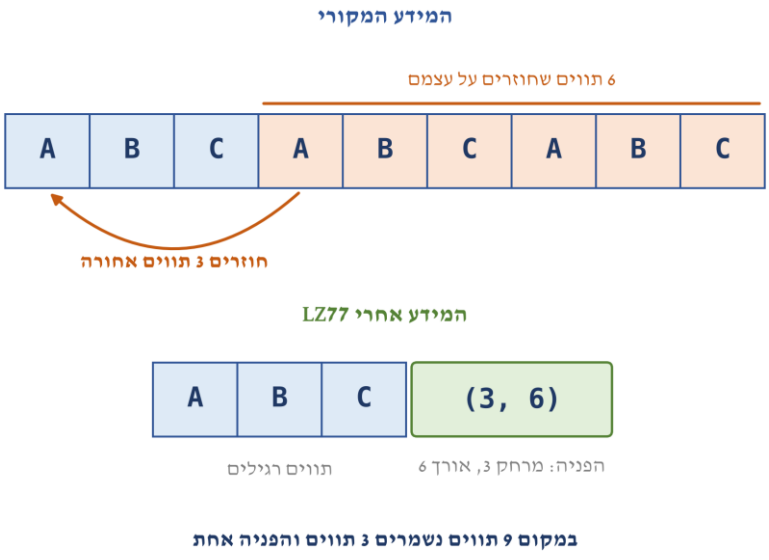
ב DEFLATE חלון החיפוש יכול להגיע עד 32KB, כך שהאלגוריתם יכול למצוא חזרות גם במרחק גדול.

דוגמה פשוטה

נניח שיש לנו את המידע הבא:

ABCABCABC

שלושת התווים הראשונים, A, B ו C, מופיעים בפעם הראשונה ולכן נכתבים כמו שהם. מהתו הרביעי והלאה כל המידע הוא חזרה על ABC.



איור 40: דוגמה ל LZ77

במקום לכתוב את ששת התווים האחרונים, LZ77 כותב הפניה עם מרחק 3 ואורך 6. כלומר, צריך לחזור 3 תווים אחורה ולהעתיק משם 6 תווים.

בדוגמה הזו האורך גדול מהמרחק, וזה אפשרי כי ההעתקה מתבצעת תו אחרי תו, כך שכל תו שהועתק כבר יכול לשמש כמקור לתו הבא.

פתיחת הדחיסה

הפתיחה של LZ77 פשוטה ומהירה יותר מהדחיסה. הצד המקבל קורא את המידע לפי הסדר, כל תו רגיל נכתב כמו שהוא, וכל הפניה מתורגמת להעתקה של תווים מתוך מה שכבר נכתב.

בניגוד לדחיסה, בפתיחה לא צריך לחפש התאמות, ולכן אין צורך לשמור עץ או טבלה. מספיק לזכור את התווים האחרונים שנכתבו.

LZ77 בפרויקט

בפרויקט זה LZ77 הוא השלב הראשון במאיץ הדחיסה שב FPGA. הוא מקבל את ה Payload של החבילה, מחליף את הרצפים החוזרים בהפניות, ומעביר את התוצאה לקידוד הופמן.

החיפוש אחרי התאמות הוא החלק שלוקח הכי הרבה זמן בדחיסה, ולכן הוא מתאים במיוחד לחומרה. ב FPGA אפשר להשוות כמה מיקומים בחלון החיפוש באותו מחזור שעון, במקום לבדוק אותם אחד אחרי השני כמו במעבד.

אלגוריתם הופמן (Huffman)

אלגוריתם הופמן הוא אלגוריתם לדחיסת מידע ללא איבוד מידע.

מטרת האלגוריתם היא להקטין את כמות המקום הדרושה לשמירת מידע, באמצעות שימוש בקודים קצרים יותר עבור תווים שמופיעים בתדירות גבוהה, וקודים ארוכים יותר עבור תווים שמופיעים בתדירות נמוכה.

הרעיון המרכזי של אלגוריתם הופמן

הרעיון המרכזי באלגוריתם הוא שלא כל התווים במידע מופיעים באותה תדירות.

לדוגמה, בטקסט מסוים יכול להיות שתו אחד מופיע מאות פעמים, בעוד שתו אחר מופיע רק מספר פעמים. אם כל התווים יקבלו אותו מספר ביטים, יתבזבז מקום על תווים שמופיעים בתדירות גבוהה.

אלגוריתם הופמן פותר את הבעיה הזאת באמצעות קוד באורך משתנה.

לתווים שמופיעים הרבה פעמים ניתן קוד קצר, ולתווים שמופיעים מעט פעמים ניתן קוד ארוך יותר.

כך ניתן להקטין את מספר הביטים הכולל הדרוש לייצוג המידע.

קוד הופמן

אלגוריתם הופמן משתמש בקודים בינאריים, כלומר קודים המורכבים מהספרות 0 ו 1.

לכל תו מוקצה קוד בינארי ייחודי. הקודים אינם חייבים להיות באותו אורך.

לדוגמה אם במידע מסוים התו A מופיע בתדירות גבוהה מאוד, ניתן לתת לו קוד קצר כמו 0.

לעומת זאת לתו שמופיע לעיתים רחוקות ניתן קוד ארוך יותר, לדוגמה 1101.

המשמעות היא שהתווים הנפוצים מיוצגים באמצעות פחות ביטים, ולכן ניתן לחסוך במקום.

כדי ליצור את הקודים, האלגוריתם מתחיל בספירת מספר הפעמים שכל תו מופיע במידע.

לאחר מכן יוצרים רשימה של כל התווים יחד עם התדירות שלהם.

האלגוריתם מחפש בכל פעם את שני התווים או הצמתים בעלי התדירות הנמוכה ביותר ומאחד אותם לצומת חדש.

התדירות של הצומת החדש היא הסכום של שתי התדירויות שחוברו.

התהליך ממשיך עד שכל התווים מחוברים לעץ אחד שנקרא Huffman Tree.

לאחר בניית העץ, ניתן לעבור מהשורש אל כל תו וליצור עבורו קוד בינארי.

בדרך כלל מעבר לצד אחד בעץ מסומן ב 0 ומעבר לצד השני מסומן ב 1.

כך לכל תו מתקבל קוד ייחודי.

עץ הופמן

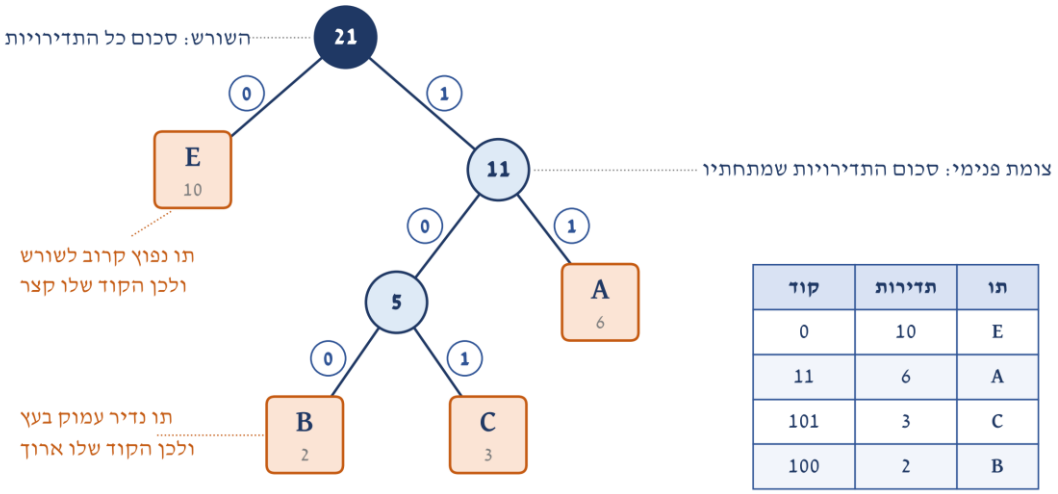
עץ הופמן הוא מבנה הנתונים שבו משתמש האלגוריתם כדי ליצור את הקודים הבינאריים.

בתחתית העץ נמצאים התווים עצמם, ובכל צומת נשמרת תדירות הופעת התווים שנמצאים מתחתיו.

כאשר עוברים מהשורש לכיוון תו מסוים, אוספים את הביטים שהתקבלו בדרך. רצף הביטים הזה הוא הקוד של אותו תו.

תווים שמופיעים בתדירות גבוהה ממוקמים בדרך כלל קרוב יותר לשורש ולכן מקבלים קוד קצר יותר.

תווים שמופיעים בתדירות נמוכה נמצאים בדרך כלל עמוק יותר בעץ ולכן מקבלים קוד ארוך יותר.



תו	תדירות	קוד
E	10	0
A	6	11
C	3	101
B	2	100

הביטים 0 ו 1 שעל הקווים נאספים בדרך מהשורש אל התו

איור 41: עץ הופמן

דוגמה פשוטה

נניח שיש לנו את המידע הבא:

AAAAABBC

התדירות של התווים היא:

A מופיע 5 פעמים.

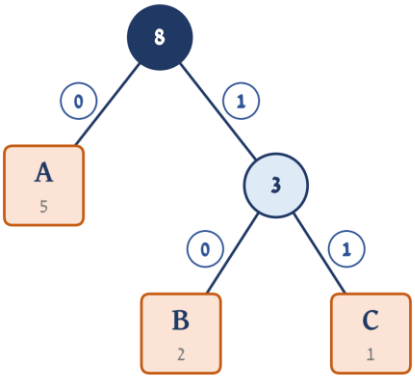
B מופיע 2 פעמים.

C מופיע פעם אחת.

AAAAABBC

תו	תדירות	קוד	ביטים במידע
A	5	0	5 × 1 = 5
B	2	10	2 × 2 = 4
C	1	11	1 × 2 = 2
סך הכול	8		11 ביטים

בקוד באורך קבוע של 2 ביטים לכל תו
המידע תופס 8 × 2 = 16 ביטים



שלב 1: מחברים את שני התווים הנדירים, C ו B, לצומת 3.

שלב 2: מחברים את הצומת 3 עם A, והשורש מקבל 8.

איור 42: דוגמה לעץ הופמן

מכיוון שהתו A מופיע בתדירות הגבוהה ביותר, האלגוריתם ישאף לתת לו קוד קצר יותר.

לעומת זאת, התו C מופיע פעם אחת בלבד ולכן יקבל קוד ארוך יותר.

לדוגמה, קוד אפשרי יכול להיות:

A = 0

B = 10

C = 11

במקום להשתמש באותו מספר ביטים עבור כל תו, התו הנפוץ A מיוצג באמצעות ביט אחד בלבד.

כאשר מקודדים את המידע באמצעות הקודים האלה, מספר הביטים הכולל יכול להיות קטן יותר לעומת שיטה שבה לכל תו יש קוד באורך קבוע.

בטקסט אחר הקודים יהיו שונים, כי הם תלויים בתדירויות של התווים ובמבנה עץ הופמן שנוצר.

תכונה חשובה: Prefix Code

אחת התכונות החשובות של קוד הופמן היא שאף קוד של תו אחד אינו מהווה התחלה של קוד של תו אחר.

תכונה זו נקראת Prefix Free Code.

לדוגמה, אם לתו מסוים יש את הקוד 0, לא ניתן לתת לתו אחר קוד שמתחיל ב 0.

הדבר חשוב מכיוון שהוא מאפשר למערכת להבין היכן מסתיים קוד של תו אחד ומתחיל הקוד של התו הבא, גם כאשר אורך הקודים שונה.

בזכות תכונה זו ניתן לפענח את המידע ללא צורך בסימני הפרדה בין התווים.

פתיחת הדחיסה

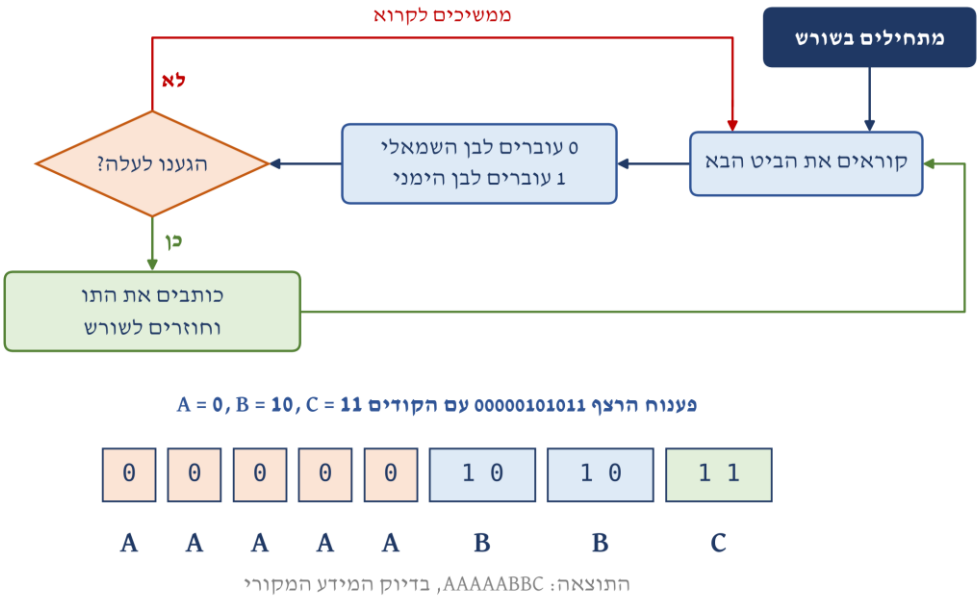
כדי לפתוח מידע שנדחס באמצעות הופמן, המקבל צריך לדעת את עץ הופמן או מידע אחר שמאפשר לו לבנות מחדש את הקודים.

הפענוח מתבצע באמצעות מעבר בעץ לפי הביטים שהתקבלו.

כאשר מתקבל 0 עוברים לכיוון אחד בעץ, וכאשר מתקבל 1 עוברים לכיוון השני.

כאשר מגיעים לתו בעץ, יודעים שזהו התו הבא במידע המקורי וחוזרים לשורש כדי לפענח את הביטים הבאים.

בגלל שהקודים הם Prefix Free, ניתן לפענח את המידע בצורה חד משמעית.



איור 43: פתיחת הדחיסה

למה הופמן חוסך מקום?

החיסכון של אלגוריתם הופמן נובע מכך שהתווים הנפוצים מקבלים קודים קצרים יותר.

אם תו מסוים מופיע מספר גדול של פעמים, אפילו חיסכון של ביט אחד בכל הופעה שלו יכול ליצור חיסכון משמעותי בגודל המידע הכולל.

לעומת זאת, תווים נדירים מקבלים קודים ארוכים יותר, אך מכיוון שהם מופיעים מעט פעמים, ההשפעה שלהם על הגודל הכולל קטנה יותר.

לכן השיטה מתאימה במיוחד למידע שבו יש הבדל משמעותי בתדירות ההופעה של התווים.

דחיסה ללא איבוד מידע

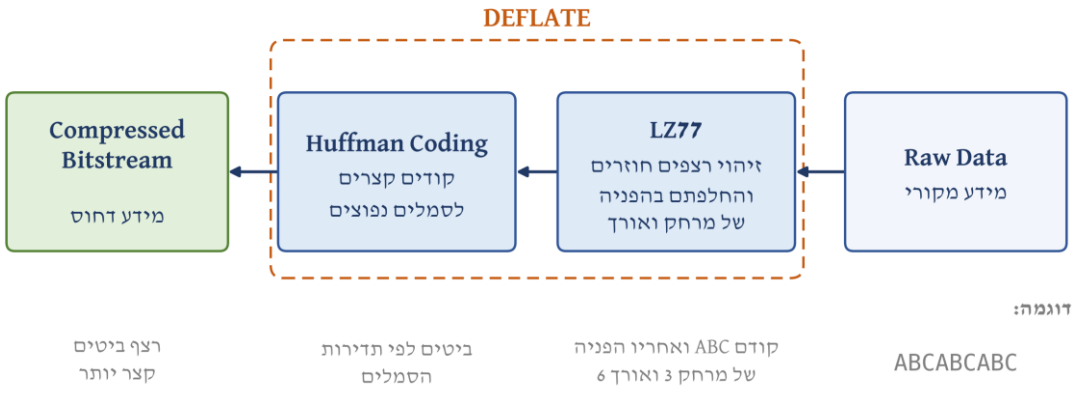
אלגוריתם הופמן הוא אלגוריתם דחיסה ללא איבוד מידע.

המשמעות היא שהמידע לאחר פתיחת הדחיסה זהה לחלוטין למידע המקורי.

היתרון המרכזי של אלגוריתם הופמן הוא האפשרות להקטין את גודל המידע ללא איבוד נתונים.

אלגוריתם הופמן נמצא בשימוש במגוון שיטות דחיסה והוא משמש גם כחלק מאלגוריתמי דחיסה מורכבים יותר.

אחת הדוגמאות המוכרות היא שיטת DEFLATE, שבה נעשה שימוש בפרויקט, והיא משלבת את LZ77 יחד עם קידוד הופמן.



איור 44: DEFLATE

LZ77 והופמן הם שני אלגוריתמים שונים, אך שניהם משמשים לדחיסה ללא איבוד מידע. LZ77 מתמקד בזיהוי רצפים שחוזרים על עצמם, ובמקום לשמור את אותו רצף מספר פעמים הוא משתמש בהפניה לרצף שכבר הופיע. הופמן, לעומת זאת, מתמקד בתדירות ההופעה של תווים, ונותן קודים קצרים לתווים שמופיעים הרבה.

ב DEFLATE משתמשים קודם ב LZ77 כדי לזהות חזרות, ולאחר מכן בקידוד הופמן כדי לייצג את התוצאה בצורה יעילה יותר. כך מתקבלת דחיסה יעילה יותר מאשר בכל אחת מהשיטות בנפרד.

תוכנה, קוד החומרה

פרק זה יושלם בהגשה הסופית

טבלת הסבר קוד החומרה

פרק זה יושלם בהגשה הסופית

מפרט טכני של כלל הרכיבים

בטבלה מרוכזים כל הרכיבים שמהם בניתי את המערכת, עם הכמות של כל רכיב, המאפיינים הכלליים שלו והמפרט הטכני העיקרי.

טבלה 7: מפרט טכני של כלל הרכיבים

שם הרכיב	כמות	מאפיינים כלליים	מפרט טכני כללי
כרטיס פיתוח DE10-Nano של חברת Terasic	1	כרטיס פיתוח עם שבב Cyclone V SoC, שמשלב בתוכו HPS ו FPGA	מעבד ARM Cortex-A9 עם שתי ליבות בתדר 800MHz, 110K תאים לוגיים, זיכרון DDR3 של 1GB, שני מחברי GPIO של 40 פינים, מתח הפעלה 5V
כרטיס PHY עם הרכיב KSZ8061 של חברת Microchip	2	רכיב PHY של Ethernet שמתחבר ל MAC בממשק RMII	מהירות 10 או 100Mbps, שעון ייחוס של 50MHz, ניהול דרך MDC ו MDIO, מתח הפעלה 3.3V
מחבר RJ45	2	מחבר פיזי לכבל הרשת, מורכב על כרטיס ה PHY	שמונה מגעים, ארבעה זוגות של חוטים שזורים
מייצב מתח PTR08060W של חברת Texas Instruments	1	מודול ייצוב מתח שמספק 3.3V לשני כרטיסי ה PHY	ממיר ממותג יורד, מתח כניסה 4.5V עד 14V, מתח יציאה 0.6V עד 5.5V, זרם עד 6A, יעילות עד 96%, תדר מיתוג 300kHz
קבלים CI ו CO	2	קבלי סינון בכניסה וביציאה של המייצב	100uF כל אחד
נגד RSET	1	קובע את מתח היציאה של המייצב ל 3.3V	432 אוהם, דיוק של 1%, הספק 0.05W
שקע כניסת מתח DC	1	כניסת המתח ללוח המטריצה	5V
מתאם מתח	1	מספק מתח לכרטיס הפיתוח	5V
כרטיס microSD	1	מכיל את מערכת ההפעלה של ה HPS	שלוש מחיצות, A2, FAT ו EXT4
לוח מטריצה	1	הבסיס שעליו מורכבים כל רכיבי המערכת	לוח חורים להלחמה
עמודי תמיכה מתכתיים	4	מחזיקים את לוח המטריצה מעל השולחן	ללא
חוטי חיבור	לפי הצורך	מחברים בין מחבר GPIO_1 לבין כרטיסי ה PHY	ללא

רפלקציה

כשהתחלתי את הפרויקט כמעט לא הכרתי את רוב הנושאים שהוא דורש. ידעתי מה אני רוצה שהמערכת תעשה, אבל לא ידעתי איך עובד FPGA, מה קורה בתוך כרטיס רשת או איך בונים מערכת הפעלה לכרטיס פיתוח.

את החופש הגדול כולו הקדשתי ללמידה. רוב הימים עברו מול דפי נתונים, מסמכי RFC ומדריכים, ובניסיונות שהרבה פעמים לא הצליחו. היו ימים שבהם ישבתי שעות על שגיאה אחת בלי להתקדם, והיו רגעים שבהם חשבתי שלקחתי על עצמי יותר ממה שאני יכול. בימים האלה למדתי הכי הרבה, כי כדי לפתור תקלה הייתי צריך להבין מה באמת קורה מאחוריה.

לאט לאט הדברים התחילו להתחבר. הבנתי איך החבילה שאני רואה ב Wireshark קשורה לביטים שעוברים בקווי ה RMII, ואיך תוכנה שרצה על המעבד מדברת עם הלוגיקה שב FPGA. שמחתי מאוד כשהמערכת עלתה לראשונה על הכרטיס, אחרי שבועות של ניסיונות.

הפרויקט לימד אותי גם דברים שלא קשורים לטכנולוגיה. למדתי להתמודד עם תסכול, לחלק בעיה גדולה לשלבים קטנים ולבקש עזרה כשאני תקוע. הבנתי גם כמה עוד יש לי ללמוד בכל אחד מהנושאים שנגעתי בהם, ואני שמח שהפרויקט נתן לי את ההזדמנות להתחיל.

מבט קדימה

השלב הבא בפרויקט הוא להשלים את מסלול הדחיסה ב FPGA ולהריץ על המערכת המלאה את הבדיקות שתיארתי בפרק סביבת הבדיקות ואימות המערכת.

מעבר לזה יש שני כיוונים שבהם אפשר לפתח את המערכת. הראשון הוא מעבר ממשק RMII, שעובד בקצב של 100Mbps, לממשק RGMII שתומך בקצב של 1Gbps, יחד עם רכיב PHY שתומך במהירות הזו. השני הוא הוספה של מנוע הצפנה בחומרה ב FPGA, לצד מנוע הדחיסה, כך שאותה מערכת תחסוך ברוחב הפס וגם תשמור על אבטחת המידע שעובר ברשתות החיל.

מילון מונחים

בעמודים האלה ריכזתי את המונחים והקיצורים שמופיעים לאורך הספר, עם הסבר קצר על כל אחד מהם.

טבלה 8: מילון מונחים

מונח	הסבר
ARP	Address Resolution Protocol, פרוטוקול שמוצא את כתובת ה MAC שמתאימה לכתובת IP ברשת המקומית.
Avalon MM	Avalon Memory Mapped, ממשק שמאפשר לרכיבים בתוך ה FPGA לגשת זה לזה באמצעות כתובות, בדומה לגישה לזיכרון.
AXI	Advanced eXtensible Interface, ממשק תקשורת שדרכו ה HPS מגיע לרכיבים שב FPGA.
Buck	ממיר מתח ממותג יורד, שמדליק ומכבה את מתח הכניסה בתדר גבוה ומחליק אותו דרך סליל וקבלים. כזה הוא המייצב PTR08060W.
CRC	ערך בדיקה שהשולח מחשב ומכניס לשדה ה FCS, כדי שהמקלט יוכל לבדוק שהמסגרת הגיעה תקינה.
debootstrap	כלי שבונה מערכת קבצים בסיסית של Debian מאפס, ובעזרתו בניתי את ה RootFS.
DEFLATE	שיטת דחיסה שמשלבת את LZ77 עם קידוד הופמן.
Embedded Linux	מערכת ההפעלה Linux שרצה על מעבד משובץ, בפרויקט על ה HPS.
Ethernet	פרוטוקול התקשורת המרכזי ברשתות מקומיות, שפועל בעיקר בשכבה 2 של מודל OSI.
Fabric	אזור הלוגיקה המתכנתת של ה FPGA, שבו נמצאים הרכיבים והחיבורים שמגדירים בתכנון.
FCS	Frame Check Sequence, השדה בסוף מסגרת Ethernet שמשמש לבדיקת תקינות המידע.
FPGA	רכיב לוגיקה מתכנתת שבו אפשר לממש חומרה לפי התכנון.
FTP	File Transfer Protocol, פרוטוקול להעברת קבצים מעל TCP, עם חיבור בקרה בפורט 21 וחיבור נתונים נפרד.
Frame	מסגרת, יחידת המידע שעוברת ברשת Ethernet.
HPS	Hard Processor System, החלק בשבב שבו נמצא המעבד ורצה מערכת ה Linux.
Huffman	אלגוריתם דחיסה ללא איבוד מידע, שנותן קודים קצרים לתווים שמופיעים הרבה.
ICMP	Internet Control Message Protocol, פרוטוקול הודעות בקרה של IP, שבו משתמשת הפקודה ping.

מונח	הסבר
IPComp	IP Payload Compression Protocol, פרוטוקול לדחיסת ה Payload של חבילת IP, שמוגדר ב RFC 3173.
IPv4	פרוטוקול בשכבת הרשת שמעביר חבילות לפי כתובות IP בגודל 32 ביט.
LZ77	אלגוריתם דחיסה שמזהה רצפים שחוזרים על עצמם ומשתמש בהפניה לרצף שכבר הופיע.
MAC	Media Access Control, החלק שמטפל במסגרות Ethernet ומעביר אותן אל ה PHY. בפרויקט הוא ממומש ב FPGA.
mmap	פונקציה ב Linux שממפה אזור זיכרון לתוך התוכנית. דרכה תוכנית C ניגשת לכתובות של רכיבי החומרה ב FPGA.
MSEL	המתג SW10 על כרטיס ה DE10-Nano, שקובע איך ה FPGA מקבל את קובץ התכנון שלו בהדלקה.
MII	Media Independent Interface, ממשק דיגיטלי בין ה MAC לבין ה PHY עם ארבעה קווי נתונים.
OSI	מודל שמחלק את תהליך התקשורת ברשת לשבע שכבות.
Payload	החלק בחבילה שמכיל את המידע עצמו.
PHY	Physical Layer Transceiver, רכיב שממיר בין מידע דיגיטלי לבין האותות שעוברים בכבל.
Port	מספר שמזהה לאיזה יישום מיועד המידע.
QEMU	אמולטור שמאפשר להריץ במחשב x86 פקודות של מעבד ARM, ושימש אותי בבניית ה RootFS.
RFC	מסמך טכני שמתאר תקנים ופרוטוקולים של האינטרנט.
RJ45	המחבר הפיזי עם שמונה מגעים שמחבר את כבל הרשת.
RMII	Reduced Media Independent Interface, ממשק בין ה MAC לבין ה PHY עם שני קווי נתונים ושעון של 50MHz.
SoC	System on Chip, שבב שמשלב בתוכו מעבד ולוגיקה מתכנתת.
TCP	פרוטוקול בשכבת התעבורה שמעביר מידע בצורה אמינה ומסודרת.
U-Boot	מנהל האתחול של המערכת. התוכנה הראשונה שרצה כשהכרטיס נדלק, מכינה את החומרה, צורבת את ה FPGA וטוענת את Linux.
UDP	פרוטוקול פשוט ומהיר בשכבת התעבורה, בלי יצירת חיבור ובלי שליחה מחדש.
Wireshark	תוכנה שלוכדת את החבילות שעוברות בכרטיס הרשת ומציגה כל חבילה לפי השכבות שלה.

רשימה ביבליוגרפית/ רשימת מקורות

מס׳	תיאור המקור	קישור / שם כותב / שם הספר
1.	תקן התקשורת IETF RFC 3173 - IP Payload Compression Protocol	https://www.rfc-editor.org/rfc/rfc3173
2.	IPComp	https://en.wikipedia.org/wiki/IP_Payload_Compression_Protocol
3.	אלגוריתם הופמן (Huffman Coding) לדחיסת נתונים - רקע תיאורטי ומימוש	https://courses.cs.tau.ac.il/0368-2158/07b/tirgul_notes/tirgul_presentations/recitation10huffman.pdf
4.	מדריכים (כולל YouTube) לפיתוח בסביבת Embedded Linux ותכנות בC/Verilog-	https://www.youtube.com/watch?v=zTK2ZQdXhPs&list=PLiqmqQ7XKuf51Gexjk3wAB_0ljP47EymZ&index=2
5.	המאגר שלי ב GitHub: מדריך בניית Embedded Linux ל DE10-Nano	https://github.com/yashfe-cohen/DE10_Nano_FPGA_Project/tree/master/Embedded-Linux
6.	קוד המקור של U-Boot	https://github.com/u-boot/u-boot
7.	קוד המקור של Linux עבור SoC FPGA של Altera	https://github.com/altera-opensource/linux-socfpga
8.	דפי נתונים (Datasheets) של רכיבי התקשורת ורכיבי הPHY-, הרכיב KSZ8061 של חברת Microchip	https://www.radiolocman.com/datasheet/data.html?di=477767
9.	דפי נתונים ותיעוד רשמי של חברת Intel/Altera עבור לוח הפיתוח DE10-Nano	https://ftp.intel.com/Public/Pub/fpgaup/pub/Intel_Material/Boards/DE10-Nano/DE10_Nano_User_Manual.pdf
10.	דף נתונים של מייצב המתח PTR08060W של חברת Texas Instruments	https://www.ti.com/product/PTR08060W
11.	מימוש IPComp במערכת ההפעלה Linux	https://googlers.googlesource.com/maze/linux/+/v3.9/net/xfrm/xfrm_ipcomp.c
12.	Intel QuickAssist Technology, הספרייה QATzip לדחיסה בחומרה	https://github.com/intel/QATzip
13.	דף הנתונים המלא של מייצב המתח PTR08060W (Texas Instruments, SLTS289G), כולל ערך RSET והקבלים	https://www.ti.com/lit/ds/symlink/ptr08060w.pdf
14.	מדריך התחלה לכרטיס DE10-Nano של Terasic ו Intel, כולל מתג ה MSEL	https://www.intel.com/content/www/us/en/developer/articles/guide/terasic-de10-nano-get-started-guide.html
15.	Cyclone V Hard Processor System Technical Reference Manual, הגשר Lightweight HPS to FPGA	https://www.intel.com/content/www/us/en/docs/programmable/683126/21-2/functional-description-of-the-lightweight.html

מס׳	תיאור המקור	קישור / שם כותב / שם הספר
16.	A Universal .(1977) .Lempel, A ,.Ziv, J Algorithm for Sequential Data Compression, IEEE Transactions on Information Theory, 23(3), 337 עד 343	https://doi.org/10.1109/TIT.1977.1055714
17.	תקן DEFLATE, IETF RFC 1951	https://www.rfc-editor.org/rfc/rfc1951
18.	התיעוד הרשמי של Wireshark	https://www.wireshark.org/docs/
19.		

תיעוד הפרויקט

המערכת המורכבת

בניית מערכת הפעלה Embedded Linux ל DE10-Nano

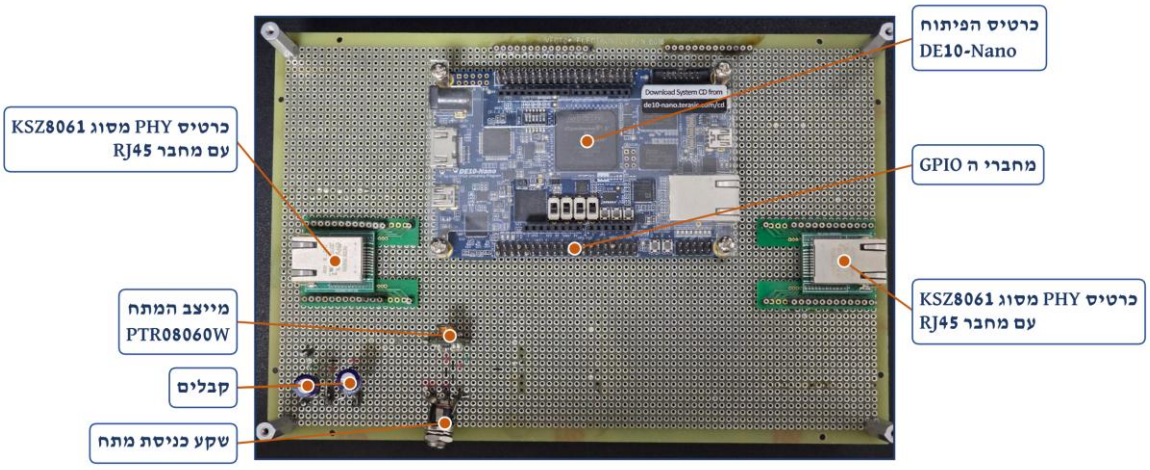
סביבת הבדיקות ואימות המערכת

תיעוד השימוש ב AI

המערכת המורכבת

בתמונה רואים את המערכת כמו שהיא בנויה על לוח מטריצה. בחלק העליון של הלוח נמצא כרטיס הפיתוח DE10-Nano, ומשני הצדדים, מתחתיו, נמצאים שני כרטיסי ה PHY מסוג KSZ8061, כל אחד עם מחבר RJ45 משלו.

בחלק התחתון של הלוח נמצא אזור אספקת המתח, עם המייצב PTR08060W, הקבלים ושקע כניסת המתח. החיבור בין כרטיס הפיתוח לבין כרטיסי ה PHY עובר דרך מחברי ה GPIO של הכרטיס.



איור 45: המערכת המורכבת

בניית מערכת הפעלה Embedded Linux ל DE10-Nano

Embedded Linux היא מערכת ההפעלה Linux שרצה על מעבד משובץ. בפרויקט זה היא רצה על ה HPS של כרטיס הפיתוח DE10-Nano, ומנהלת את המידע שעובר ברשת ואת התקשורת עם החומרה שב FPGA.

את מערכת ההפעלה הזאת לא לקחתי מוכנה. בניתי אותה בעצמי מאפס, חלק אחרי חלק.

למה הייתי צריך לבנות מערכת הפעלה בעצמי

כרטיס הפיתוח DE10-Nano מגיע עם מערכת הפעלה מוכנה בשם Angstrom. המערכת הזאת ישנה מאוד, והתמיכה שלה ברשת לא טובה. בפרויקט שכל כולו עוסק בתעבורת רשת, זו בעיה שאי אפשר לעבוד איתה.

בנוסף, רציתי ללמוד לעומק את הבקר עצמו. כדי להפעיל דברים בחומרה, לעבוד עם הגשרים שמחברים בין ה HPS לבין ה FPGA ולגשת לרכיבים בעזרת mmap, הייתי צריך שליטה מלאה על מערכת ההפעלה ועל ההגדרות שלה. לכן החלטתי לבנות בעצמי את מנהל האתחול, את ליבת המערכת ואת מערכת הקבצים.

שלושת החלקים של המערכת

מערכת Embedded Linux בנויה משלושה חלקים, שעולים אחד אחרי השני בזמן ההדלקה של הכרטיס:



איור 46: סדר העלייה של המערכת

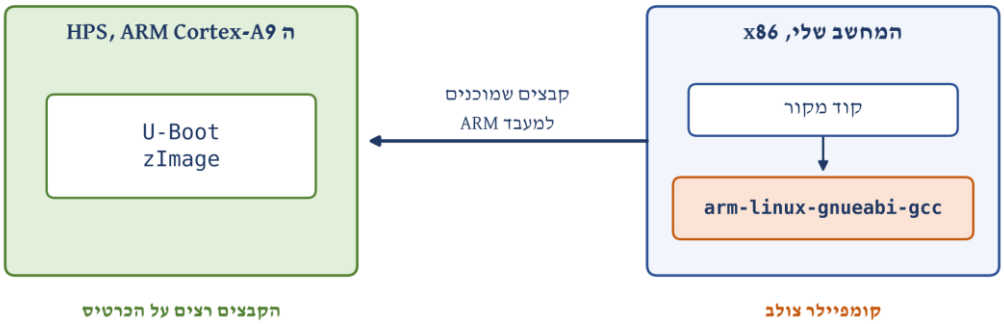
הראשון הוא U-Boot, מנהל האתחול. זו התוכנה הראשונה שרצה כשהכרטיס נדלק. היא מכינה את החומרה, טוענת את ה FPGA ומעבירה את השליטה לליבה.

אחריו עולה ה Kernel, ליבת המערכת, שמכירה את החומרה של הכרטיס ומנהלת את המעבד, הזיכרון, הרשת והחיבור ל FPGA.

החלק האחרון הוא ה RootFS, מערכת הקבצים. בה נמצאים כל הקבצים, התוכנות וההגדרות של מערכת ההפעלה, ואצלי זו מערכת Debian.

סביבת העבודה: Cross Compilation

את כל החלקים קימפלתי על המחשב שלי, שמבוסס על מעבד x86, אבל הם צריכים לרוץ על מעבד ARM של ה HPS. לכן הייתי צריך קומפיילר צולב, Cross Compiler, שמקמפל במחשב אחד קוד שירוץ על מעבד אחר.



איור 47: קימפול צולב

לפני כל קימפול הגדרתי שני משתני סביבה:

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabi-
```

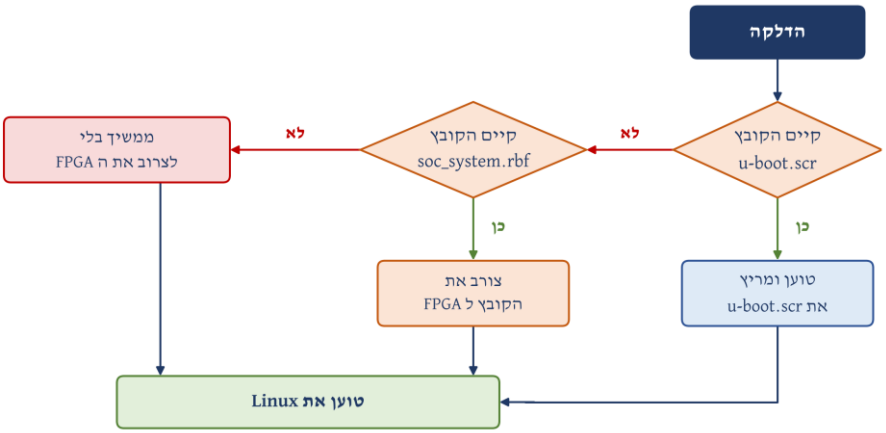
ARCH מגדיר שהבנייה היא עבור ארכיטקטורת ARM, ו CROSS_COMPILE מגדיר באיזה קומפיילר צולב להשתמש.

שלב 1: בניית U-Boot

את קוד המקור של U-Boot הורדתי מהמאגר הרשמי ועבדתי עם הגרסה היציבה v2021.07. כדי לשמור את השינויים שלי בנפרד מהקוד המקורי, יצרתי Branch חדש ועשיתי בו שני שינויים.

טעינה אוטומטית של ה FPGA בזמן האתחול

שיניתי את פקודת האתחול בקובץ config_distro_bootcmd.h, כך ש U-Boot יצרוב את ה FPGA בעצמו בכל הדלקה של הכרטיס. האתחול עובד לפי הסדר הזה:



איור 48: טעינת ה FPGA ב U-Boot

הקובץ soc_system.rbf הוא קובץ התכנון של ה FPGA.

כך התכנון שלי ב FPGA עולה בכל הדלקה, בלי שאצטרך לצרוב אותו ידנית מתוך Linux.

כתובת MAC קבועה

כברירת מחדל ה DE10-Nano מקבל כתובת MAC אקראית בכל הדלקה, ולכן גם כתובת ה IP שלו משתנה. זה מאוד הקשה עליי להתחבר לכרטיס דרך SSH.

יצרתי כתובת MAC חדשה בעזרת הכלי gen_eth_addr שנמצא בתוך U-Boot, והוספתי אותה להגדרות הסביבה בקובץ socfpga_common.h. מאז הכרטיס מקבל תמיד את אותה כתובת.

הגדרה וקימפול

את ההגדרות של U-Boot יצרתי מהקובץ המוכן socfpga_de10_nano_defconfig, ואחרי זה קימפלתי:

```
make ARCH=arm socfpga_de10_nano_defconfig
make ARCH=arm -j24
```

בסוף הקימפול נוצר הקובץ u-boot-with-spl.sfp. הקובץ כולל את U-Boot ואת ה SPL, Secondary Program Loader, וזה הקובץ שנצרב על כרטיס ה SD.

שלב 2: קימפול ליבת המערכת

את קוד המקור של הליבה הורדתי מהמאגר של Altera, linux-socfpga. זהו Fork של Linux, גרסה מותאמת שתומכת ברכיבי ה SoC FPGA. בחרתי בגרסה socfpga-6.7, כדי להימנע מתקלות ובעיות תאימות שהיו בגרסאות הישנות.

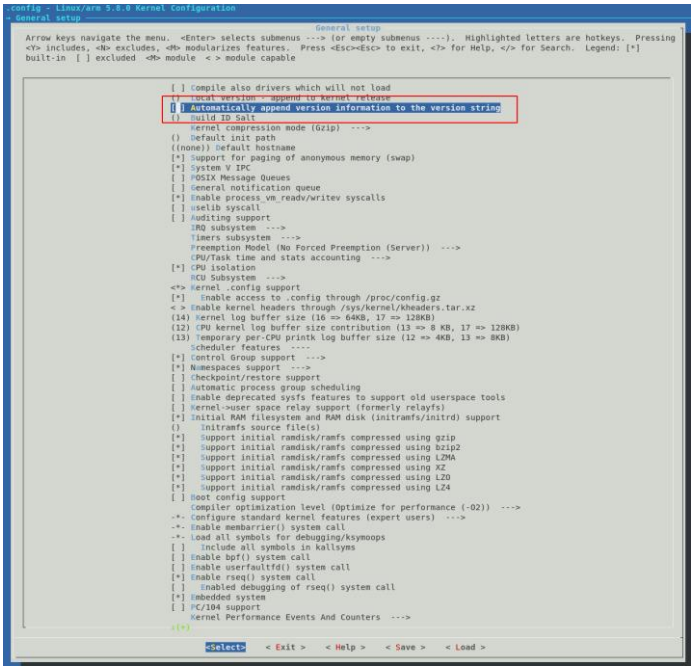
לפני ההורדה התקנתי על המחשב את כל הספריות שנדרשות לקימפול הליבה. אחרי זה יצרתי קובץ הגדרות בסיסי שמתאים לכרטיס:

```
make ARCH=arm socfpga_defconfig
```

שינויים בתפריט ההגדרות

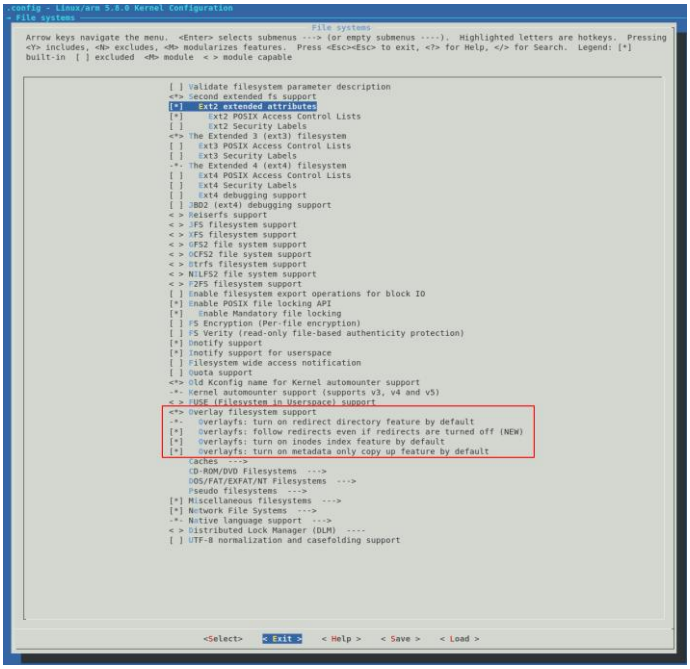
בתפריט ההגדרות של הליבה, menuconfig, עשיתי שלושה שינויים.

הראשון היה תחת General setup. ביטלתי שם את האפשרות שמוסיפה אוטומטית את מספר הגרסה לשם הליבה, כדי שיהיה לי קל יותר לנהל ולבדוק דרייברים.



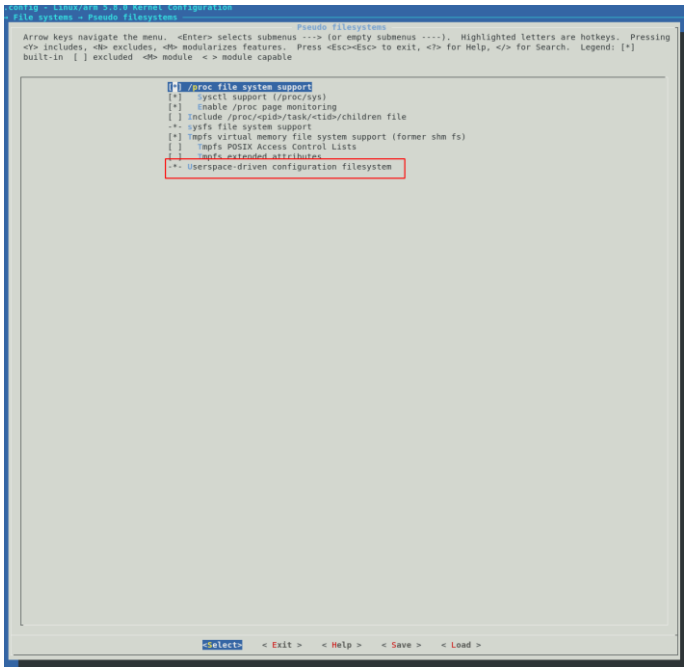
איור 49: ביטול הוספת גרסה אוטומטית

תחת File systems הפעלתי את Overlay filesystem support עם כל אפשרויות המשנה שלה. בלי האפשרות הזאת אי אפשר לצרוב את ה FPGA מתוך Linux בזמן ריצה.



איור 50: הפעלת Overlay filesystem

ובתוך Pseudo filesystems וידאתי שהאפשרות Userspace-driven configuration filesystem, שנקראת גם CONFIGFS, מופעלת.



איור 51: הפעלת CONFIGFS

קימפול

אחרי שתיקנתי את שגיאות הקוד שמתוארות בהמשך הפרק, קימפלתי את הליבה:

```
make ARCH=arm LOCALVERSION= zImage -j24
```

הקימפול לוקח בין 5 ל 10 דקות. בסופו נוצר הקובץ zImage, שהוא ליבת Linux דחוסה ומוכנה לשימוש.

שלב 3: בניית מערכת הקבצים RootFS

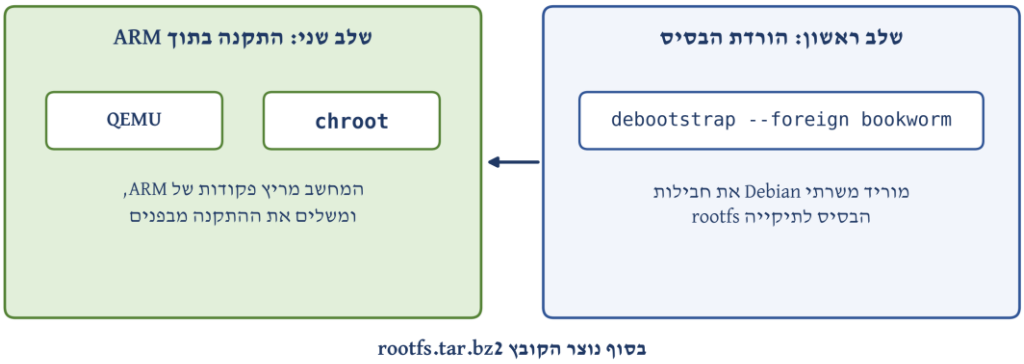
למערכת הקבצים בחרתי ב Debian, בגרסה Bookworm, Debian 12. היא נותנת סביבת עבודה מלאה על הכרטיס, שמרגישה כמו עבודה על מחשב Linux רגיל.

שקלתי גם את Buildroot ואת Yocto, שנמצאים בשימוש רחב בתעשייה. הם מתאימים למערכות סגורות, קטנות ומאוד ספציפיות, כמו נתב או ציוד תקשורת. הפרויקט שלי צריך מערכת כללית שנוח לפתח עליה ולהתקין בה חבילות, ולכן Debian התאימה יותר.

בניית הבסיס

את מערכת הקבצים בניתי בעזרת הכלי debootstrap. מכיוון שהמחשב שלי מבוסס x86 והמערכת נבנית עבור ARM, השתמשתי באמולטור QEMU, שמאפשר להריץ במחשב שלי פקודות של מעבד ARM.

הבנייה נעשתה בשני שלבים. בשלב הראשון הורדתי את חבילות הבסיס של Debian עבור ARM. בשלב השני נכנסתי לתוך מערכת הקבצים החדשה בעזרת chroot, והשלמתי את ההתקנה מתוך סביבת ה ARM המדומה.



איור 52: שני שלבי בניית ה RootFS

הגדרות המערכת

עוד לפני שהמערכת עלתה על הכרטיס, הגדרתי בה את הדברים הבסיסיים. נתתי לכרטיס את השם de10-nano ברשת, הגדרתי סיסמה למשתמש root, וקבעתי בקובץ fstab שהמחיצות ייטענו אוטומטית בכל הדלקה.

כדי שאוכל לראות את הודעות האתחול ולהתחבר לכרטיס בכבל, הפעלתי את החיבור הסריאלי. את הרשת הקווית הגדרתי כך שהכרטיס יקבל כתובת IP אוטומטית משרת DHCP ברגע שמחברים כבל, והתקנתי שרת SSH כדי להתחבר אליו גם דרך הרשת.

בסוף התקנתי כלי פיתוח שאני צריך בהמשך הפרויקט, כמו net-tools, build-essential ו device-tree-compiler, וארזתי את כל מערכת הקבצים לקובץ אחד, rootfs.tar.bz2, שמחולץ ישירות למחיצה של כרטיס ה SD.

תקלות שנתקלתי בהן ואיך פתרתי אותן

שגיאות Assembler בקימפול

בהתחלה הקומפיילר ניסה לקמפל את הליבה עבור המחשב שלי ולא עבור ARM, וקיבלתי שגיאות Assembler קשות. פתרתי את זה בעזרת הגדרת המשתנים ARCH ו CROSS_COMPILE לפני כל קימפול. כשהשגיאות חזרו, ניקיתי את כל קבצי הקימפול הקודמים עם make mrproper ויצרתי מחדש את קובץ ההגדרות.

שימוש ב sudo בזמן הקימפול

גיליתי שהרצה של sudo make משנה את משתני הסביבה, ואז הקומפיילר הצולב לא נמצא והקימפול נכשל. מאז אני מקמפל את U-Boot ואת הליבה תמיד בלי sudo.

שגיאות בקוד המקור של Altera

בגרסה של הליבה שבחרתי היו שני קבצים שגרמו לשגיאות קימפול. הראשון הוא דרייבר של מסך Newhaven שחסר במערכת, אז ביטלתי אותו בקובץ ה Makefile. השני הוא הדגל FBINFO_FLAG_DEFAULT בדרייבר הווידאו של Altera, שכבר לא נתמך בגרסאות החדשות של Linux, אז החלפתי אותו בערך 0. את שני התיקונים עשיתי עם פקודות sed, בלי לערוך את הקבצים ידנית.

תחביר בהגדרות של U-Boot

כשהוספתי את כתובת ה MAC להגדרות של U-Boot, למדתי שבסוף כל שורה חייבים להישאר שני תווים: התו 0\, שמסמן סוף של מחרוזת, והתו \, שממשיך את ההגדרה לשורה הבאה. כשמחקתי אחד מהם, הקימפול נכשל.

שם ממשק הרשת

בגרסאות חדשות של Linux שמות ממשקי הרשת השתנו. כשהחיבור לרשת לא עבד עם השם eth0, החלפתי אותו בהגדרות הרשת בשם end0.

עלייה איטית של שרת ה SSH

שרת ה SSH עלה לאט, כי למערכת לקח הרבה זמן ליצור מספרים אקראיים. התקנתי את הכלי haveged, שמאיץ את יצירת המספרים האקראיים, והעיכוב נעלם.

מה למדתי מבניית מערכת ההפעלה

כל אחת מהתקלות האלה לקחה זמן, ולפעמים ימים שלמים. בהתחלה ניסיתי לחפש פתרון מוכן לכל שגיאה, ועם הזמן הבנתי שזה לא עובד. רק כשהתחלתי לקרוא את הודעות השגיאה עד הסוף ולהבין מה כל פקודה עושה, הצלחתי להתקדם.

בזכות העבודה על מערכת ההפעלה הבנתי גם איך תוכנה וחומרה עובדות יחד על אותו שבב. תוכנית שרצה ב Linux על ה HPS מגיעה לרכיבים שב FPGA דרך גשר ה AXI וממשק ה Avalon MM, ואת החיבור הזה ראיתי בפועל כשכתבתי את תוכנית ה mmap הקצרה שמופיעה בפרק על Avalon MM.

קבצי המערכת לכרטיס ה SD

בסוף התהליך היו לי שלושה קבצים שמהם בניתי את כרטיס ה SD של המערכת:

טבלה 9: קבצי המערכת לכרטיס ה SD

הקובץ	מה הוא מכיל
u-boot-with-spl.sfp	מנהל האתחול U-Boot יחד עם ה SPL
zImage	ליבת Linux דחוסה, מותאמת לכרטיס DE10-Nano
rootfs.tar.bz2	מערכת הקבצים של Debian עם כל ההגדרות שלי

את הקבצים האלה חילקתי בין שלוש מחיצות בכרטיס ה microSD:



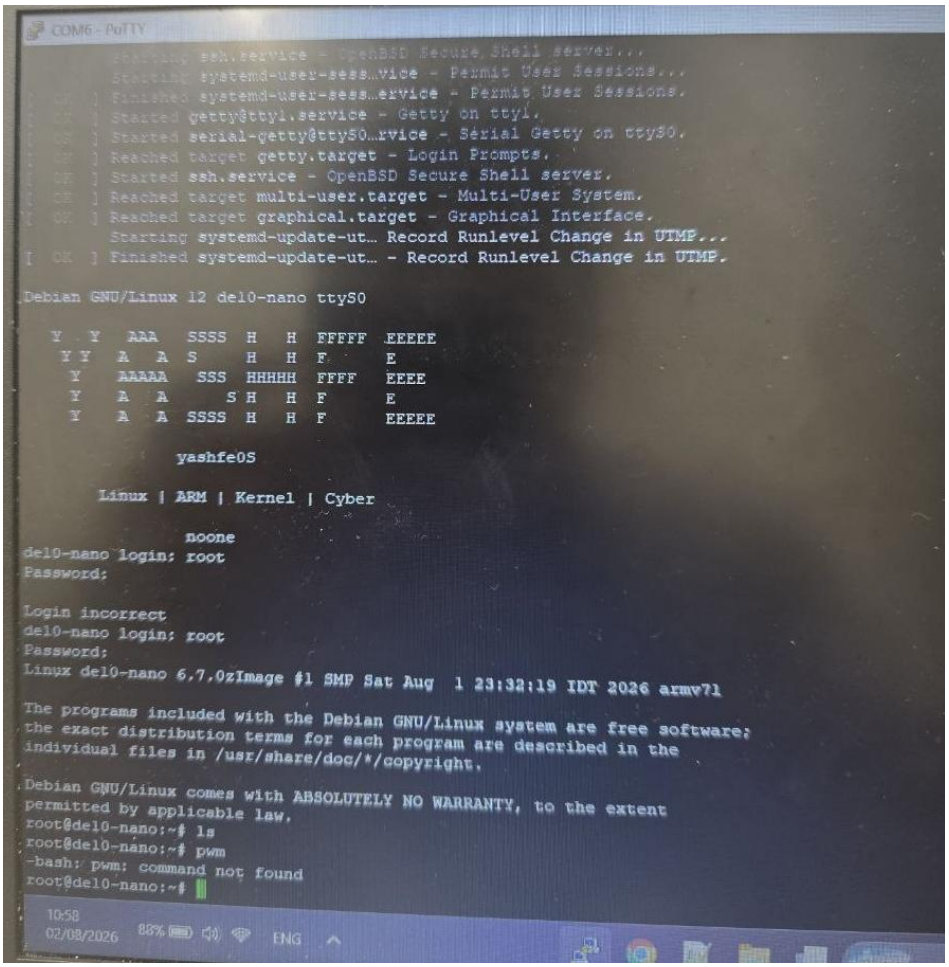
סדר העלייה: U-Boot מהמחיצה A2, אחריו הליבה וה FPGA מה FAT, ובסוף Linux עם מערכת הקבצים מה EXT4

איור 53: המחיצות בכרטיס ה microSD

בהדלקה הכרטיס מחפש את המחיצה מסוג A2 ומעלה ממנה את ה SPL ואת U-Boot. אחרי זה U-Boot טוען מהמחיצה הראשונה, מסוג FAT, את הקובץ soc_system.rbf וצורב אותו ל FPGA, ואז את zImage ואת ה Device Tree. בסוף Linux עולה ומשתמש במחיצה השנייה, מסוג EXT4, כמערכת הקבצים הראשית, כמו שהגדרתי בקובץ fstab.

המערכת עולה על הכרטיס

ככה נראה החיבור הסריאלי לכרטיס דרך PuTTY, אחרי שהמערכת עלתה בפעם הראשונה:



איור 54: המערכת עולה על הכרטיס

בחלק העליון רואים את סוף תהליך האתחול, כולל שרת ה SSH והחיבור הסריאלי ttyS0 שהגדרתי. מתחת מופיעה השורה Debian GNU/Linux 12 de10-nano, ואחריה הלוגו של מערכת ההפעלה שלי, yashfeOS.

אחרי ההתחברות כמשתמש root מופיעה גרסת הליבה 6.7.0zImage. זו בדיוק הליבה שקימפלתי מהענף socfpga-6.7, עם השם zImage שנתתי לה בפקודת הקימפול, והכיתוב armv7l מראה שהיא רצה על מעבד ה ARM של ה HPS. בניסיון הראשון גם טעיתי בסיסמה, כמו שאפשר לראות בתמונה.

את כל התהליך תיעדתי צעד אחרי צעד במאגר הפרויקט שלי ב GitHub, כמדריך שאפשר לבנות לפיו את המערכת מחדש.

סביבת הבדיקות ואימות המערכת

אחרי שהמערכת תהיה מחוברת, צריך לבדוק שהיא באמת עושה את מה שתכננתי. קודם צריך לוודא שהחבילות עוברות דרך הכרטיס משני הצדדים, ואחר כך שהדחיסה מקטינה את המידע ושהפתיחה מחזירה אותו בדיוק כמו שהיה. בפרק הזה אני מתאר את הכלים שבהם אשתמש ואת סדר הבדיקות.



בדיקת ping והעברת קובץ ב FTP בין השרת ללקוח

איור 55: סביבת הבדיקות

Wireshark

Wireshark היא תוכנה שלוכדת את כל החבילות שעוברות בכרטיס הרשת של המחשב ומציגה כל חבילה לפי השכבות שלה, מה Ethernet דרך ה IP וה TCP ועד המידע עצמו.

אני אריץ את Wireshark גם במחשב של השרת וגם במחשב של הלקוח. כך אפשר להשוות בין מה שיצא מצד אחד של הכרטיס לבין מה שהגיע לצד השני, ולראות בדיוק מה השתנה בדרך.

בדיקת מעבר עם ping

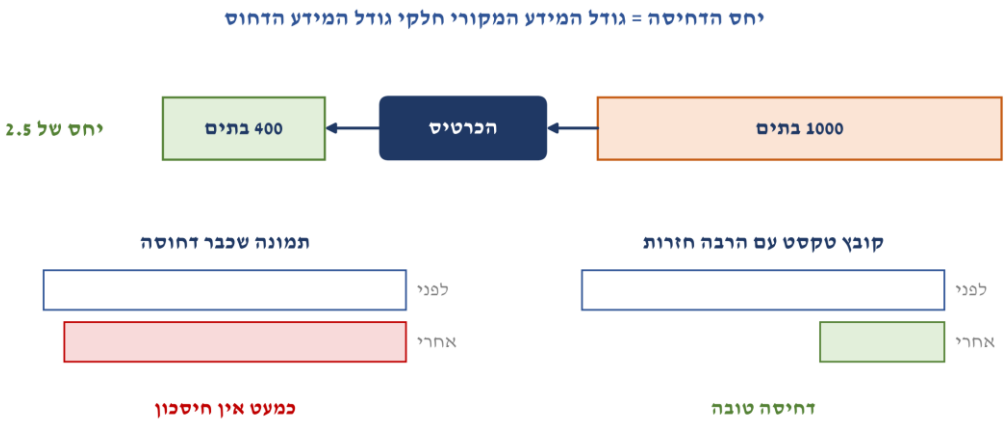
הבדיקה הראשונה היא שהכרטיס מעביר את התעבורה הרגילה של הרשת בלי לשנות אותה.

הפקודה ping שולחת הודעות ICMP Echo Request ומקבלת בחזרה Echo Reply. כך בודקים שגם חבילות IP עוברות דרך הכרטיס, ולפי הזמן שעובר בין הבקשה לתשובה אפשר לדעת כמה השהיה הכרטיס מוסיף.

מדידת יחס הדחיסה

כדי לבדוק את הדחיסה אשלח קובץ דרך הרשת ואלכוד את התעבורה בשני הצדדים של הכרטיס. בצד אחד אספור כמה בתים של מידע נכנסו לכרטיס, ובצד השני כמה בתים יצאו ממנו אחרי הדחיסה.

יחס הדחיסה הוא גודל המידע המקורי חלקי גודל המידע הדחוס. לדוגמה, אם נכנסו לכרטיס 1000 בתים ויצאו ממנו 400 בתים, יחס הדחיסה הוא 2.5.



איור 56: מדידת יחס הדחיסה

בצד של הלקוח בודקים שהקובץ שהגיע זהה לקובץ המקורי, כי הדחיסה צריכה להיות ללא איבוד מידע. את הבדיקה כדאי לעשות עם כמה סוגי קבצים. בקובץ טקסט עם הרבה חזרות הדחיסה תהיה טובה, ובקובץ שכבר דחוס, כמו תמונה, כמעט לא יהיה חיסכון.

תוכנית הבדיקות

טבלה 10: תוכנית הבדיקות

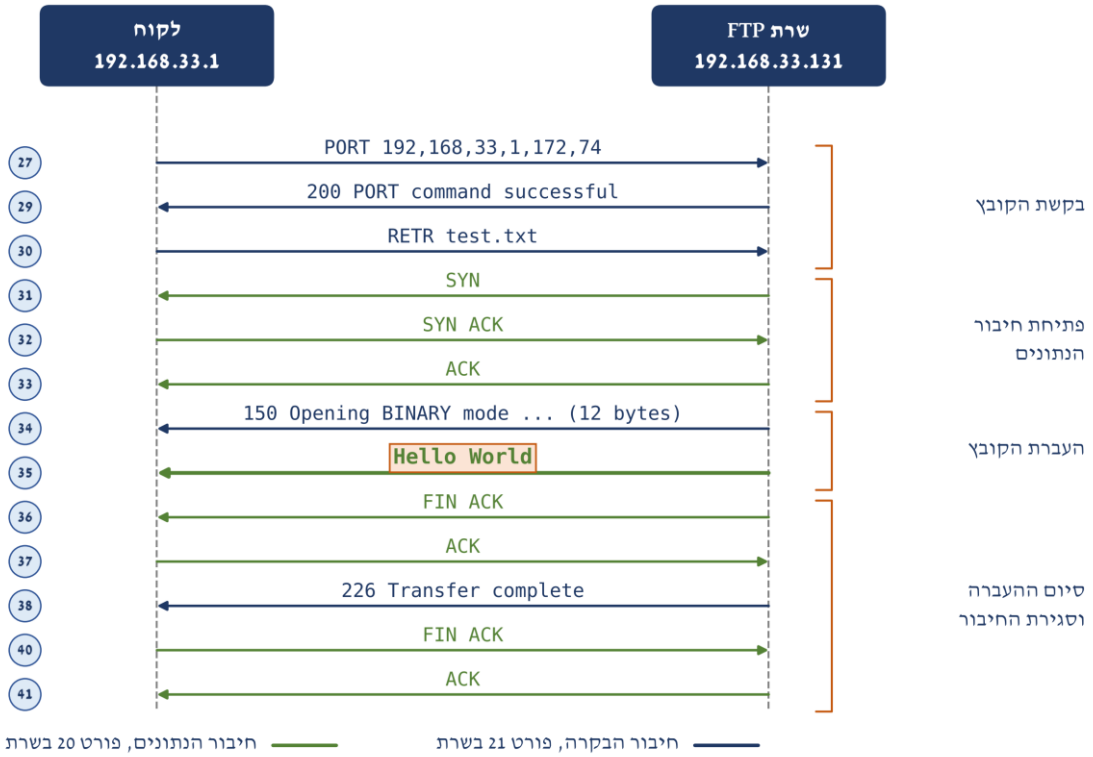
בדיקה	כלי	מה בודקים	תוצאה מצופה
מעבר IP	ping	חבילות ICMP עוברות בשני הכיוונים	תשובה לכל בקשה, בלי איבוד חבילות
העברת קובץ	FTP ו Wireshark	הקובץ מגיע שלם	הקובץ שהגיע זהה לקובץ המקורי
יחס דחיסה	Wireshark	כמות הבתים לפני הכרטיס ואחריו	פחות בתים בצד הדחוס

העברת קובץ ב FTP

את הקבצים אשלח דרך המערכת בפרוטוקול **FTP**, File Transfer Protocol, שמיועד להעברת קבצים ועובד מעל TCP. כדי להכיר את שלבי התקשורת שלו לפני שהיא תעבור דרך הכרטיס, ביצעתי העברה של קובץ ברשת וירטואלית של VMware ולכדתי אותה ב Wireshark.

השרת הוא מכונה וירטואלית בכתובת 192.168.33.131, והלקוח הוא המחשב שלי בכתובת 192.168.33.1. הקובץ שהעברתי נקרא test.txt ומכיל את הטקסט Hello World.

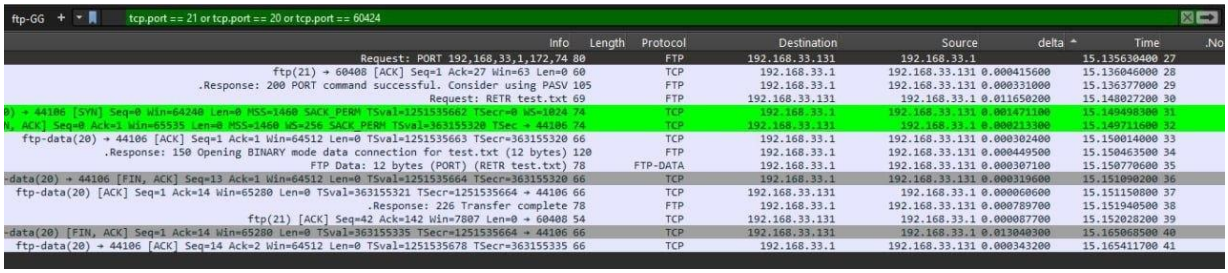
ב FTP יש שני חיבורי TCP נפרדים. בחיבור הבקרה, שעובר בפורט 21 של השרת, הלקוח שולח פקודות והשרת עונה עליהן בקודים של שלוש ספרות. חיבור הנתונים נפתח רק בשביל העברת הקובץ עצמו, ובלכידה שלי הוא יוצא מפורט 20 של השרת.



איור 57: שלבי התקשורת ב FTP

המספרים בעיגולים בתרשים הם מספרי החבילות בלכידה. הלכידה מתחילה אחרי ההתחברות לשרת, ולכן לא רואים בה את פקודות ה USER וה PASS.

כך נראית הלכידה עצמה ב Wireshark, אחרי סינון לפי הפורטים של ה FTP:



איור 58: הלכידה ב Wireshark

שלבי התקשורת

טבלה 11: שלבי התקשורת

חבילה	ההודעה	מה קורה
27	PORT 192168,33,1,172,74,	הלקוח מודיע לשרת לאיזו כתובת ולאיזה פורט לפתוח את חיבור הנתונים
29	200 PORT command successful	השרת מאשר את הפקודה
30	RETR test.txt	הלקוח מבקש להוריד את הקובץ test.txt
31 עד 33	SYN, SYN ACK, ACK	השרת פותח את חיבור הנתונים מפורט 20 אל פורט 44106 של הלקוח
34	150 Opening BINARY mode	השרת מודיע שהעברת הקובץ מתחילה ושגודל הקובץ 12 בתים
35	Hello World	תוכן הקובץ עובר בחיבור הנתונים
36 עד 37	FIN ACK, ACK	השרת סוגר את חיבור הנתונים
38	226 Transfer complete	השרת מודיע בחיבור הבקרה שההעברה הסתיימה בהצלחה
40 עד 41	FIN ACK, ACK	הלקוח סוגר את חיבור הנתונים מהצד שלו

הפורט בפקודת PORT

בפקודת PORT ארבעת המספרים הראשונים הם כתובת ה IP של הלקוח, 192.168.33.1, ושני המספרים האחרונים הם הפורט שבו הלקוח מחכה לחיבור הנתונים. את הפורט מחשבים כך:

$$172 \times 256 + 74 = 44106$$

ואכן בחבילה 31 השרת פותח את חיבור הנתונים בדיוק לפורט 44106.

בצורת העבודה הזו, שנקראת Active, השרת הוא זה שפותח את חיבור הנתונים אל הלקוח. בתשובה שלו השרת אפילו ממליץ להשתמש ב PASV, מצב שבו הלקוח פותח את חיבור הנתונים אל השרת.

הקודים 150 ו 226

הספרה הראשונה בקוד מראה את סוג התשובה. קוד שמתחיל ב 1 אומר שהפעולה התחילה ושיש עוד המשך, וקוד שמתחיל ב 2 אומר שהפעולה הסתיימה בהצלחה. לכן גם התשובה 200, שהשרת שלח אחרי פקודת PORT, מתחילה ב 2.

הקוד 150 נשלח מיד אחרי שחיבור הנתונים נפתח, והשרת מציין בו גם את גודל הקובץ, 12 בתים. הקוד 226 נשלח אחרי שהקובץ עבר והשרת סגר את חיבור הנתונים. בין שני הקודים האלה עובר הקובץ עצמו בחיבור הנתונים.

FTP-DATA

החבילה היחידה בלכידה שמכילה את המידע של הקובץ היא חבילה 35, ו Wireshark מסמן אותה כ FTP-DATA. ה payload שלה הוא 12 בתים, אחד עשר התווים של Hello World ותו סוף שורה, וזה בדיוק הגודל שהשרת הודיע עליו בקוד 150.

התוכן של חבילה 35, כל בית בכתיב הקסדצימלי

1	2	3	4	5	6	7	8	9	10	11	12
48	65	6c	6c	6f	20	57	6f	72	6c	64	0a
H	e	l	l	o	␣	W	o	r	l	d	\n

12 בתים, אחד עשר התווים של Hello World ותו סוף שורה

איור 59: תוכן חבילה 35

בהעברה הזו המידע לא נדחס, ולכן כמות הבתים שיצאה מהשרת שווה לכמות שהגיעה ללקוח. כשההעברה תעבור דרך המערכת, החבילה עם המידע היא זו שתידחס בדרך, וב Wireshark אפשר יהיה להשוות בין הגודל שלה לפני הכרטיס לבין הגודל שלה אחריו.

ARP בלכידה

בלכידה רואים גם את ARP בפעולה. בחבילה 24 המכונה הווירטואלית שולחת ARP Request ושואלת מי מחזיק בכתובת 192.168.33.2, ובחבילה 25 מגיע ה ARP Reply עם כתובת ה MAC המבוקשת, כמו שתיארתי בפרק על ARP.

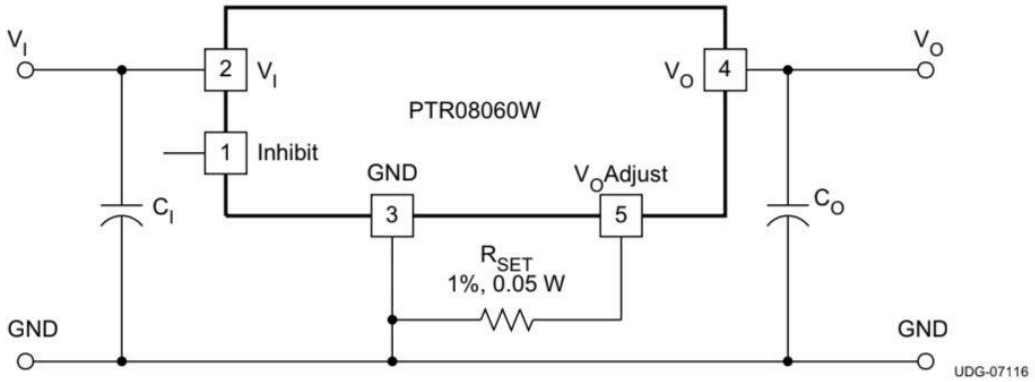
תיעוד השימוש ב AI

פרק זה יושלם בהגשה הסופית

תיעוד העבודה בתוכנה

פרק זה יושלם בהגשה הסופית

נספח א: תרשים חשמלי של מייצב המתח PTR08060W



איור 60: מייצב מתח PTR08060W

הסבר התרשים

המייצב PTR08060W של חברת Texas Instruments הוא מודול מתח שאחראי לספק מתח יציב של 3.3V לשני כרטיסי ה PHY. כרטיס הפיתוח DE10-Nano מקבל מתח של 5V ממתאם משלו, וחולק עם המייצב ועם כרטיסי ה PHY אדמה משותפת. המודול מקבל מתח כניסה בטווח של 4.5V עד 14V, ומוציא מתח מיוצב בטווח של 0.6V עד 5.5V בזרם של עד 6A.

כניסת מתח, רגל 2 VI

לרגל 2 נכנס מתח הכניסה של המערכת. בין הכניסה לבין ה GND מחובר הקבל CI של 100uF, שמסנן רעשים ושומר על מתח כניסה יציב.

הדלקה וכיבוי, רגל 1 Inhibit

רגל 1 משמשת להדלקה ולכיבוי של המודול. דרכה אפשר להפסיק את פעולת המייצב בלי לנתק את מתח הכניסה.

הארקה, רגל 3 GND

רגל 3 מחוברת להארקה המשותפת של המעגל. אליה מחוברים גם שני הקבלים וגם הנגד RSET.

קביעת מתח היציאה, רגל 5 VO Adjust

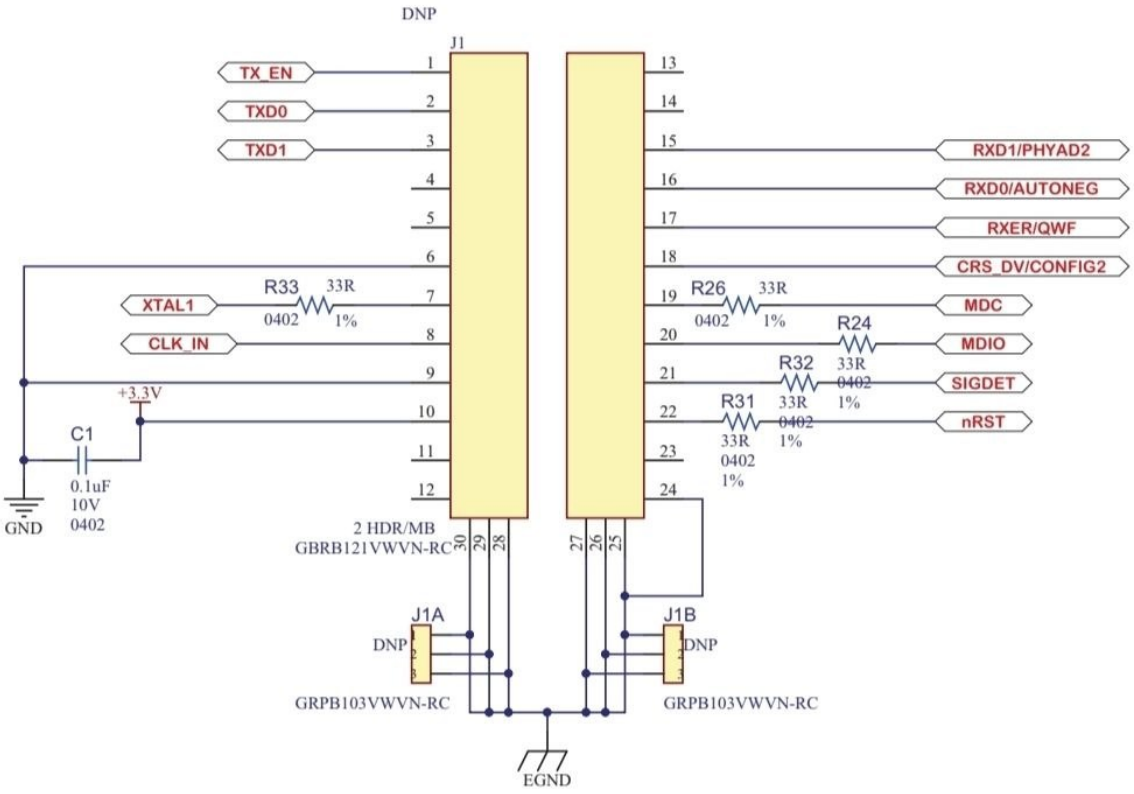
בין רגל 5 לבין ה GND מחובר הנגד RSET של 432 אוהם, נגד בדיוק של 1% ובהספק של 0.05W. ערך הנגד הזה קובע מה יהיה מתח היציאה של המייצב, כך שבעזרת נגד אחד אפשר להתאים את המתח לרכיבים במערכת.

יציאת מתח, רגל 4 VO

מרגל 4 יוצא המתח המיוצב אל שאר המערכת. גם ביציאה מחובר קבל, CO, שמחליק את המתח ושומר שלא יהיו קפיצות כשצריכת הזרם משתנה.

בנוסף, במייצב יש הגנה מזרם יתר והגנה ממתח כניסה נמוך מדי, כך שבמקרה של תקלה הוא מגן על עצמו ועל הרכיבים שמחוברים אליו. היעילות שלו מגיעה עד 96%, ורוב האנרגיה שנכנסת אליו מגיעה לרכיבים ולא הולכת לאיבוד כחום.

נספח ב: תרשים חשמלי של כרטיס PHY מסוג KSZ8061



איור 61: כרטיס PHY מסוג KSZ8061

הסבר התרשים

כרטיס ה PHY במערכת מבוסס על הרכיב KSZ8061 של חברת Microchip. זהו רכיב Ethernet שעובד במהירויות של 10 ו 100Mbps, מתחבר לכבל הרשת דרך מחבר ה RJ45, ומתחבר לכרטיס הפיתוח דרך ממשק RMII. הרכיב עובד במתח של 3.3V.

בתרשים רואים את המחבר J1, שדרכו עוברים כל האותות בין ה PHY לבין ה MAC שממומש ב FPGA.

קווי השידור, רגליים 1 עד 3

ברגליים 1, 2 ו 3 של המחבר נמצאים קווי השידור של ממשק ה RMII, **TX_EN**, **TXD0** ו **TXD1**. דרך TXD0 ו TXD1 ה MAC מעביר אל ה PHY שני ביטים בכל מחזור שעון, ו TX_EN מסמן מתי המידע בקווים תקף ויש לשדר אותו.

קווי הקליטה, רגליים 15 עד 18

ברגליים 15 עד 18 נמצאים קווי הקליטה, **RXD0**, **RXD1**, **CRS_DV** ו **RXER**. דרך RXD0 ו RXD1 ה PHY מעביר אל ה MAC את המידע שהתקבל מהכבל, CRS_DV מודיע שהנתונים בקווים תקפים, ו RXER מסמן שהייתה שגיאה בקליטה.

לחלק מהרגליים האלה יש שני שמות, כמו RXD0/AUTONEG או CRS_DV/CONFIG2. בזמן ההדלקה של הרכיב הן משמשות להגדרות ההתחלתיות של ה PHY, ואחרי זה הן עובדות כקווי מידע רגילים.

שעון, רגליים 7 ו 8

ברגליים 7 ו 8 נמצאים קווי השעון, **XTAL1** ו **CLK_IN**. ממשק RMII עובד עם שעון ייחוס של 50MHz, וכל העברת הנתונים בין ה MAC לבין ה PHY מסונכרנת לפיו.

ניהול ובקרה, רגליים 19 עד 22

דרך **MDC** ו **MDIO** ה MAC יכול לקרוא ולכתוב לרגיסטרים של ה PHY, לבדוק את מצב החיבור ולשנות הגדרות. MDC הוא קו השעון של הניהול ו MDIO הוא קו הנתונים.

SIGDET מציין שה PHY מזהה אות בכבל, ו **nRST** משמש לאיפוס של הרכיב.

מתח והארקה

רגל 10 מקבלת מתח של 3.3V, ורגליים 6 ו 9 מחוברות להארקה. בין ה 3.3V לבין ה GND מחובר הקבל C1 של 0.1uF, שמסנן רעשים במתח שמגיע לרכיב.

רגליים 25 עד 30 של המחבר, יחד עם המחברים J1A ו J1B, מחוברות להארקה EGND.

נגדים טוריים של 33 אוהם

על רוב קווי השעון והבקרה מחוברים נגדים טוריים של 33 אוהם, R32, R24, R26, R33 ו R31. הנגדים האלה מחליקים את האות ומונעים החזרות והפרעות בקווים המהירים.

נספח ג: דפי נתונים

פרק זה יושלם בהגשה הסופית